

**CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA
SOUZA DA ETEC DA ZONA LESTE
Mtec Desenvolvimento de Sistemas AMS**

**Murillo Castro De Jesus
Rafael Ferreira Lopes
Vinicius Rafael Rodrigues
Vitor Daisuke Iwamoto**

**EYE TRUCK: Sistema de Prevenção de Colisão com Viadutos e
Pontes**

**São Paulo
2025**

Murillo Castro De Jesus
Rafael Ferreira Lopes
Vinicius Rafael Rodrigues
Vitor Daisuke Iwamoto

EYE TRUCK: Sistema de Prevenção de Colisão com Viadutos e Pontes

Trabalho de Conclusão de Curso apresentado ao Curso Ensino Médio com Habilitação Profissional de Técnico em Desenvolvimento de Sistemas AMS da Etec Zona Leste, orientado pelo Professor Jeferson Roberto de Lima, como requisito parcial para obtenção do título de técnico em Desenvolvimento de Sistemas.

São Paulo
2025

Resumo

Esta monografia apresenta o EyeTruck, um sistema embarcado de apoio à prevenção de colisões de caminhões com viadutos e pontes por excesso de altura. Os objetivos desse projeto são detectar a sinalização R-15, extrair a altura máxima permitida e comparar com a altura cadastrada do veículo para alertar o condutor. A metodologia aborda pesquisa quantitativa, de caráter documental, utilizando dados secundários como notícias e relatórios setoriais para contextualização do problema e testes em ambientes controlados com o protótipo para validação funcional. A problemática envolve acidentes por excesso de altura que ocorrem recorrentemente, gerando riscos e custos à infraestrutura, como apontado por levantamentos públicos e setoriais. As principais conclusões mostram que em ambiente controlado, o protótipo identificou a placa R-15 com alta confiança, extraiu corretamente o valor de altura e emitiu alertas quando o limite foi excedido, indicando viabilidade técnica. A aplicação em cenários reais depende de ampliar a base de imagens e de testes de campo.

Palavras-chave: Sistema embarcado; Altura máxima permitida; Reconhecimento de placas; Prevenção de colisões; Veículos de carga.

Abstract

This technical report presents EyeTruck, an embedded system designed to support the prevention of truck collisions with viaducts and bridges due to excess height. The objectives of this project are to detect R-15 signs, extract the maximum permitted height, and compare it with the registered height of the vehicle in order to warn the driver. The methodology involves quantitative, documentary research, using secondary data such as news reports and sectoral studies to contextualize the problem, as well as tests in controlled environments with the prototype for functional validation. The issue addressed involves height-related accidents that occur recurrently, generating risks and costs for infrastructure, as indicated by public and sectoral surveys. The main findings show that, in a controlled environment, the prototype identified the R-15 sign with high confidence, correctly extracted the height value, and issued alerts when the limit was exceeded, indicating technical feasibility. Application in real-world scenarios depends on expanding the image database and conducting field tests.

Keywords: Internet of Things (IoT); Maximum allowed height; Sign recognition; Collision prevention; Cargo vehicles.

LISTA DE ILUSTRAÇÕES

Figura 1 - Raspberry PI 3, Modelo B	12
Figura 2 - Fluxo de um sistema baseado em visão computacional	13
Figura 3 - Trecho do código que realiza a captura da face.....	14
Figura 4 - Imagem da face alinhada com marcação nos olhos	15
Figura 5 - Imagem da região de interesse.....	15
Figura 6 - Exemplo da tecnologia YOLO em ambiente urbano	16
Figura 7 - Abordagem tradicional	19
Figura 8 - Abordagem do aprendizado de máquina	19
Figura 9 - Adaptando-se automaticamente à mudança	20
Figura 10 - Exemplo do código em Python.....	21
Figura 11 - Resultado do código em Python.....	22
Figura 12 - Outro resultado do código em Python	22
Figura 13 - Funcionamento do Firebase.....	24
Figura 14 - Explicação de um Dado	25
Figura 15 - Instalação da biblioteca EasyOCR.....	27
Figura 16 - Exemplo de código em JavaScript.....	30
Figura 17 – Exemplo de Caso de Uso.....	32
Figura 18 - Diagrama de atividade Fazer Login	33
Figura 19 – Diagrama de sequência Fazer Login.....	34
Figura 20 - Diagrama de máquina de estado Fazer Login	35
Figura 21 - Diagrama de Casos de Uso	39
Figura 22 - Diagrama de Atividade: Cadastrar Conta	54
Figura 23 - Diagrama de Atividade: Fazer Login	55
Figura 24 - Diagrama de Atividade: Consultar Perfil.....	56
Figura 25 - Diagrama de Atividade: Alterar Informações Do Perfil	57
Figura 26 - Diagrama de Atividade: Enviar Altura Do Caminhão	58
Figura 27 - Diagrama de Atividade: Reconhecer Placa	59
Figura 28 - Diagrama de Atividade: Retirar Altura	60
Figura 29 - Diagrama de Atividade: Comparar Altura	61
Figura 30 - Diagrama de Atividade: Exibir Alerta	62
Figura 31 - Diagrama de Atividade: Conectar Com o Bando de Dados	63
Figura 32 - Diagrama de Sequência: Cadastrar Conta	64
Figura 33 - Diagrama de Sequência: Fazer Login.....	65

Figura 34 - Diagrama de Sequência: Consultar Perfil	66
Figura 35 - Diagrama de Sequência: Alterar Informações do Perfil.....	67
Figura 36 - Diagrama de Sequência: Enviar Altura Do Caminhão.....	68
Figura 37 - Diagrama de Sequência: Reconhecer Placa.....	69
Figura 38 - Diagrama de Sequência: Retirar Altura	70
Figura 39 - Diagrama de Sequência: Comparar Altura.....	71
Figura 40 - Diagrama de Sequência: Exibir Alerta.....	72
Figura 41 - Diagrama de Sequência: Conectar Com o Banco de Dados	73
Figura 42 - Diagrama de Máquina de Estado: Cadastrar Conta.....	74
Figura 43 - Diagrama de Máquina de Estado: Fazer Login	75
Figura 44 - Diagrama de Máquina de Estado: Consultar Perfil	76
Figura 45 - Diagrama de Máquina de Estado: Alterar Informações do Perfil.....	77
Figura 46 - Diagrama de Máquina de Estado: Enviar Altura do Caminhão	78
Figura 47 - Diagrama de Máquina de Estado: Reconhecer Placa.....	79
Figura 48 - Diagrama de Máquina de Estado: Retirar Altura	80
Figura 49 - Diagrama de Máquina de Estado: Comparar Altura	81
Figura 50 - Diagrama de Máquina de Estado: Exibir Alerta.....	82
Figura 51 - Diagrama de Máquina de Estado: Conectar Com o Banco de Dados	83
Figura 52 - Wireframe de Baixa Fidelidade: Tela de carregamento	85
Figura 53 - Wireframe de Alta Fidelidade: Tela de carregamento.....	86
Figura 54 - Wireframe de Baixa Fidelidade: Tela de início	87
Figura 55 - Wireframe de Alta Fidelidade: Tela de início	88
Figura 56 - Wireframe de Baixa Fidelidade: Tela de encontrar dispositivo	89
Figura 57 - Wireframe de Alta Fidelidade: Tela de encontrar dispositivo	90
Figura 58 - Wireframe de Baixa Fidelidade: Tela de cadastro	91
Figura 59 - Wireframe de Alta Fidelidade: Tela de cadastro	92
Figura 60 - Wireframe de Baixa Fidelidade: Tela de login	93
Figura 61 - Wireframe de Alta Fidelidade: Tela de login	94
Figura 62 - Wireframe de Baixa Fidelidade: Tela da home	95
Figura 63 - Wireframe de Alta Fidelidade: Tela home	96
Figura 64 - Wireframe de Baixa Fidelidade: Tela de editar altura	97
Figura 65 - Wireframe de Alta Fidelidade: Tela de editar altura	98
Figura 66 - Wireframe de Baixa Fidelidade: Tela de editar perfil	99
Figura 67 - Wireframe de Alta Fidelidade: Tela de editar perfil	100

Figura 68 - Conjunto de imagens anotadas para treinamento do modelo	101
Figura 69 – Treinamento do modelo.....	102
Figura 70 - Detecção da placa e valor extraído	103
Figura 71 - Alerta visual exibido ao motorista	103
Figura 72 - Trecho de código responsável pela comparação das alturas.....	104
Figura 73 - Carregador portátil responsável pela alimentação elétrica	105
Figura 74 - Visão de cima da case	106

LISTA DE QUADROS

Tabela 1 - Documentação do Caso de Uso: Fazer Login	43
Tabela 2 - Documentação do Caso de Uso: Cadastrar Conta.....	44
Tabela 3 - Documentação do Caso de Uso: Enviar Altura do Caminhão.....	45
Tabela 4 - Documentação do Caso de Uso: Consultar Perfil.....	46
Tabela 5 - Documentação do Caso de Uso: Alterar informações do Perfil	47
Tabela 6 - Documentação do Caso de Uso: Conectar com o Banco de Dados	49
Tabela 7 - Documentação do Caso de Uso: Exibir Alerta	50
Tabela 8 - Documentação do Caso de Uso: Comparar Altura	51
Tabela 9 - Documentação do Caso de Uso: Retirar Altura	52
Tabela 10 - Documentação do Caso de Uso: Reconhecer Placa.....	53

LISTA DE ABREVIATURAS E SIGLAS

Cascading Style Sheet (CSS)
Create, Read, Update e Delete (CRUD).
Deep Learning (DL)
HyperText Markup Language (HTML)
Internet of Things (IoT)
Node Package Manager (NPM)
Non-Uniform Rational B-Splines (NURBS)
Open Source Computer Vision Library (OpenCV)
Optical Character Recognition (OCR)
Package Installer for Python (PIP)
Rede Neural Artificial (RNA)
Regra de Negócio (RN)
Requisito Funcional (RF)
Requisito Não Funcional (RNF)
Unified Modeling Language (UML)
Universal Serial Bus (USB)
Veículo Aéreo Não Tripulado (VANT)
You Only Look Once (YOLO)

SUMÁRIO

1	INTRODUÇÃO.....	8
2	REFERENCIAL TEÓRICO.....	10
2.1	Acidentes envolvendo caminhões e viadutos	10
2.2	Tecnologias Utilizadas	10
2.2.1	Internet of Things (IoT)	11
2.2.2	Raspberry Pi.....	12
2.2.3	Visão Computacional.....	13
2.2.4	OpenCV (Open Source Computer Vision Library)	14
2.2.5	YOLO (You Only Look Once).....	16
2.2.6	Aprendizado de Máquina	18
2.2.7	Python	21
2.2.8	Firebase.....	23
2.2.9	EasyOCR.....	26
2.2.10	Package Installer for Python (PIP).....	27
2.2.11	React Native	28
2.2.12	CSS (Cascading Style Sheet).....	29
2.2.13	JavaScript.....	30
2.2.14	Node.JS.....	31
2.2.15	UML (Unified Modeling Language)	32
2.2.16	Wireframe	36
2.2.17	Modelagem 3D	37
3	DESENVOLVIMENTO	38
3.1	Diagrama de Caso de Uso.....	38
3.1.1	Documentação dos Casos de Uso	40
3.2	Diagrama de Atividade.....	54
3.3	Diagrama de Sequência	64
3.4	Diagrama de Máquina de Estado	74
3.5	Prototipação das telas do aplicativo	84
3.6	Desenvolvimento do hardware de detecção de placas.....	100
3.7	Desenvolvimento da case.....	106
4	CONSIDERAÇÕES FINAIS.....	107
	REFERÊNCIAS.....	109

1 INTRODUÇÃO

O Eyetruck é um dispositivo para prevenção de colisões de veículos de carga com viadutos e pontes, por meio do reconhecimento das placas R-15, que indicam a altura máxima permitida.

Para cumprir esse objetivo, o sistema é composto por hardware, câmera e uma aplicação móvel, sendo capaz de comparar a altura máxima informada na sinalização com a altura do caminhão, inserida previamente pelo motorista. Dessa forma, caso haja risco de colisão, alertas serão emitidos ao condutor, contribuindo para a segurança viária.

A problemática que motivou a realização deste trabalho está relacionada à recorrência de acidentes e bloqueios de vias provocados por veículos de carga que ultrapassam a altura máxima permitida em trechos específicos. Embora exista uma lei que obriga a existência de sinalização indicando a altura máxima em viadutos e pontes, esses métodos não têm sido eficientes o bastante para evitar os acontecimentos. Além disso, pode-se observar que as estruturas citadas apresentam variações nas alturas, o que leva a situações em que o veículo consegue passar por uma via, mas não por outra.

Inicialmente, acreditava-se que a principal causa do problema fosse a falta de sinalização para alertar o motorista. No entanto, foi descoberto que é obrigatória a existência de sinalizações especificando a altura máxima permitida dos veículos, mas certas variáveis, como a falta de padronização no tamanho das estruturas, a desatenção e a falta de familiaridade dos motoristas com determinadas vias, contribuem para que os acidentes continuem ocorrendo.

Diante desse cenário, o desenvolvimento do EyeTruck justifica-se pela necessidade de um outro método de prevenção de acidentes, pela redução de custos relacionados a danos na infraestrutura viária e pela segurança tanto para o motorista do caminhão quanto para os demais motoristas, assim, contribuindo para uma viagem mais segura.

Quanto ao escopo, este projeto está delimitado à cidade de São Paulo e à sua região metropolitana, utilizadas como estudo de caso, devido ao tráfego intenso de veículos de grande porte e ao elevado número de infraestruturas, como viadutos e

pontes, visando atender cidades com acesso à internet e grande concentração de estruturas que contenham restrições de altura.

Por fim, esta monografia tem como objetivo apresentar o desenvolvimento do EyeTruck, um sistema de prevenção de colisões de caminhões com viadutos e pontes que integra um aplicativo móvel, um dispositivo embarcado baseado em Raspberry Pi, técnicas de visão computacional, detecção de objetos com YOLO, reconhecimento óptico de caracteres e armazenamento de dados em nuvem.

2 REFERENCIAL TEÓRICO

Neste capítulo serão apresentados os principais conceitos, tecnologias e uma visão geral da problemática, tendo em vista melhor entendimento do contexto em que este trabalho foi desenvolvido.

2.1 Acidentes envolvendo caminhões e viadutos

Segundo notícia publicada pela Folha de S. Paulo (2019), ao menos 15 caminhões por mês ficam bloqueados por ultrapassarem a altura permitida em pontes e viadutos. Entre janeiro de 2017 e outubro de 2018, foram registrados 213 casos desse tipo de ocorrência, que geraram riscos a outros condutores de veículos e danos às infraestruturas de transportes.

Dados recentes reforçam a magnitude do problema. Segundo análise da Pamcary (2025), em 2024 a taxa de sinistros envolvendo caminhões nas rodovias brasileiras foi 4,5 vezes superior à de furtos de carga, e 36% desses episódios envolveram terceiros. Esses indicadores evidenciam o potencial de dano humano e material associado a ocorrências com veículos de grande porte, justificando iniciativas tecnológicas voltadas à prevenção de colisões por excesso de altura, como o EyeTruck.

Com relação a isso, de acordo com a R7 (2019), a cada dois dias acontecia um acidente envolvendo caminhões com excesso de altura que não conseguiram passar por viadutos e pontes na capital paulista, sendo que esses dados foram tirados de um levantamento feito pela CET (Companhia de Engenharia de Tráfego) revelando que de janeiro a setembro foram 157 ocorrências. Apenas o mês de setembro foram 22 casos, enfatizando que além dos danos à estrutura, geravam problemas no tráfego com congestionamento e enormes filas de carros.

2.2 Tecnologias Utilizadas

A seguir, serão explicadas as tecnologias utilizadas no desenvolvimento do projeto Eye Truck, abordando linguagens de programação, componentes eletrônicos e conceitos utilizados.

2.2.1 Internet of Things (IoT)

De acordo com Santos (2019), a Internet das Coisas é uma conexão de dispositivos que se comunicam entre si com a finalidade de realizar tarefas de forma autônoma, sem a necessidade de interação humana.

Carrion e Quaresma (2019) destacam que o cientista da computação e inventor John Romkey criou uma torradeira que podia ser ligada e desligada pela internet, sendo considerada o primeiro de muitos dispositivos IoT.

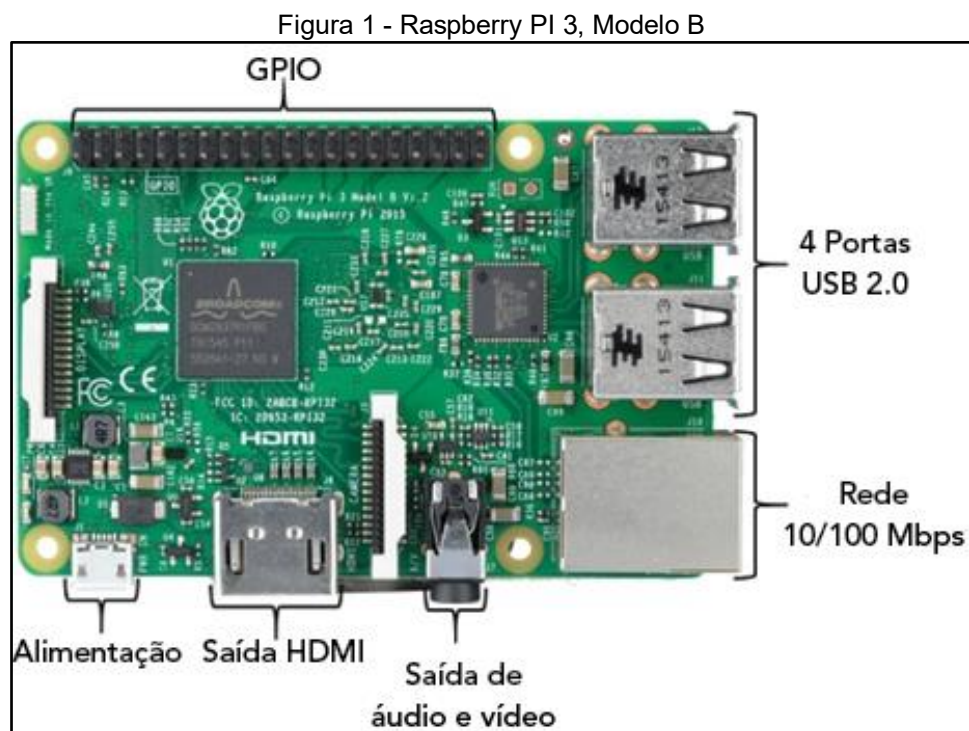
Diante disso, percebe-se que a Internet das Coisas é muito mais do que apenas conectar um dispositivo à internet. Seu objetivo é tornar as “coisas” inteligentes, sendo capazes de aproveitar a conexão e utilizar os dados coletados do ambiente ou das redes às quais estão conectadas (OLIVEIRA, 2021).

2.2.2 Raspberry PI

Em 2006, Eben Upton, Rob Mullins, Jack Lang e Alan Mycroft criaram um computador portátil e acessível para crianças na Universidade de Cambridge, na Inglaterra. A iniciativa surgiu após a percepção de que os estudantes que se inscreviam para participar do laboratório de Ciências da Computação apresentavam pouco conhecimento prévio sobre linguagens de programação e arquitetura dos computadores quando comparado à dos alunos de 1990. Assim, o projeto teve como objetivo ajudar jovens a adquirirem conhecimentos de *software* e *hardware*. (EBERMAM et al. 2017).

Em 2012, a empresa Raspberry PI, fundada por Eben e seus colegas, criou um microcomputador de baixo custo com o mesmo nome. Esse pequeno dispositivo é um ótimo meio para a introdução aos conceitos de Eletrônica, podendo ser utilizado tanto por novatos na área da tecnologia quanto por especialistas nas áreas de Computação. (OLIVEIRA; NABARRO; ZANETTI, 2018).

Conforme Silva, Dias e Escudero (2022), o Raspberry PI possui diferentes modelos disponíveis. A principal diferença entre eles está na sua capacidade computacional. Alguns exemplos para melhor entendimento seriam a potência e o tipo do processador, quantidade de entradas USB (*Universal Serial Bus*), tecnologias implementadas, entre outros.



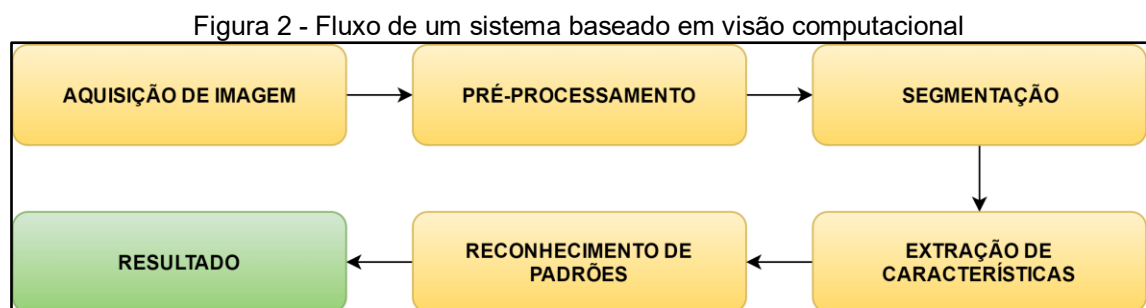
Fonte: Oliveira, Nabarro, Zanetti, 2018.

2.2.3 Visão Computacional

Segundo Barelli (2018) Visão Computacional é o estudo de máquinas que possam reconhecer elementos de um ambiente, através de imagens capturadas, sensores e outras tecnologias, essas informações podem ser extraídas e manipuladas pelo desenvolvedor conforme seus objetivos.

A visão computacional é um tema muito recente no mundo tecnológico, com sua primeira aparição em 1955, com Selfridge. Na década de 70 começaram as pesquisas de visão computacional com a inteligência artificial, os cientistas acreditavam que em breve teriam o sentido completo da visão das máquinas, atrasando-se em decorrência da complexidade que o tema possuía na época para a ciência, surgindo pela falta de informações biológicas de como nosso cérebro consegue interpretar um ambiente ou objeto (MILANO; HONORATO, 2014).

De acordo com Barelli (2018) os sistemas de visão computacional, por mais que pareçam complexos, seguem um fluxo em comum, como mostrado no exemplo a seguir.



Fonte: Barelli, 2018, p. 18.

Visão Computacional não é somente limitado ao controle de qualidade, esta tecnologia pode ser usada em diversas áreas do mundo, desde agricultura até sistemas de reconhecimento autônomo, um desses exemplos é o "Alvin" um robô produzido por Dean Pomerleau, um carro que podia adquirir conhecimento através da experiência, ou seja, ele reconhecia as ruas e "memorizava" elas. (MILANO; HONORATO, 2014).

2.2.4 OpenCV (Open Source Computer Vision Library)

OpenCV é uma biblioteca multiplataforma feita pela Intel que divulgou sua criação por volta dos anos 2000, desenvolvida especialmente para o carregamento de imagens e estruturação de dados em aplicações, considerado um grande aliado da visão computacional o OpenCV, por ser de código aberto (Open Source), ou seja, é disponível para uso do público. (BARELLI, 2018).

Consoante a Marengoni e Stringhini (2010), o OpenCV é uma biblioteca gigante com diversas funções disponíveis de acesso gratuito, é dividido em cinco principais grupos, que são: Manipulação e tratamento de imagens, análise da estrutura de cenas, monitoramento de movimento e rastreamento de objetos, identificação de padrões e ajuste de câmera com reconstrução em 3D. A ideia do projeto foi criar um projeto que facilitasse o acesso dos desenvolvedores a tecnologias de visão computacional.

Agora iremos exibir um exemplo que consiga identificar, capturar e salvar uma imagem de um rosto.

Figura 3 - Trecho do código que realiza a captura da face

```
cam = cv2.VideoCapture(1)
cam.set(3, 640) # set video width
cam.set(4, 480) # set video height

face_detector = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')
```

Fonte: Sette, Carvalho, Basile, 2021, p. 5.

No exemplo do código, dois métodos principais da biblioteca OpenCV são usados para executar o reconhecimento facial. O primeiro, `cv2.VideoCapture()`, ativa a câmera do dispositivo, permitindo a captura de imagens em tempo real. O segundo, `cv2.CascadeClassifier`, aplica modelos de detecção baseados em classificadores em cascata já treinados, para identificar os rostos nas imagens capturadas. Aí também estão algumas outras funções, como `cv2.cvtColor`, para converter as imagens em tons de cinza, e `cv2.imwrite`, que salva os arquivos gerados, que também são frequentes ao longo do código (SETTE; CARVALHO; BASILE, 2021).

Figura 4 - Imagem da face alinhada com marcação nos olhos



Fonte: Sette, Carvalho, Basile, 2021, p. 6.

Nesta imagem o OpenCV está destacando o ponto de interesse, ou seja, ele descarta o que não é relevante para o desenvolvedor e para o sistema para facilitar a extração de dados e informações úteis com mais facilidade.

Figura 5 - Imagem da região de interesse



Fonte: Sette, Carvalho, Basile, 2021, p. 6.

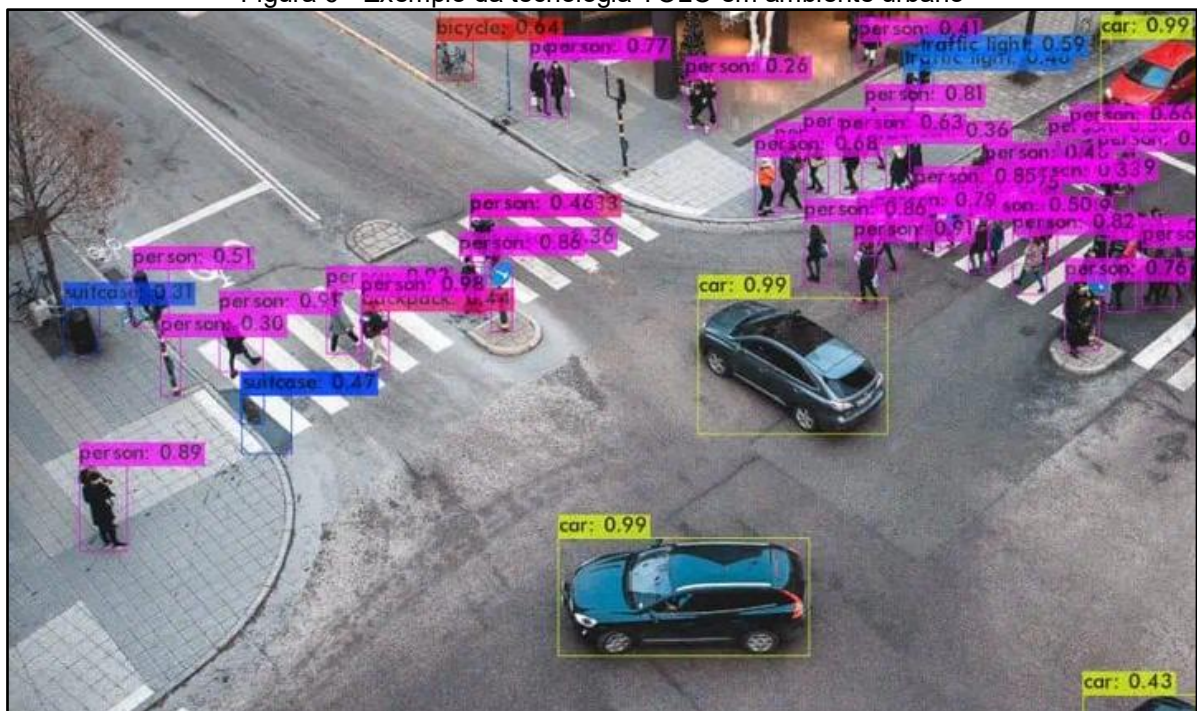
Por fim ele separou a região de interesse do resto da imagem utilizando as bibliotecas do OpenCV para o desenvolvedor poder manipulá-la da forma que quiser, como por exemplo, reconhecimento por íris ou photoshop. (SETTE; CARVALHO; BASILE, 2021).

2.2.5 YOLO (You Only Look Once)

YOLO (You Only Look Once) é conhecido por suas práticas com redes neurais, é um modelo de detecção de objetos criado por Joseph Redmon e Ali Farhadi na universidade de Washington em 2015. Seu diferencial dentre as outras tecnologias de detecção de objetos é sua velocidade para a detecção, porém pelo YOLO olhar somente uma vez para a imagem acontece uma perda de acurácia de acordo com Araújo (2022).

Conforme Silva (2024), YOLO cria grids, que são divisões na imagem, para que dentro de cada grid exista uma caixa delimitadora (bounding box) para armazenar a probabilidade de existir um objeto naquela região.

Figura 6 - Exemplo da tecnologia YOLO em ambiente urbano



Fonte: Ultralytics, 2023.

Cada Bounding box tem um ponto de confiança que vai indicar se existe ou não a chance de existir um objeto na região, caso a probabilidade seja muito baixa a bounding box vai desaparecer consoante ao Leocádio et al. (2021).

Concomitante ao Pacheco (2018), o YOLO vem de uma Rede Neural Artificial (RNA), o RNA surgiu por meio de um modelo de matemática do neurônio biológico realizado por McCulloch e Pitts na data de 1943. No RNA é feito diversas camadas que

agem de forma analógica a um neurônio realizando um processamento de uma parte da informação total, essas camadas são: camada de entrada, camadas intermediárias ou ocultas, e camada de saída.

A camada de entrada irá inicialmente receber os primeiros dados da rede, logo depois na camada intermediária irá acontecer o maior número de processamento e por último, dentro da camada de saída, acontecerá a conclusão que é exibida de acordo com Kovács (2023).

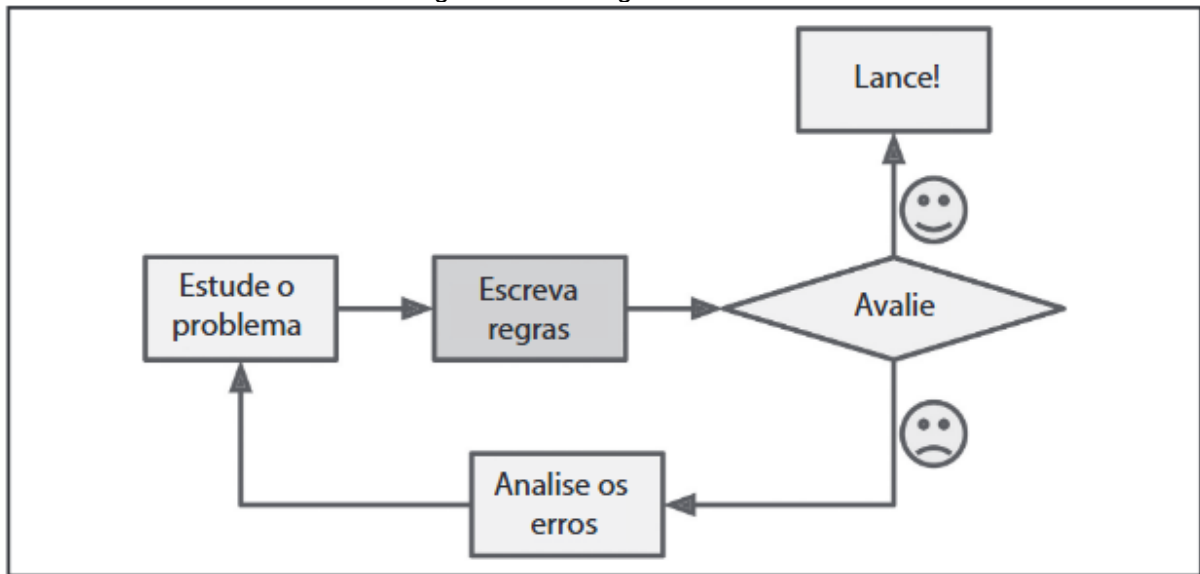
Segundo Pereira (2017), Deep Learning (DL) também chamado de Aprendizado Profundo envolve RNA, o DL foi criado como exemplo para analisar e assimilar arquiteturas que têm quantidades maiores de dados, com isso ele é capaz de criar exemplos parciais que são assimilados durante o treinamento, assim ocupando a necessidade do pré-processamento e extração de recursos manualmente designados.

2.2.6 Aprendizado de Máquina

Segundo Almeida (2019), quando pensamos no tópico “aprendizado de máquina” pensamos em um robô que irá dominar o mundo ou irá fazer todas as nossas tarefas do dia a dia, porém isso já existe desde 1990 quando foi criado o “filtro de spam”, se pararmos para notar os pequenos detalhes como “recomendados” entendemos que o aprendizado de máquina já existe em diversas tecnologias. Para resumir em poucas palavras, o aprendizado de máquina é uma arte ou ciência que permite uma máquina ser treinada e possa aprender com dados.

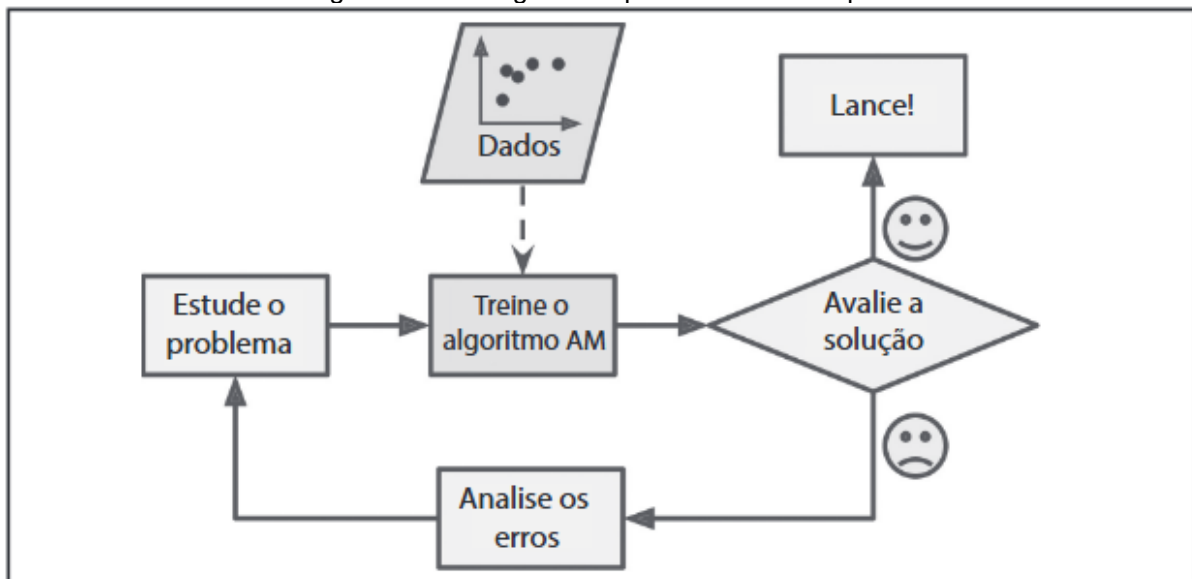
Uma das principais vantagens de se utilizar o Aprendizado de Máquina é sua tecnologia que permite se adaptar, por exemplo, vamos pensar que você esteja criando um programa de “filtro de spam”, caso você fosse criar em uma abordagem comum (sem o Aprendizado de Máquina), você teria que criar diversas regras de forma que quando chegasse um e-mail “Oferta imperdível” ele fosse para a caixa de spam pelas palavras, até o momento seu programa funcionaria, porém se o e-mail passasse a chegar como “Esse desconto só para você”, seria inviável conforme Santos (2019).

Figura 7 - Abordagem tradicional



Fonte: Géron, 2020, p. 4.

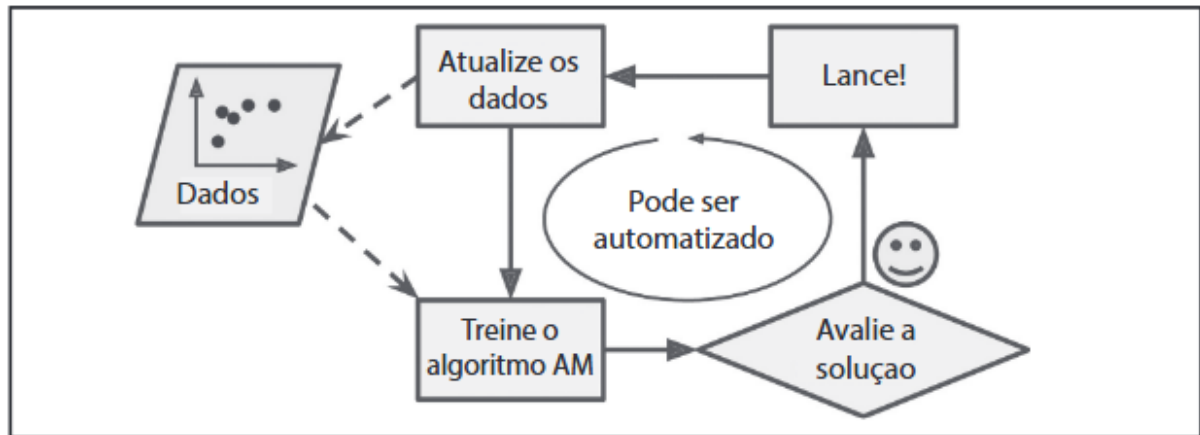
Figura 8 - Abordagem do aprendizado de máquina



Fonte: Géron, 2020, p. 5.

Consoante ao Géron (2020), notamos a diferença entre os tipos de abordagem, na (Figura 2) o caso do spam seria diferente, caso o aprendizado de máquina percebesse que “Esse desconto só para você” se tornou um email frequente na caixa de spam dos outros usuários, automaticamente ele começa a redirecionar todos os próximos e-mails para a caixa de spam. Aqui está um exemplo de como funcionaria essa atualização de dados para que fosse funcional:

Figura 9 - Adaptando-se automaticamente à mudança



Fonte: Géron, 2020, p. 5.

2.2.7 Python

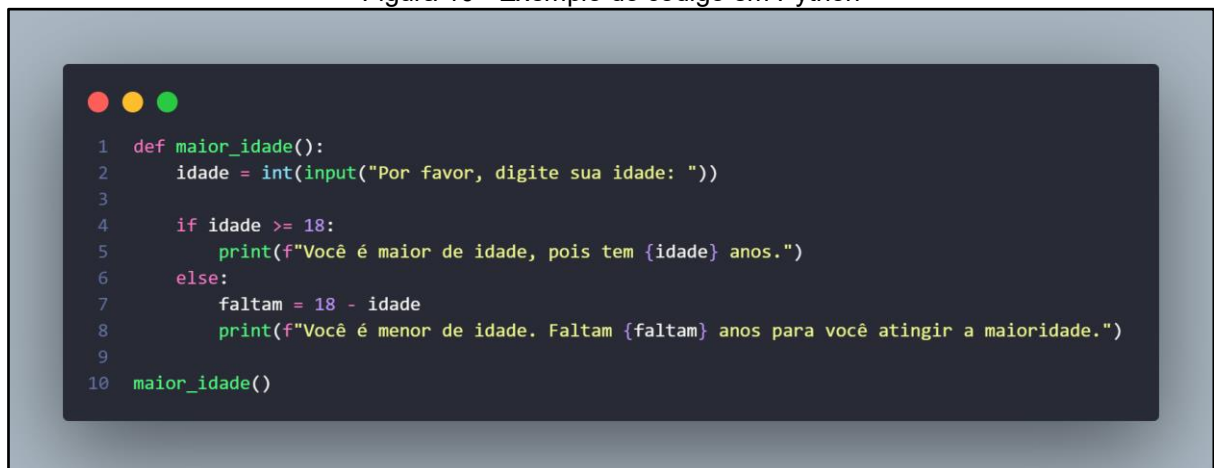
O Python é uma linguagem de fácil acesso e consolidada no mercado, isso graças as empresas envolvidas em sua criação, podendo ser usada em diversos sistemas operacionais como Linux, Windows, Mac OS; e por sua alta flexibilidade podendo ser utilizada em outras áreas da programação como por exemplo: Inteligência Artificial, aplicações mobile e até mesmo animações 3D como destaca Nilo Menezes (2010).

Segundo Matthes (2016), Python tem uma grande performance diante de sua simplicidade, com uma sintaxe objetiva e legível, grande versatilidade em uso geral, fácil manutenção e alta concentração de pacotes para o auxílio de desenvolvedores, sendo capaz de realizar desde pequenos códigos até grandes aplicações.

Nos últimos 20 anos, ele passou de uma linguagem de computação científica inovadora [...] para uma das mais importantes linguagens para ciência de dados, machine learning e desenvolvimento de softwares em geral no ambiente acadêmico e no setor empresarial. (McKinney, 2023, p.18)

Python consolidou seu espaço na indústria global, sendo aplicada em grandes empresas, como Google, Yahoo, Microsoft, Nokia e Disney para aplicações web, desenvolvimento de softwares para celulares e animações 3D, como destaca Borges (2014).

Figura 10 - Exemplo do código em Python



```
1 def maior_idade():
2     idade = int(input("Por favor, digite sua idade: "))
3
4     if idade >= 18:
5         print(f"Você é maior de idade, pois tem {idade} anos.")
6     else:
7         faltam = 18 - idade
8         print(f"Você é menor de idade. Faltam {faltam} anos para você atingir a maioridade.")
9
10 maior_idade()
```

Fonte: Autoria Própria, 2025.

O código acima é um exemplo de uma função básica em Python que realiza uma verificação para saber se a idade digitada pelo usuário indica se ele é maior de idade ou não. A seguir, uma breve explicação do código:

Linha 1: Cria uma função denominada maior_idade.

Linha 2: Utiliza a função input() para o usuário digitar sua idade e retornar como string. Através da função int(), ele transforma esse valor em um número inteiro, que será armazenado dentro da variável idade.

Linha 4: Usa o comando de decisão if para verificar se o valor armazenado na variável idade é maior ou igual a 18 por meio do operador de comparação “maior ou igual a” (>=) .

Linha 5: Aplica a função print() para aparecer a mensagem no console. O f fora das aspas aponta que é uma f-string, que possibilita adicionar variáveis no texto, como a variável idade adicionada na mensagem.

Linha 6: Caso a condição idade >= 18 for falsa, emprega-se outro comando de decisão, o else, que executará outro código.

Linha 7: Calcula quantos anos faltam para o usuário completar 18 anos por meio de uma subtração, 18 – idade, que é armazenada na variável faltam.

Linha 8: Exibe uma mensagem no console informando que o usuário ainda é menor de idade e mostrando quantos anos faltam para atingir a maior idade, através da variável faltam.

Linha 10: Chama a função maior_idade para que o código seja executado.

Figura 11 - Resultado do código em Python

```
Por favor, digite sua idade: 25
Você é maior de idade, pois tem 25 anos.
```

Fonte: Autoria Própria, 2025.

Figura 12 - Outro resultado do código em Python

```
Por favor, digite sua idade: 15
Você é menor de idade. Faltam 3 anos para você atingir a maioridade.
```

Fonte: Autoria Própria.

Acima, há uma demonstração de dois exemplos do retorno do código em Python, nos quais foram necessários fornecer a idade do usuário. Foi possível fazer uma verificação por meio de um comando de decisão e do uso de um operador de comparação para indicar se ele era maior de idade ou não.

2.2.8 Firebase

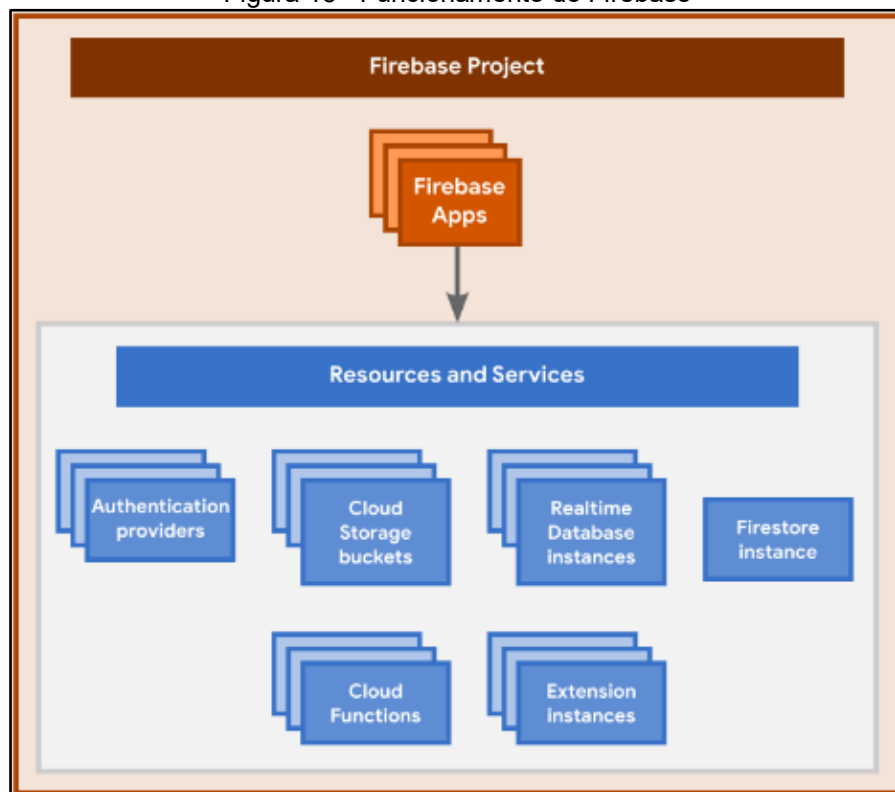
Gonçalves (2014) explica que um banco de dados pode ser entendido como um conjunto de informações relacionadas, que podem ser armazenadas tanto em um computador quanto, atualmente, em serviços de nuvem. Esses dados são organizados em linhas e colunas, estruturados em tabelas criadas dentro do banco, existe neste meio de dados uma forma de descrever as 4 principais formas de manipular as informações chamada CRUD (Create, Read, Update e Delete), o Create como o próprio nome diz, cria/insere uma nova informação no banco, seguido do Read que é listar esses dados inseridos, em seguida o Update é poder atualizar/alterar essas informações a qualquer momento e por fim o Delete é a função que exclui os dados inseridos no banco.

De acordo com Carvalho (2015), o banco de dados influencia em todas as etapas de um projeto: desde a fase inicial de criação até o desempenho do sistema e sua manutenção ao longo do tempo. Por esse motivo, contar com uma base de dados bem estruturada e confiável é essencial para o sucesso de qualquer aplicação.

Machado (2021) reforça essa ideia ao afirmar que todo software moderno depende de uma boa organização e de uma manutenção adequada dos dados. Nesse contexto, o Google disponibiliza o Firebase, uma plataforma robusta que oferece diversas ferramentas, escalabilidade, respostas rápidas às consultas, flexibilidade de armazenamento e alta disponibilidade.

Dentro de um projeto no Firebase, é possível registrar um ou mais aplicativos, todos com os mesmos acessos e recursos compartilhados, funcionando de forma integrada. A plataforma disponibiliza diferentes planos de uso, como o Spark e o Blaze, que são planos pagos da plataforma e variam conforme as limitações e necessidades de cada projeto (GOOGLE, 2025).

Figura 13 - Funcionamento do Firebase



Fonte: Google, 2025.

2.2.8.1 Cloud Firestore

O Firestore é um banco de dados NoSQL que possibilita aos usuários armazenar e sincronizar informações tanto no servidor quanto no cliente. Ele pode ser aplicado em aplicativos da web, móveis e de servidor, garantindo a integração e a sincronização entre diferentes aplicativos. As informações são organizadas em grupos de documentos que se alocam no banco. (MATIOLA, 2023).

Conforme a Google (2025), o Cloud Firestore é hospedado na nuvem, assim podendo ser acessado juntamente com os aplicativos da Apple, Android e da Web com um conjunto de ferramentas disponibilizado pela empresa chamado SDKs, trabalhando com o NoSQL do Cloud Firestore que armazena dados em blocos de documentos, assim mapeando valores para ajudar o usuário a se organizar e desenvolver consultas.

Figura 14 - Explicação de um Dado



Fonte: GOOGLE (2025).

Em suma, o Firestore é um banco de dados flexível e escalável que estrutura dados em coleções e documentos de forma hierárquica. Ele permite consultas rápidas e eficazes, além de sincronização em tempo real entre cliente e servidor, assegurando desempenho e integração constante (FREITAS, 2023).

2.2.9 EasyOCR

Com o objetivo de facilitar o processo de desenvolvimento de modelos de aprendizado de máquina, o Facebook lançou em 2016 o PyTorch, uma biblioteca de código aberto baseada na linguagem de programação Python, conforme Florencio (2020).

De acordo com Souza (2024), a sua eficácia em construir modelos dinâmicos, possibilita um processo de desenvolvimento eficiente, flexível e altamente otimizado para o uso com GPUs, permitindo que o usuário tenha uma melhor performance utilizando a potência máxima do hardware para treinar seus modelos de forma rápida.

Através dessa tecnologia, foi possível desenvolver a biblioteca EasyOCR, responsável pelo reconhecimento óptico de caracteres, empregada na extração de textos presentes em imagens ou documentos, atualmente conseguindo reconhecer mais de 80 idiomas, segundo Cordeiro (2024).

2.2.10 Package Installer for Python (PIP)

Em conformidade com o escritor Matthes (2016), o Package Installer for Python também conhecido como (PIP), é uma ferramenta crucial para o gerenciamento de pacotes que possui as principais funcionalidades de instalar, atualizar e desinstalar bibliotecas no Python.

A instrução abaixo indica como realizar a instalação da biblioteca EasyOCR e suas dependências para a linguagem de programação Python:

Figura 15 - Instalação da biblioteca EasyOCR

A screenshot of a terminal window with a dark background. The text 'pip install easyocr' is displayed in a light blue, monospaced font. The 'pip' is in a slightly larger font size than the rest of the command.

```
pip install easyocr
```

Fonte: Autoria Própria, 2025.

2.2.11 React Native

O React Native foi desenvolvido em 2013 durante um Hackathon interno do Facebook, evento conhecido como uma maratona de programação que envolve equipes de diversos setores para solucionar um problema em um determinado período de tempo. Após o evento, a empresa identificou o potencial da ideia e começou a desenvolvê-la. A primeira aparição pública ocorreu em 2015, na React.js Conference. Meses depois, em uma conferência realizada pelo Facebook, conhecida como F8, foi anunciado que o React Native seria um projeto de código aberto, disponível por meio do repositório no GitHub (Escudelario e Pinho, 2020).

De acordo com Galvão (2018), o React Native é uma tecnologia flexível, podendo ser utilizada em dispositivos Android e iOS. Ele funciona em uma arquitetura com três camadas, a principal executada no dispositivo da aplicação, a segunda, executada em uma máquina virtual com o JavaScript e a terceira, servindo de ponte entre as outras duas, as quais nunca se comunicam diretamente.

Para a criação, implementação e testes em dispositivos móveis utilizaremos o Expo, um conjunto de serviços e ferramentas para auxiliar no desenvolvimento de aplicações em React, com foco principal voltado para React Native, sendo possível empregar JavaScript em seu ambiente, tirando a necessidade de apenas utilizar os recursos da plataforma nativa, conforme Falcão (2022).

De acordo com Coelho e Antunes (2023), o framework Expo simplifica a configuração do ambiente e oferece diversas funcionalidades, com recursos que colaboram para a eficiência do processo de desenvolvimento, possibilitando testar e depurar a aplicação em um dispositivo móvel em tempo real por meio do recurso Expo Go.

2.2.12 CSS (Cascading Style Sheet)

Conforme Eis (2015), a linguagem de estilo CSS, foi criada em 1994, para o auxílio de formatar documentos em HTML (HyperText Markup Language), desenvolvidas por Bert Boss e Håkon Wium Lie, sua principal diferença é a aptidão da herança de estilos, sendo capaz de reutilizar o mesmo estilo em diversos elementos do código.

Também conhecida como Cascading Style Sheets, sua utilização serve para especificar a formatação e aparência do documento, ganhando destaque por sua manutenção de fácil atualização, com a separação do conteúdo de HTML para apresentação CSS, conforme relato por Albino (2023).

Como destaca Silva (2007), com a finalidade de tirar qualquer formatação do documento HTML para apresentação web, o CSS veio para ajustar as fontes, cores, posicionamento, tamanhos e todo o visual da página, são inúmeras vantagens, entre elas, a redução de carga da página web, agilidade na manutenção e controle completo da apresentação do site através de um arquivo central.

2.2.13 JavaScript

Nas palavras de Iepson (2018), o JavaScript foi inventado em meados de 1995 pela Netscape para ser uma linguagem que fosse universal nos navegadores da época, pois mais ou menos neste tempo havia um conflito de linguagens que funcionavam em certos navegadores e em outros não, o JavaScript chegou para apaziguar a situação.

Figura 16 - Exemplo de código em JavaScript

```
<script>  
  console.log("Hello, World!");  
</script>
```

Fonte: Autoria Própria, 2025.

A história do JavaScript é atual comparado as outras linguagens, de acordo com Silva e Sobral (2017), a linguagem nasceu na década de 90 com a intenção de ser uma nova forma de codificar Front-end (aparência) de páginas web. Com sua rápida ascensão no mercado competitivo, popularidade e sua alta adaptabilidade os desenvolvedores da época começaram a utilizar a linguagem na área de back-end (Funcionalidades), pois era mais prático o programador utilizar somente 1 linguagem para as 2 funções do que várias para chegarem no mesmo objetivo.

Conforme afirma o autor Flanagan (2012), o JavaScript é a linguagem mais utilizada nos sites e dispositivos modernos de hoje em dia, tornando-a uma das linguagens mais usadas da história, esse tipo de código pertence a uma base de linguagens que todo programador deve conhecer, que são: HTML, CSS e JS.

2.2.14 Node.JS

Na década de 2000 os computadores estavam com cada vez mais falhas de sistema e frequentes travamentos, era cada vez mais necessário um upgrade nas peças dos servidores. Como era algo com um alto custo, Ryan Dahl teve uma ideia que amenizasse este problema sem gastos exacerbados, com ajuda inicial de 14 colaboradores eles criaram o Node.js (PEREIRA, 2014),

Segundo Ihrig (2020), o Node.js foi desenvolvido principalmente como uma ferramenta para programar aplicações web que possam suportar muitos usuários ao mesmo tempo. Sua alta velocidade de execução faz com que o Node.js seja particularmente apropriado para sistemas que exigem respostas rápidas e processamento em tempo real.

Silva (2025) evidencia que o node.js é um grande fator para que o JavaScript esteja entre as linguagens mais aplicadas e confiáveis pelos programadores do mundo hoje em dia, possibilitando o uso flexível do front-end e o back-end de maneira eficiente.

Juntamente com o Node.js, acompanha o gerenciador de pacotes conhecido como Node Package Manager (NPM), que permite instalar, atualizar e remover módulos de forma simples, além de armazenar um grande acervo de pacotes e bibliotecas de código aberto em JavaScript, facilitando o gerenciamento de dependências e otimizando o desenvolvimento de projetos ao integrar funcionalidades fornecidas pela comunidade, segundo Santos (2023).

2.2.15 UML (Unified Modeling Language)

Segundo Guedes (2018), a UML é uma linguagem visual de modelagem que auxilia na compreensão do sistema, definindo os requisitos fundamentais para seu funcionamento e manutenção, sendo adotada como norma-padrão internacionalmente.

Conforme Booch (2006), a UML, assim como todas as outras linguagens, exerce a função de estabelecer um vocabulário e regras para a comunicação entre os envolvidos no desenvolvimento do projeto, possibilitando a representação conceitual e física dos elementos que compõem o sistema.

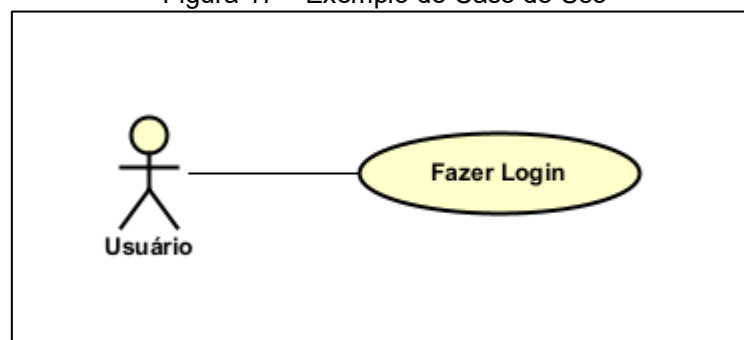
Fowler (2005) destaca que, mesmo a UML não possuindo o mesmo rigor dos métodos matemáticos, sua aplicabilidade a torna valiosa no cotidiano dos programadores. Ela é relevante mesmo sem ser totalmente formal ou detalhada.

Uma casa não se constrói com apenas uma planta, ela é dividida em várias plantas que mostram diversas partes da casa para uma melhor visualização. Devido a isso, existem 13 diagramas no total que podem ser utilizados para descrever o funcionamento tanto estrutural quanto lógico de um sistema (GUEDES, 2018).

Com isso em mente, utilizaremos o Diagrama de Casos de Uso, Diagrama de Atividade, Diagrama de Sequência e, por fim, Diagrama de Máquina de estados.

O diagrama de casos de uso nos mostra uma visão geral dos requisitos do sistema, sem se aprofundar nas especificações das funcionalidades. Através dele podemos entender as funcionalidades do sistema e quais funcionalidades o usuário tem acesso (GUEDES, 2018).

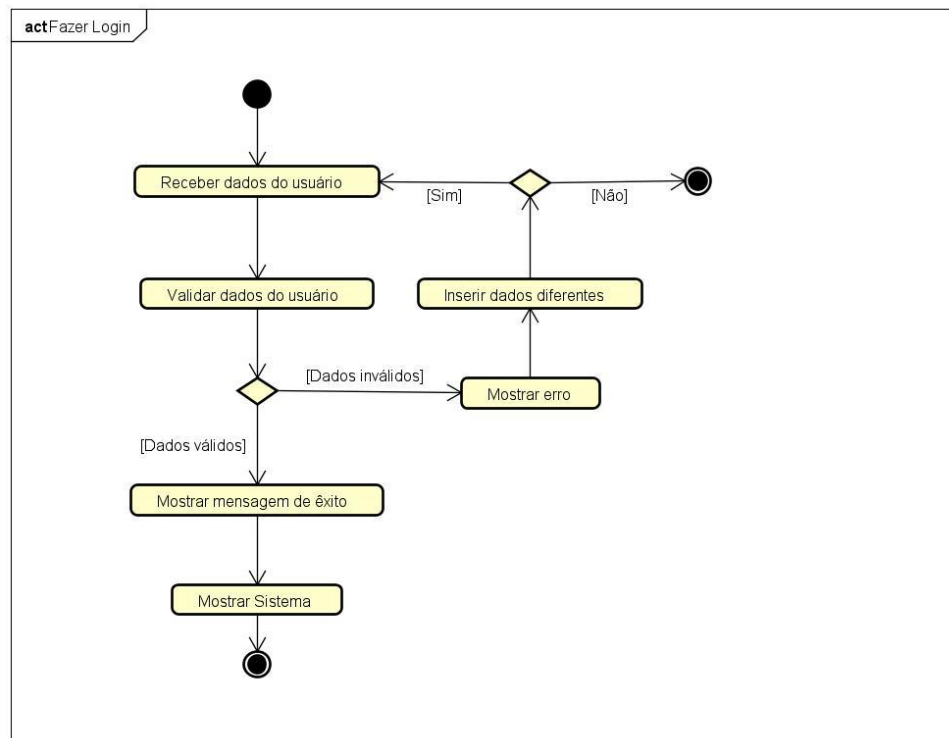
Figura 17 – Exemplo de Caso de Uso



Fonte: Autoria Própria, 2025.

O diagrama de atividade demonstra o fluxo de ações e decisões, representando a execução das atividades de forma clara e visual, desde o início até a conclusão do processo (BOOCH, 2006).

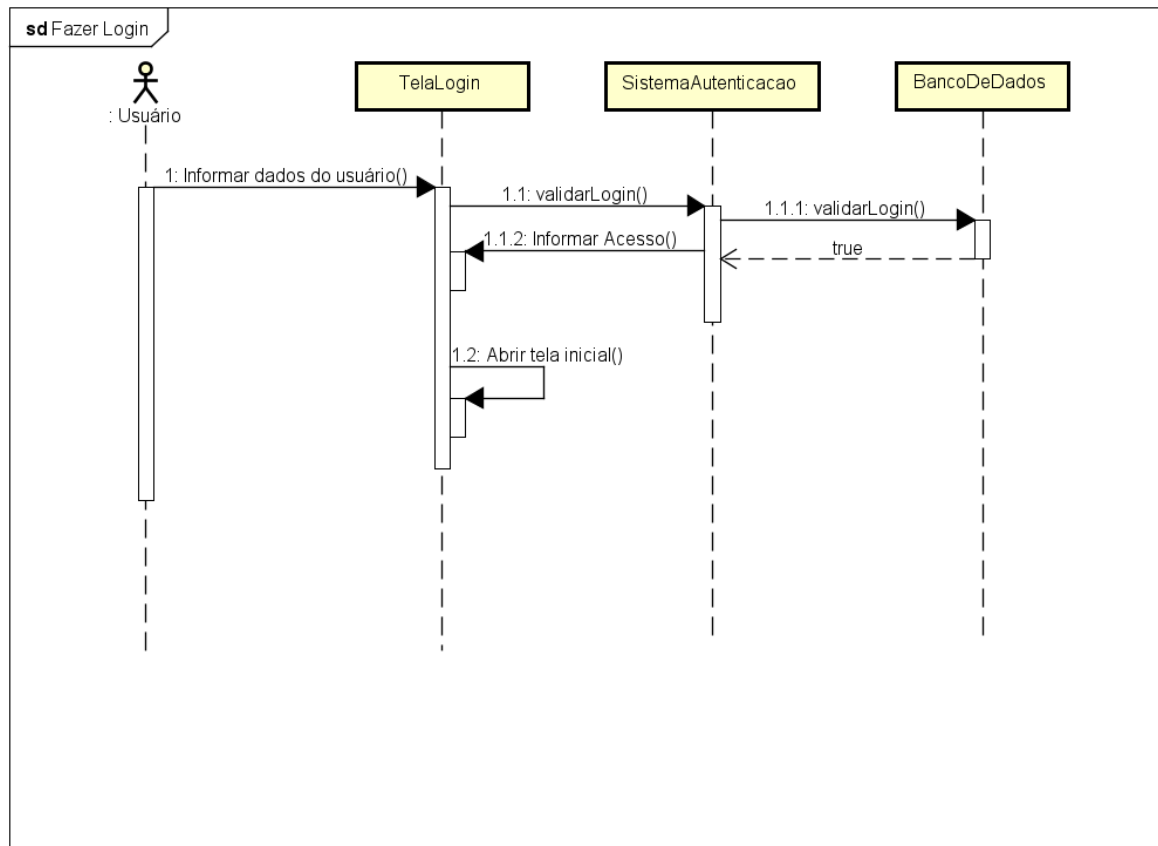
Figura 18 - Diagrama de atividade Fazer Login



Fonte: Autoria Própria, 2025.

O diagrama de sequência é uma representação das interações de um objeto durante a execução de um processo, deixando visível as mensagens trocadas entre os objetos, a ordem em que ocorrem e quais métodos são chamados. Para ser construído, ele utiliza casos de uso e o diagrama de classes (GUEDES, 2018).

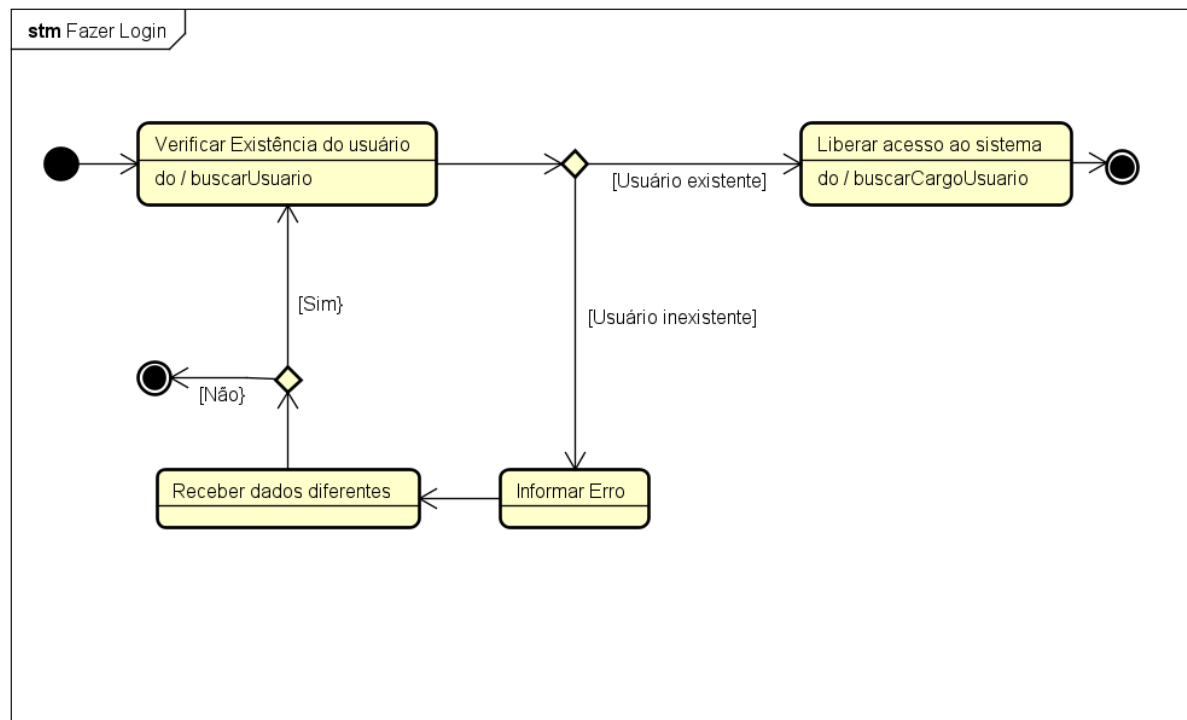
Figura 19 – Diagrama de sequência Fazer Login



Fonte: Autoria Própria, 2025.

O diagrama de máquina de estado é utilizado para representar o comportamento de um objeto ou sistema em resposta a eventos, mostrando os estados pelos quais um objeto pode passar ao longo de sua vida (FOWLER, 2005).

Figura 20 - Diagrama de máquina de estado Fazer Login



Fonte: Autoria Própria, 2025.

2.2.16 Wireframe

Em conformidade com Osório (2021), os wireframes são criados para elaborar protótipos visuais da estrutura e finalidade da aplicação, auxiliando na usabilidade ao mostrar os elementos essenciais que deverão estar presentes no layout do software antes de ser desenvolvido, fornecendo uma perspectiva visual de como ficará.

Em continuidade sobre prototipação, o Figma destaca-se como uma das principais ferramentas de design do mercado, sobressaindo-se pela sua alta performance e pela aptidão para uso simultâneo de usuários, oferecendo uma ampla variedade de recursos tecnológicos que facilitam o desenvolvimento de esboços visuais, segundo Rosa (2023).

O objetivo dessa ferramenta é facilitar, para os desenvolvedores, a criação de uma interface digital, oferecendo serviços como criação de componentes, protótipos interativos, realização de testes e colaboração do time em tempo real de forma remota e simultânea, conforme Silva (2024).

2.2.17 Modelagem 3D

Modelagem 3D é o processo de representar digitalmente a geometria de objetos/cenas, permitindo análise e pré-visualização antes da construção física. No ensino e na prática profissional, ela apoia “visualização de projetos em três dimensões” e a comunicação entre áreas como engenharia, arquitetura e design, ampliando previsibilidade e qualidade das decisões de projeto. Em aplicações científicas e patrimoniais, as tecnologias de modelagem produzem “modelo tridimensional de alta definição e precisão” (CORREA, 2023).

Existem dois jeitos principais de modelar em 3D. O primeiro utiliza polígonos para montar o objeto, sendo leve e rápido para visualização em tempo real. O segundo utiliza superfícies B-spline racionais não uniformes (NURBS), que geram formas suaves e precisas, ideais para aplicações de engenharia. Na prática, os dois se complementam: é comum iniciar pelo segundo método, devido à sua precisão, e depois converter para uma malha poligonal para visualização ou animação. Também é possível partir de uma malha, por exemplo, de escaneamento 3D, e reconstruir superfícies NURBS para uso técnico, com apoio de ferramentas como Rhinoceros/Grasshopper (VALE et al., 2022).

No pipeline contemporâneo, a modelagem pode surgir de construção manual ou de aquisição, seguida de etapas como retopologia, mapeamento UV, aplicação de materiais e preparação para uso. Estudos nacionais apontam que a fotogrametria com Veículos Aéreos Não Tripulados (VANT) é uma rota eficiente para obter modelos 3D de edificações, desde que acompanhada por procedimentos de desenvolvimento e controle de qualidade do produto (COSTA et al., 2024).

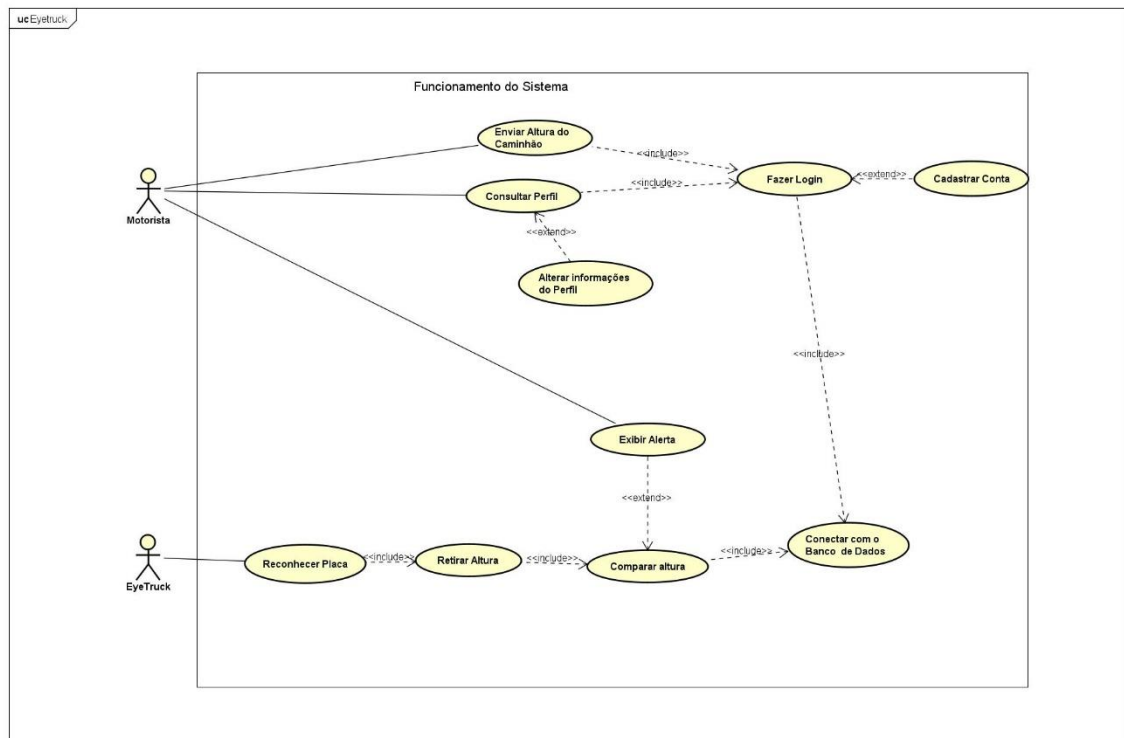
3 DESENVOLVIMENTO

Neste capítulo, serão apresentadas as etapas utilizadas para o desenvolvimento do EyeTruck. O processo inicia-se pela documentação, elaborada com base nos métodos da UML, seguida pelo processo de configuração do dispositivo Raspberry PI 5, pelo desenvolvimento das telas e pelo método utilizado para a compilação do aplicativo no dispositivo móvel. No decorrer do capítulo, serão descritos os diagramas UML de caso de uso, atividade, sequência e máquina de estado, indispensáveis para a construção do projeto. Posteriormente, é exibido o processo de criação da aplicação mobile e as suas respectivas interfaces. Essa base sólida possibilitou o desenvolvimento tanto do aplicativo quanto do dispositivo embarcado, que se complementam para entregar a solução proposta.

3.1 Diagrama de Caso de Uso

Nesta seção, será realizada uma detalhada análise do diagrama de caso de uso do EyeTruck, considerado o primeiro e principal diagrama da UML, servindo de base para todos os diagramas posteriores. O diagrama de caso de uso apresenta as interações entre os atores externos, como os motoristas, e as funcionalidades do sistema. Além disso, ele indica quais funcionalidades o usuário tem acesso, oferecendo uma visão geral do funcionamento do projeto. Essas funcionalidades estão representadas por 10 elipses, conforme ilustrado na figura abaixo.

Figura 21 - Diagrama de Casos de Uso



Fonte: Autoria Própria, 2025.

3.1.1 Documentação dos Casos de Uso

A documentação aprofunda-se nas etapas percorridas pelo usuário e pelo sistema durante a execução de um determinado caso de uso, incluindo os possíveis cenários alternativos. Complementarmente, são apresentadas as regras de negócio, os requisitos funcionais e não funcionais do EyeTruck, com a finalidade de proporcionar melhor compreensão do sistema para um indivíduo externo.

Na sequência, são elucidados os termos citados anteriormente. As regras de negócio estabelecem condições e restrições que independem da tecnologia utilizada. Os requisitos funcionais representam as funções que o sistema deve executar, enquanto os requisitos não funcionais abordam aspectos de qualidade, desempenho, segurança e usabilidade, assegurando que a solução seja eficiente e confiável.

Regras de Negócio:

- RN01: O acesso ao aplicativo requer autenticação por login.
- RN02: O suporte físico para o display é de responsabilidade do usuário.
- RN03: Apenas usuários com idade igual ou superior a 21 anos poderão utilizar o sistema.
- RN04: As senhas cadastradas devem conter no mínimo seis caracteres.
- RN05: O sistema deve garantir boa usabilidade, principalmente em dispositivos móveis.
- RN06: A empresa não se responsabiliza por acidentes decorrentes de instalação incorreta do dispositivo.
- RN07: A manutenção do hardware não é de responsabilidade da empresa.
- RN08: O sistema deve oferecer instalação simples e utilização intuitiva.

Requisitos Funcionais do Aplicativo:

- RF01: O sistema deve validar se o e-mail já está cadastrado.
- RF02: O sistema deve validar e conectar-se ao banco de dados.
- RF03: O sistema deve validar se o número do dispositivo já está cadastrado no banco de dados.

- RF04: O sistema deve exibir as informações do usuário cadastradas no banco.

Requisitos Funcionais do Motorista:

- RF01: O motorista deve poder cadastrar-se com dados pessoais e do veículo (e-mail, senha e altura do caminhão).
- RF02: O motorista deve acessar o sistema por meio de login (e-mail e senha).
- RF03: O motorista deve poder alterar sua senha.
- RF04: O motorista deve poder encerrar a sessão manualmente.
- RF05: O motorista deve poder editar suas informações pessoais.
- RF06: O motorista deve visualizar suas informações pessoais.
- RF07: O motorista deve atualizar a altura do caminhão quando necessário.

Requisitos Funcionais do EyeTruck:

- RF01: O dispositivo deve processar os dados recebidos da câmera.
- RF02: O dispositivo deve interpretar caracteres de placas de sinalização R-15.
- RF03: O dispositivo deve enviar os dados processados para o sistema, possibilitando a comparação com as medidas do veículo.
- RF04: O dispositivo deve comparar os valores processados com a altura do caminhão registrada.
- RF05: O dispositivo deve exibir um alerta ao motorista caso os valores da altura do veículo sejam maiores que os indicados na placa.

Requisitos Não Funcionais do EyeTruck:

- RNF01: O aplicativo será desenvolvido em React Native e terá design responsivo para tablets e celulares.
- RNF02: O sistema deve ser de uso simples e intuitivo.
- RNF03: A atualização dos dados enviados pelo dispositivo deve ocorrer com a menor latência possível.

- RNF04: O banco de dados será implementado utilizando os serviços do Firebase.
- RNF05: A modelagem do sistema deve seguir os padrões UML.

Abaixo estão as tabelas descritivas dos casos de uso presentes no diagrama:

Tabela 1 - Documentação do Caso de Uso: Fazer Login

Nome do Caso de Uso	Fazer Login
Caso de uso Geral	
Ator Principal	Motorista
Ator Secundário	
Resumo	Esse caso de uso descreve as etapas que o motorista deve seguir para acessar as funcionalidades principais do sistema.
Pré-Condições	Ter cadastro ativo.
Pós-Condições	Acesso autorizado à tela inicial.
Cenário Principal	
Ações do Ator	Ações do Sistema
1. O motorista informa e-mail e senha	
	2.O sistema valida as credenciais
	3.O sistema libera o acesso e redireciona para a tela inicial
Restrições/Validações	
Cenário Alternativo – Veficação de erros	
Ações do Ator	Ações do Sistema
1. O motorista insere e-mail ou senha incorretos	2. O sistema exibe mensagem de erro

Fonte: Autoria Própria, 2025.

Tabela 2 - Documentação do Caso de Uso: Cadastrar Conta

Nome do Caso de Uso	Cadastrar Conta
Caso de uso Geral	
Ator Principal	Motorista
Ator Secundário	
Resumo	Define como o motorista cria sua conta no sistema.
Pré-Condições	Possuir o código do dispositivo.
Pós-Condições	Usuário cadastrado com sucesso.
Cenário Principal	
Ações do Ator	Ações do Sistema
1. O motorista informa dados pessoais (nome completo, e-mail, altura do caminhão e senha)	
	2. O sistema valida a senha e registra as informações
	3. O sistema confirma o cadastro e direciona para a tela Login
Cenário Alternativo - Email Inválido	
Ações do Ator	Ações do Sistema
1. O motorista insere um e-mail existente	
	2. O sistema informa que o e-mail já está registrado

Fonte: Autoria Própria, 2025.

Tabela 3 - Documentação do Caso de Uso: Enviar Altura do Caminhão

Nome do Caso de Uso	Enviar Altura do Caminhão
Caso de uso Geral	
Ator Principal	Motorista.
Ator Secundário	Dispositivos.
Resumo	O motorista informa ou atualiza a altura do caminhão
Pré-Condições	Estar logado.
Pós-Condições	Altura atualizada no sistema.
Cenário Principal	
Ações do Ator	Ações do Sistema
1. O motorista acessa o menu de configurações	
2. O motorista informa a altura do caminhão	
	3. O sistema armazena a informação no Banco de Dados
Restrições/Validações	

Fonte: Autoria Própria, 2025.

Tabela 4 - Documentação do Caso de Uso: Consultar Perfil

Nome do Caso de Uso	Consultar Perfil
Caso de uso Geral	
Ator Principal	Motorista.
Ator Secundário	
Resumo	Permite ao motorista alterar informações pessoais.
Pré-Condições	O usuário deve estar logado.
Pós-Condições	
Cenário Principal	
Ações do Ator	Ações do Sistema
1. O motorista acessa a página de perfil	
	2. O sistema exibe os dados atuais
Restrições/Validações	
Cenário Alternativo - Alterar informações do Perfil	
Ações do Ator	Ações do Sistema
	1. Executar caso de uso Alterar informações do Perfil

Fonte: Autoria Própria, 2025.

Tabela 5 - Documentação do Caso de Uso: Alterar informações do Perfil

Nome do Caso de Uso	Alterar informações do Perfil
Caso de uso Geral	
Ator Principal	Motorista.
Ator Secundário	
Resumo	Esse caso de uso descreve os passos que o usuário deve seguir para editar o seu perfil.
Pré-Condições	O usuário deve estar logado.
Pós-Condições	Perfil Atualizado
Cenário Principal	
Ações do Ator	Ações do Sistema
1. Acessar a página de Perfil	
	2. Exibir os dados cadastrados
3. Preencher os campos de texto com as novas informações	
4. Clicar no botão de Editar	
	5. O sistema faz a validação dos dados inseridos
	6. As informações do usuário são atualizadas
Restrições/Validações	
Cenário Alternativo - Alterar Senha	
Ações do Ator	Ações do Sistema

1. Selecionar a opção modificar senha	
	2. Exibir o formulário de troca de senha
3. Informar a senha atual	
4. Definir uma nova senha	
5. Repetir a nova senha para confirmação	
	6. O sistema valida se a senha atual está correta
	7. O sistema verifica se a confirmação da nova senha coincide
	8. Em caso de sucesso, atualizar a senha no cadastro
Cenário Alternativo - Informações Inválidas	
Ações do Ator	Ações do Sistema
	1. Exibir mensagem de erro
Cenário Exceção - Excluir Conta	
Ações do Ator	Ações do Sistema
1. Selecionar opção de excluir conta	
	2. Exibir modal de confirmação de exclusão
3. Confirmar a exclusão da conta.	

Fonte: Autoria Própria, 2025.

Tabela 6 - Documentação do Caso de Uso: Conectar com o Banco de Dados

Nome do Caso de Uso	Conectar com o Banco de Dados
Caso de uso Geral	
Ator Principal	EyeTruck
Ator Secundário	Motorista
Resumo	Permite a conexão do aplicativo com o hardware.
Pré-Condições	Dispositivo ligado e conectado ao Wifi
Pós-Condições	
Cenário Principal	
Ações do Ator	Ações do Sistema
	1. Receber parâmetros de conexão
	2. Validar parâmetros
	3. Conectar com o banco
	4. Permitir operações
Restrições/Validações	
Cenário Exceção - Erro na conexão	
Ações do Ator	Ações do Sistema
	1. Exibir mensagem de erro ao conectar com o banco

Fonte: Autoria Própria, 2025.

Tabela 7 - Documentação do Caso de Uso: Exibir Alerta

Nome do Caso de Uso	Exibir Alerta
Caso de uso Geral	
Ator Principal	Dispositivo Eyetruck
Ator Secundário	Motorista
Resumo	Notifica o motorista sobre possíveis colisões com base na altura do veículo.
Pré-Condições	Executar Caso de Uso Comparar Altura
Pós-Condições	Motorista informado do risco.
Cenário Principal	
Ações do Ator	Ações do Sistema
	1. O dispositivo detecta risco de colisão
	2. O sistema apresenta a mensagem de alerta
Restrições/Validações	

Fonte: Autoria Própria, 2025.

Tabela 8 - Documentação do Caso de Uso: Comparar Altura

Nome do Caso de Uso	Comparar Altura
Caso de uso Geral	
Ator Principal	Dispositivo EyeTruck
Ator Secundário	
Resumo	Garante que a altura registrada do caminhão seja comparada com a altura de obstáculos.
Pré-Condições	Executar Caso de Uso Retirar Altura
Pós-Condições	
Cenário Principal	
Ações do Ator	Ações do Sistema
	1. Receber Altura do Caminhão
	2. Comparar com a Altura da placa
Restrições/Validações	

Fonte: Autoria Própria, 2025.

Tabela 9 - Documentação do Caso de Uso: Retirar Altura

Nome do Caso de Uso	Retirar Altura
Caso de uso Geral	
Ator Principal	Dispositivo EyeTruck
Ator Secundário	Motorista
Resumo	Reconhece os caracteres na placa e retira o valor para comparação.
Pré-Condições	Executar Caso de Uso Reconhecer Placa
Pós-Condições	
Cenário Principal	
Ações do Ator	Ações do Sistema
	1. Receber imagem da placa
	2. Reconhecer caracteres na placa
	3. Extrair os caracteres.
Restrições/Validações	

Fonte: Autoria Própria, 2025.

Tabela 10 - Documentação do Caso de Uso: Reconhecer Placa

Nome do Caso de Uso	Reconhecer Placa
Caso de uso Geral	
Ator Principal	Dispositivo EyeTruck
Ator Secundário	Motorista
Resumo	Reconhece o padrão da placa através do YOLO para tirar uma foto e usar posteriormente.
Pré-Condições	
Pós-Condições	
Cenário Principal	
Ações do Ator	Ações do Sistema
	1. Receber a imagem de entrada.
	2. Dividir a imagem em grades
	3. Detectar regiões com características compatíveis com placas do tipo R-15.
	4. Calcular a confiança e classificar cada região detectada.
	5. Selecionar as detecções mais confiáveis e descartar as irrelevantes.
	6. Retornar a detecção com maior confiança para a classe R-15.

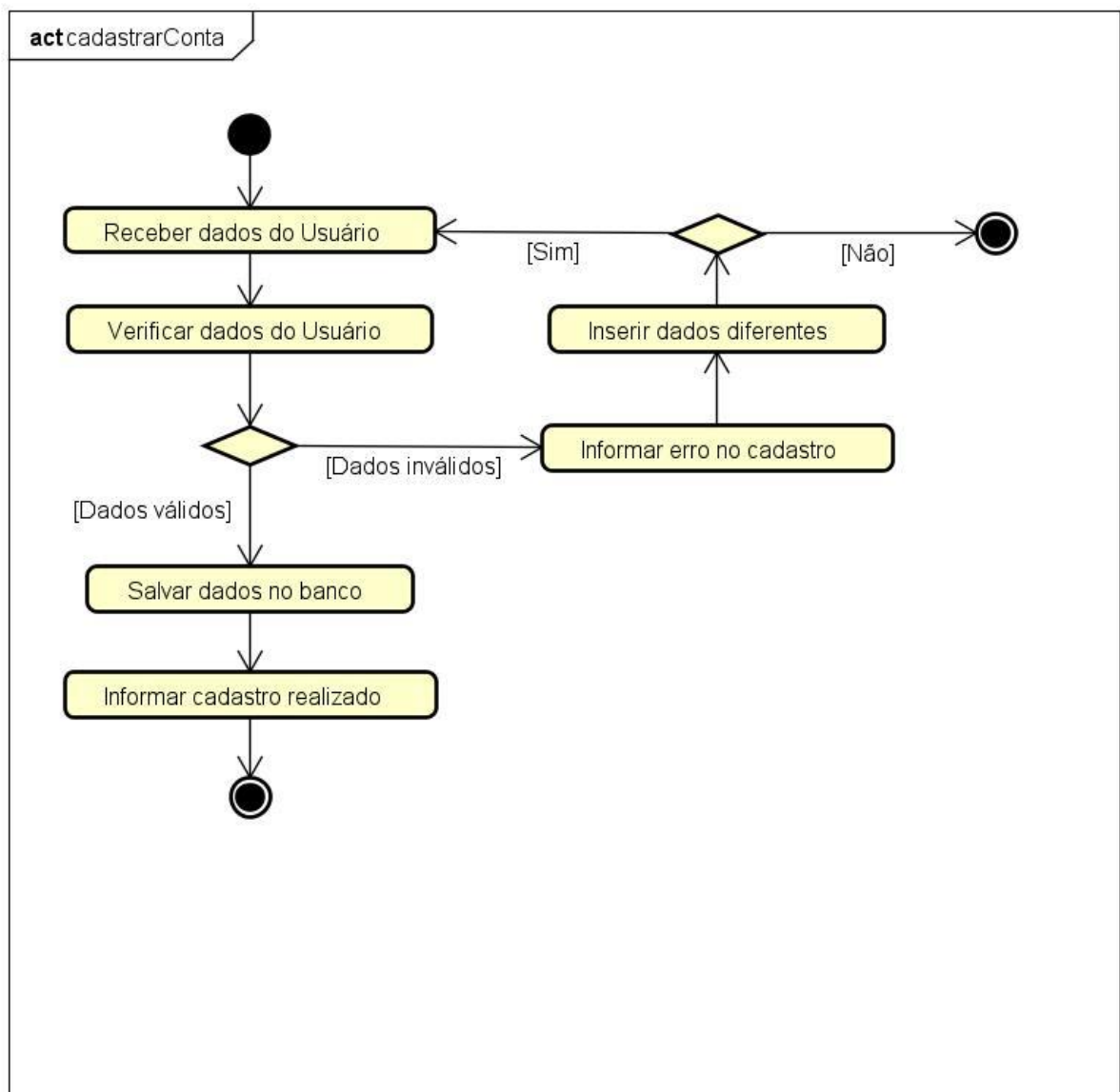
Fonte: Autoria Própria, 2025.

3.2 Diagrama de Atividade

O presente diagrama tem como objetivo representar o fluxo de ações dos casos de uso de forma visual auxiliando no planejamento e na construção das funcionalidades do projeto. Sua principal função é descrever o comportamento do sistema, dividindo em etapas para facilitar a visualização e compreensão.

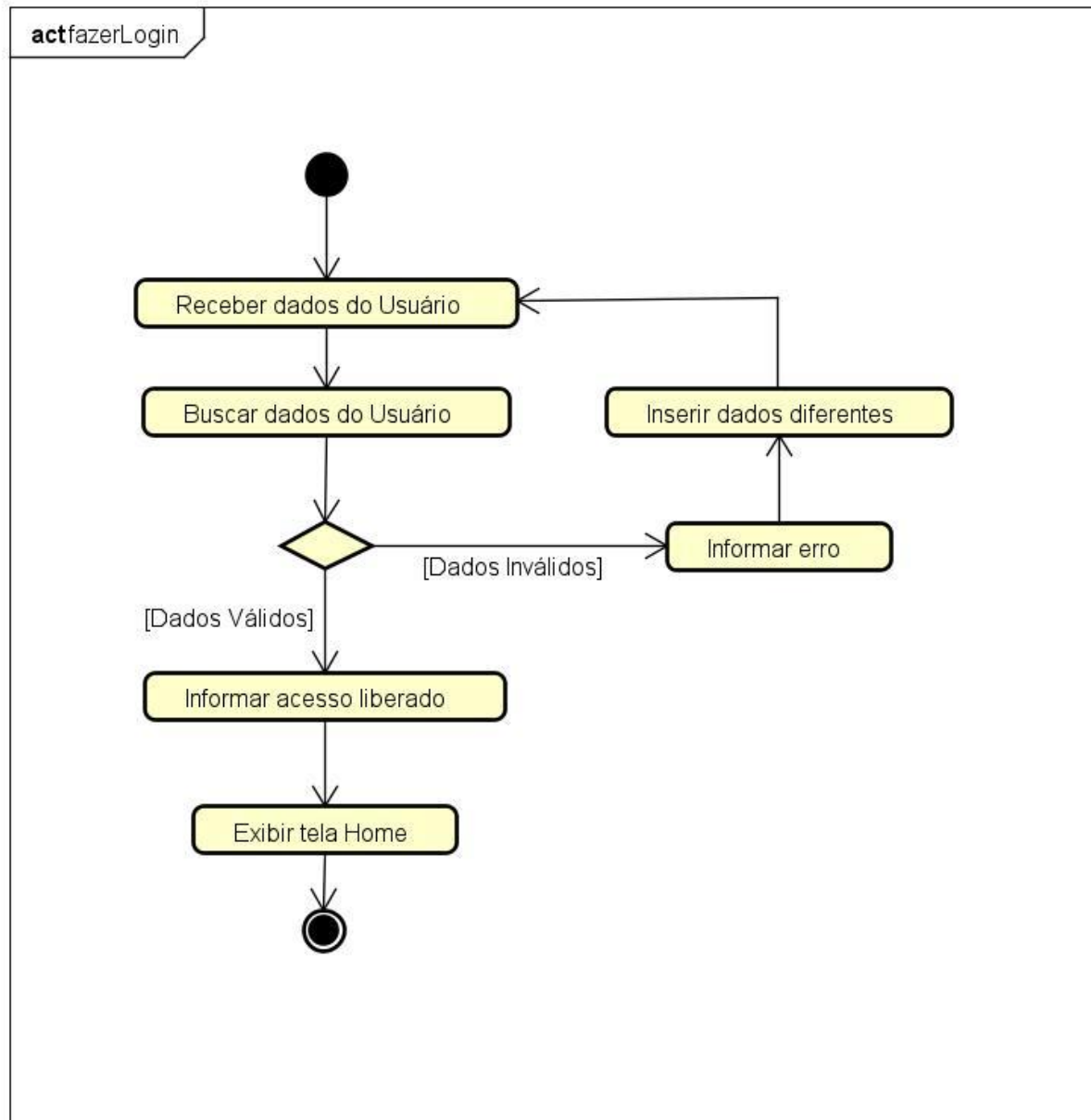
Os diagramas serão organizados com base no ator que interage com o caso de uso. Inicialmente, será apresentado os diagramas relacionados ao ator e, posteriormente, aqueles que representam as interações do sistema.

Figura 22 - Diagrama de Atividade: Cadastrar Conta



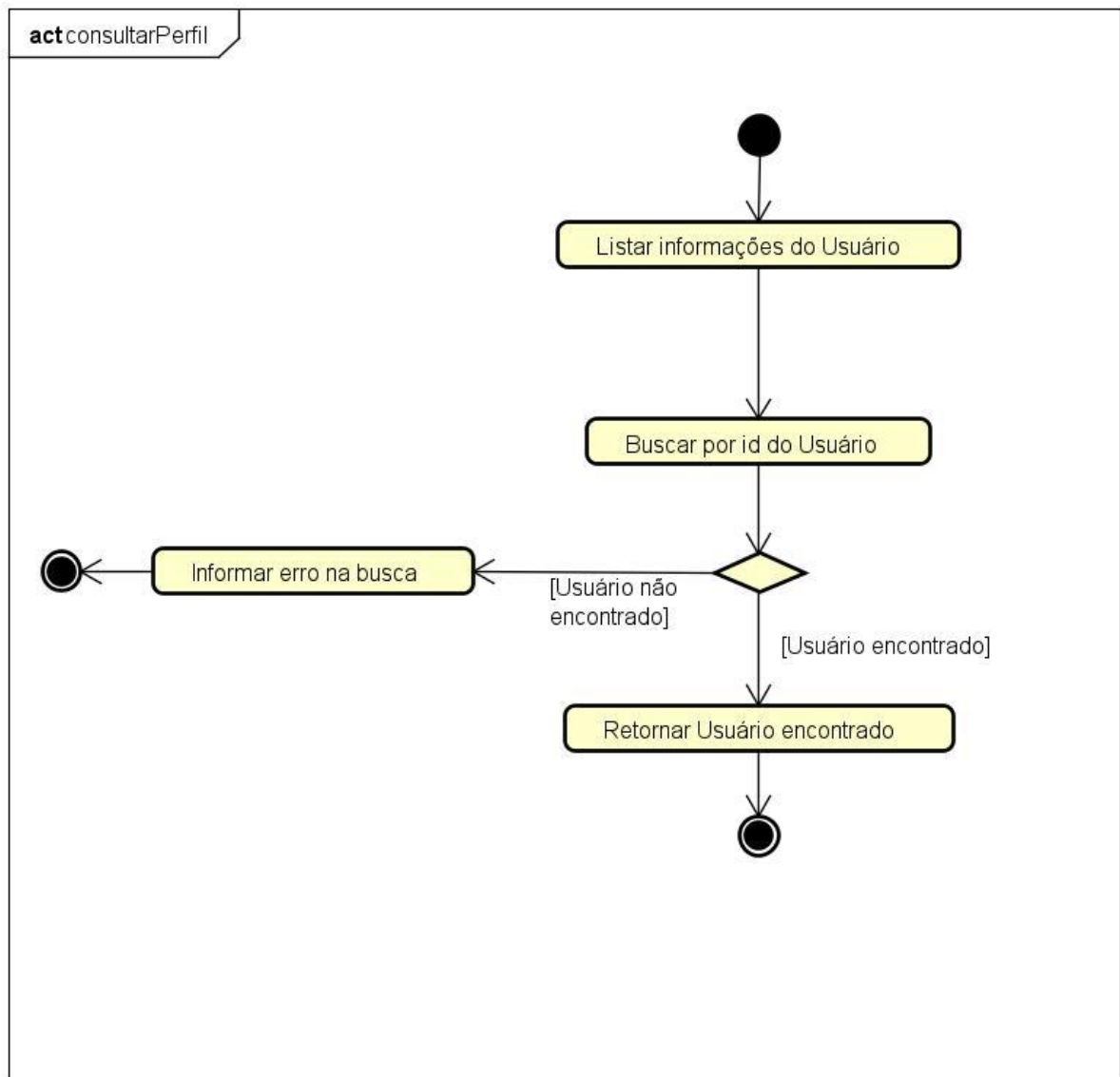
Fonte: Autoria Própria, 2025.

Figura 23 - Diagrama de Atividade: Fazer Login



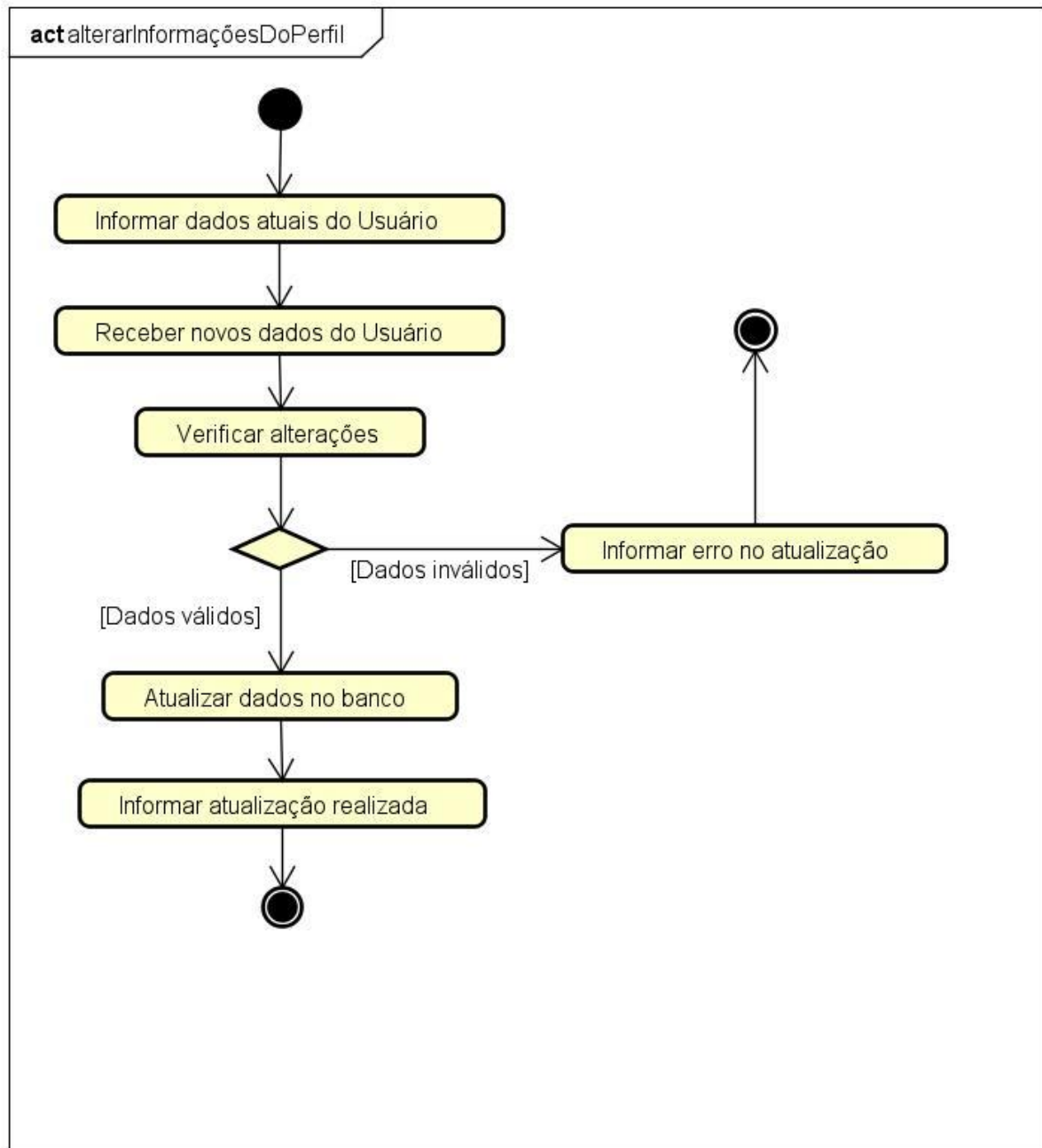
Fonte: Autoria Própria, 2025.

Figura 24 - Diagrama de Atividade: Consultar Perfil



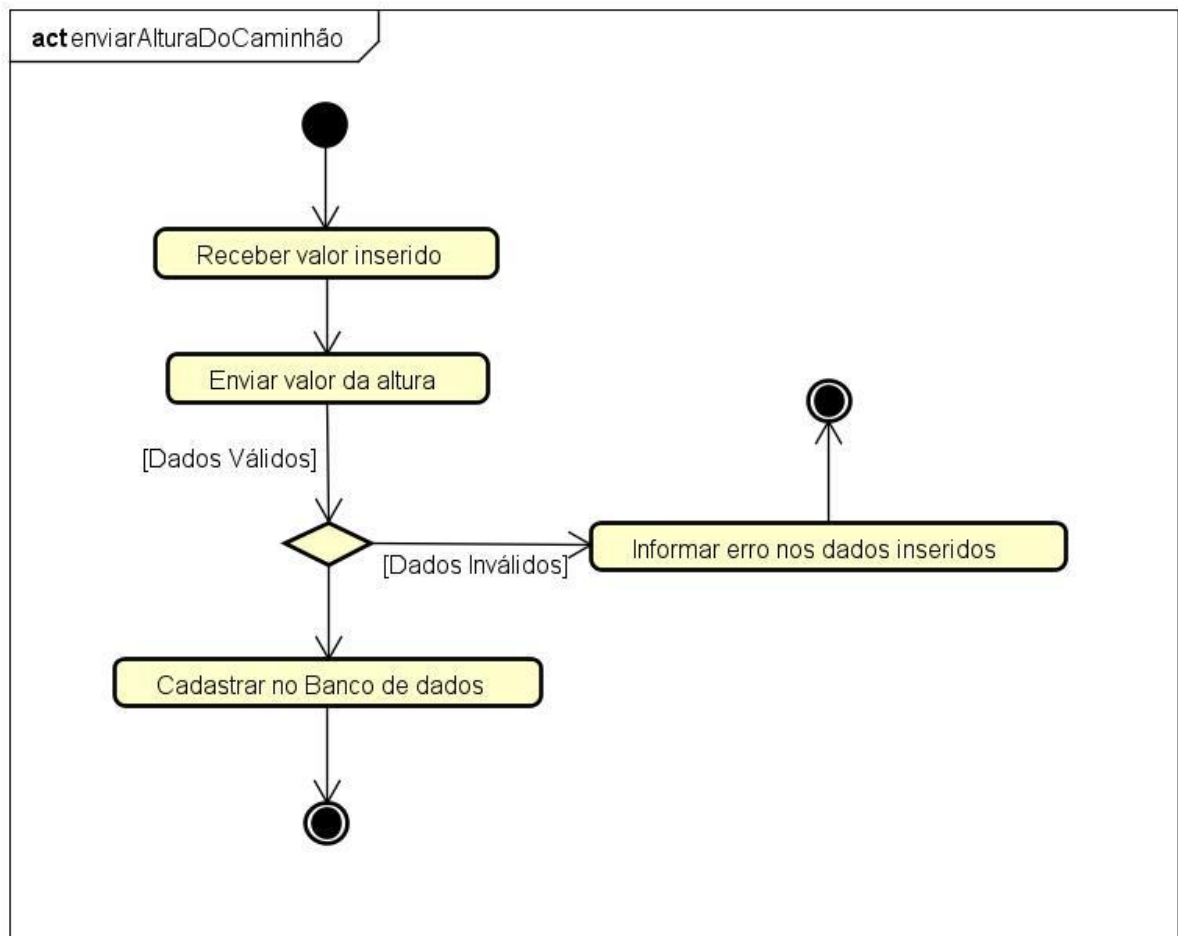
Fonte: Autoria Própria, 2025.

Figura 25 - Diagrama de Atividade: Alterar Informações Do Perfil



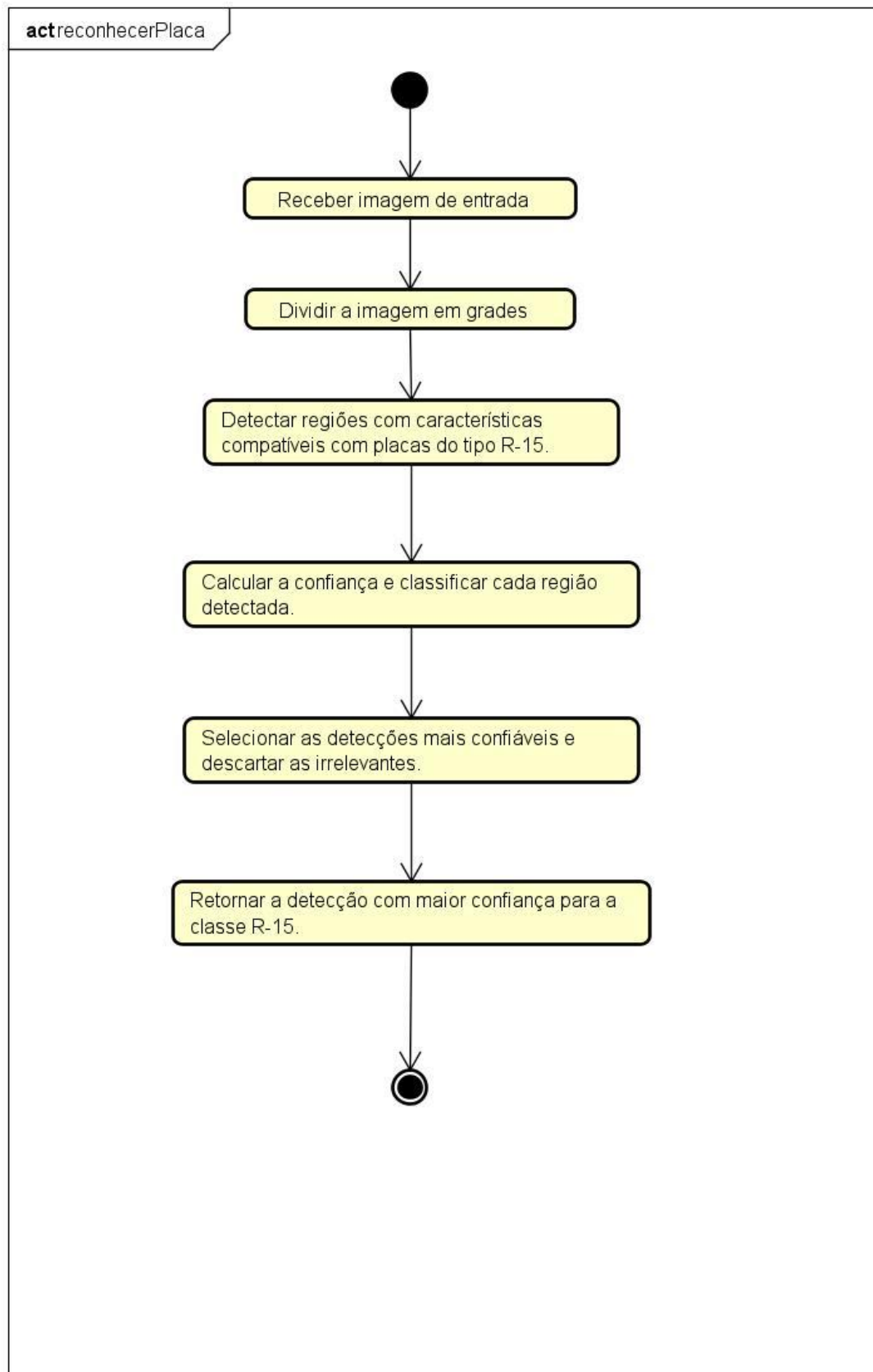
Fonte: Autoria Própria, 2025.

Figura 26 - Diagrama de Atividade: Enviar Altura Do Caminhão



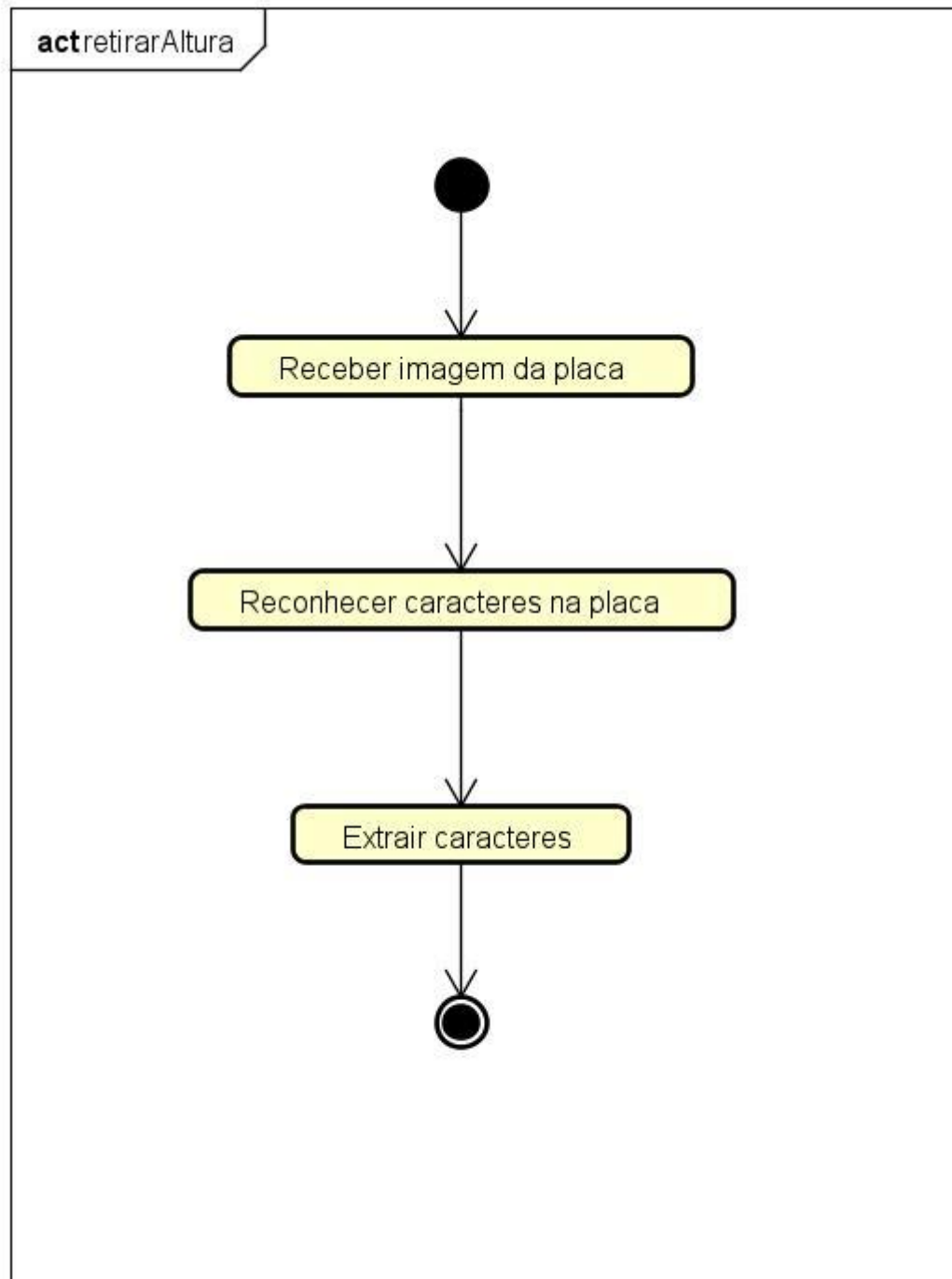
Fonte: Autoria Própria, 2025.

Figura 27 - Diagrama de Atividade: Reconhecer Placa



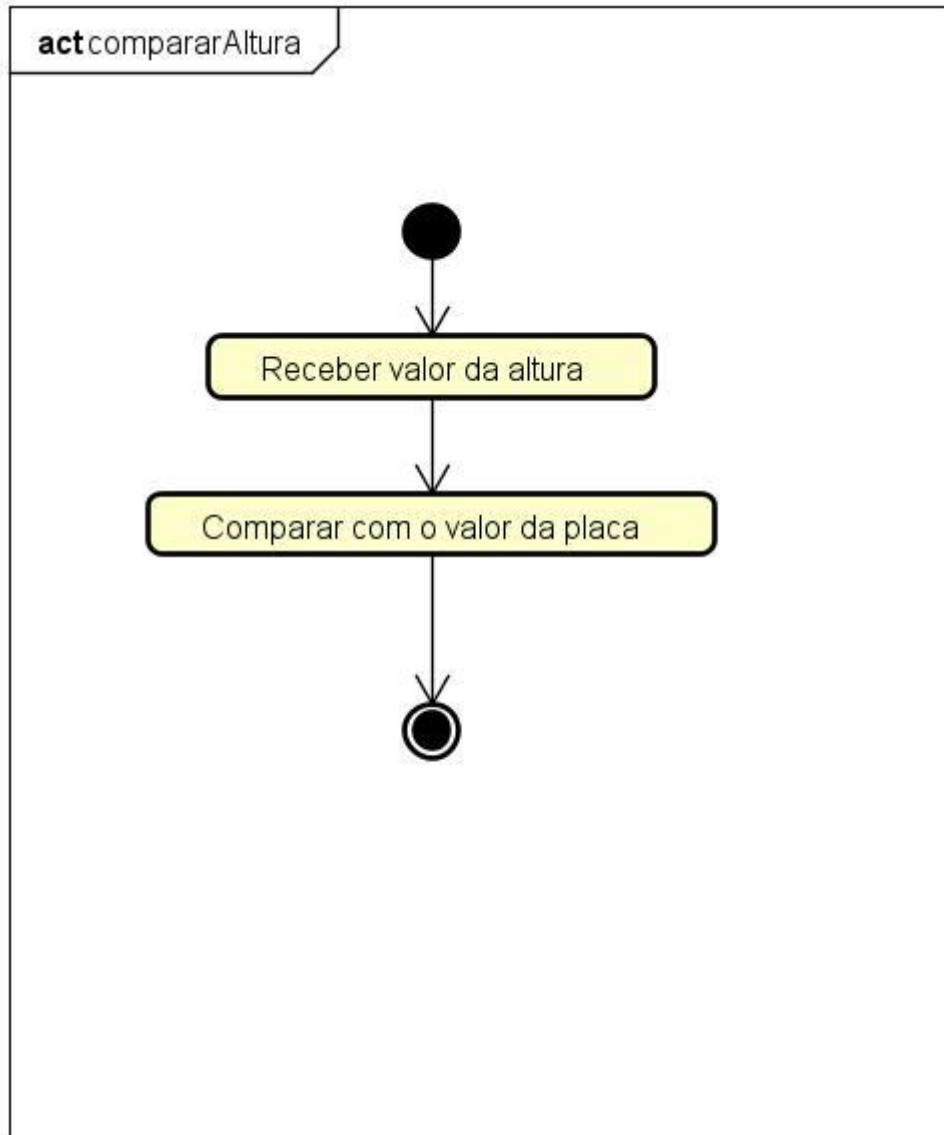
Fonte: Autoria Própria, 2025.

Figura 28 - Diagrama de Atividade: Retirar Altura



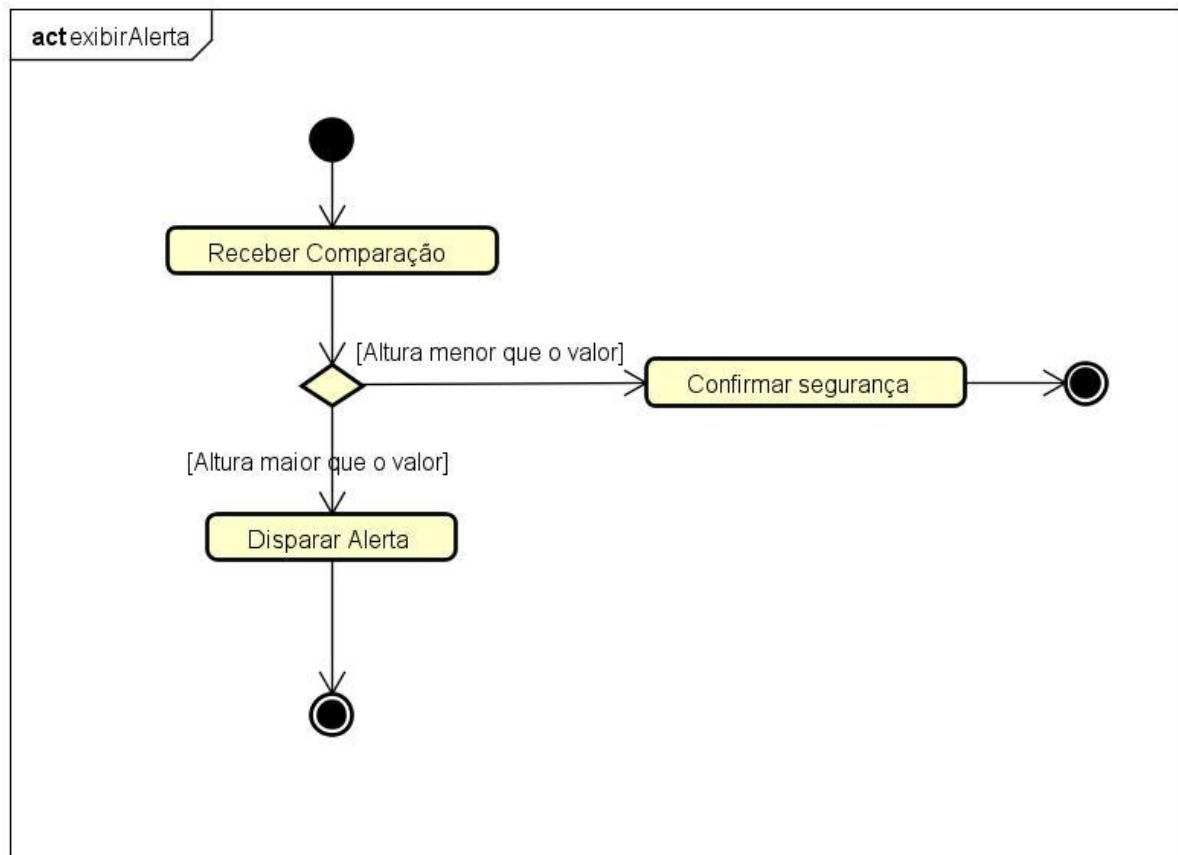
Fonte: Autoria Própria, 2025.

Figura 29 - Diagrama de Atividade: Comparar Altura



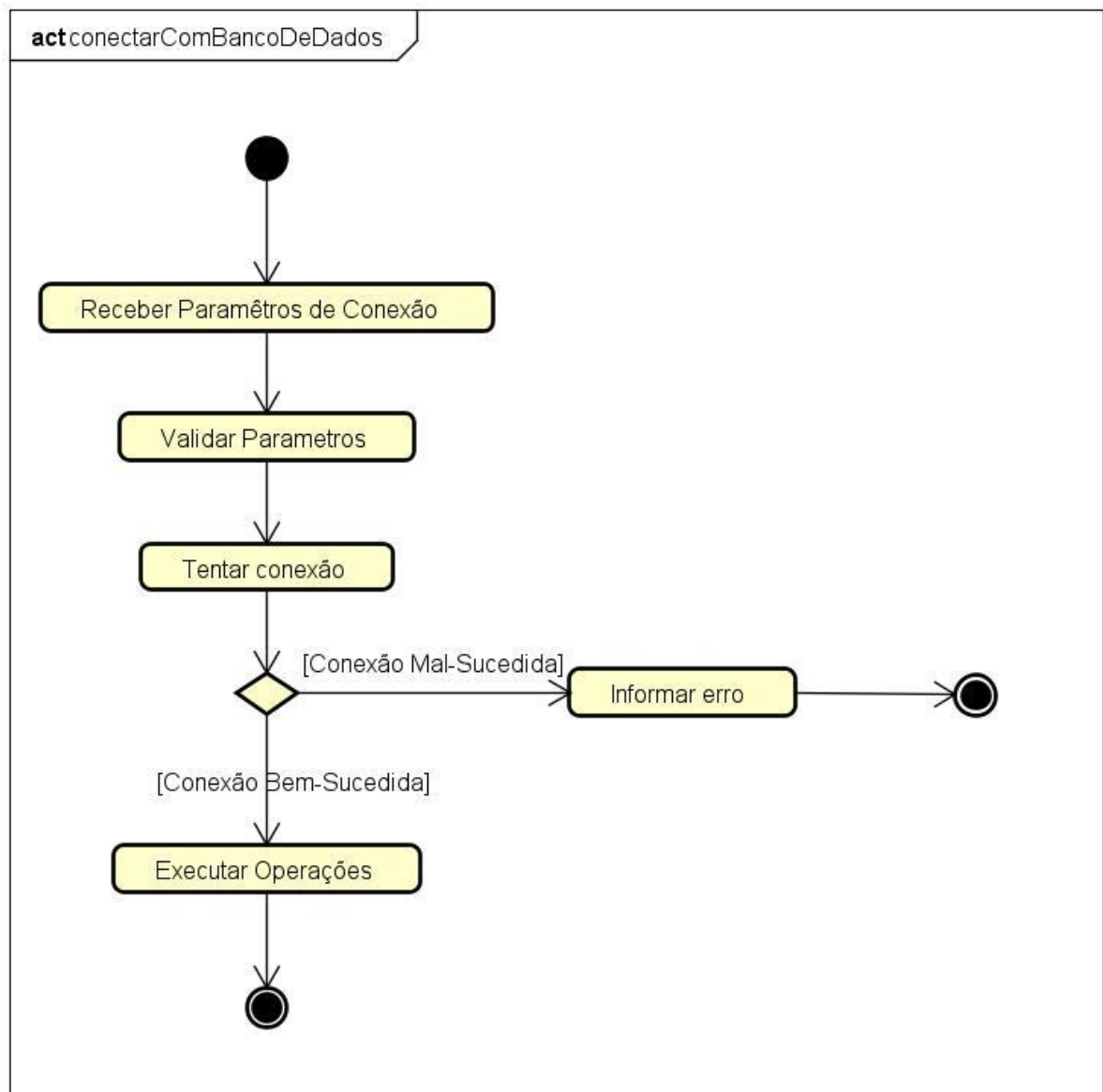
Fonte: Autoria Própria, 2025.

Figura 30 - Diagrama de Atividade: Exibir Alerta



Fonte: Autoria Própria, 2025.

Figura 31 - Diagrama de Atividade: Conectar Com o Bando de Dados

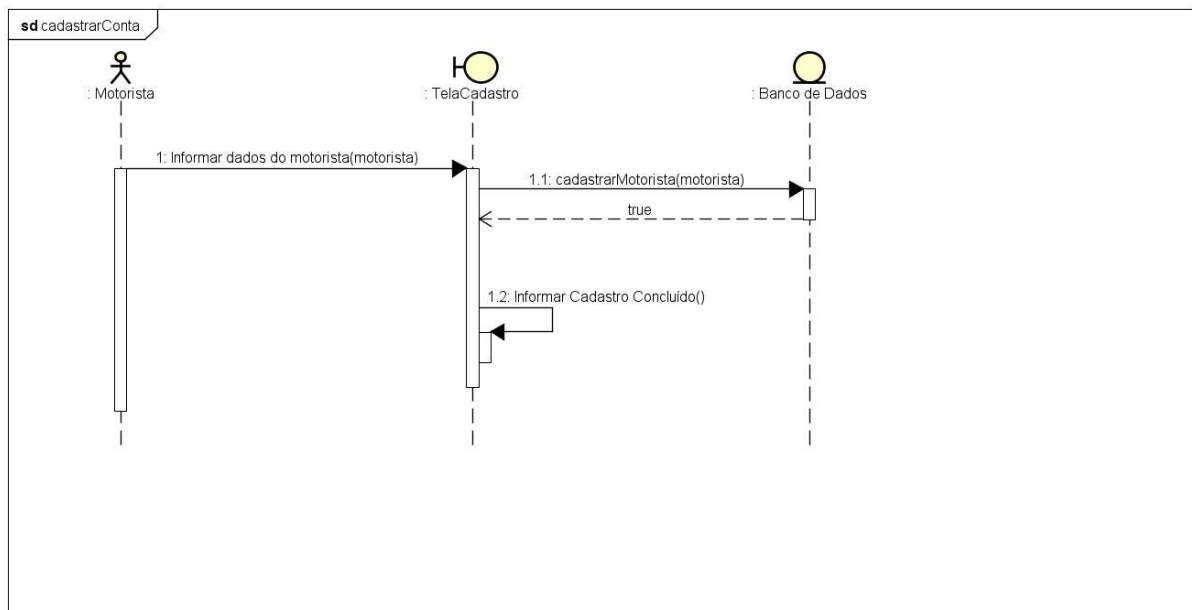


Fonte: Autoria Própria, 2025.

3.3 Diagrama de Sequência

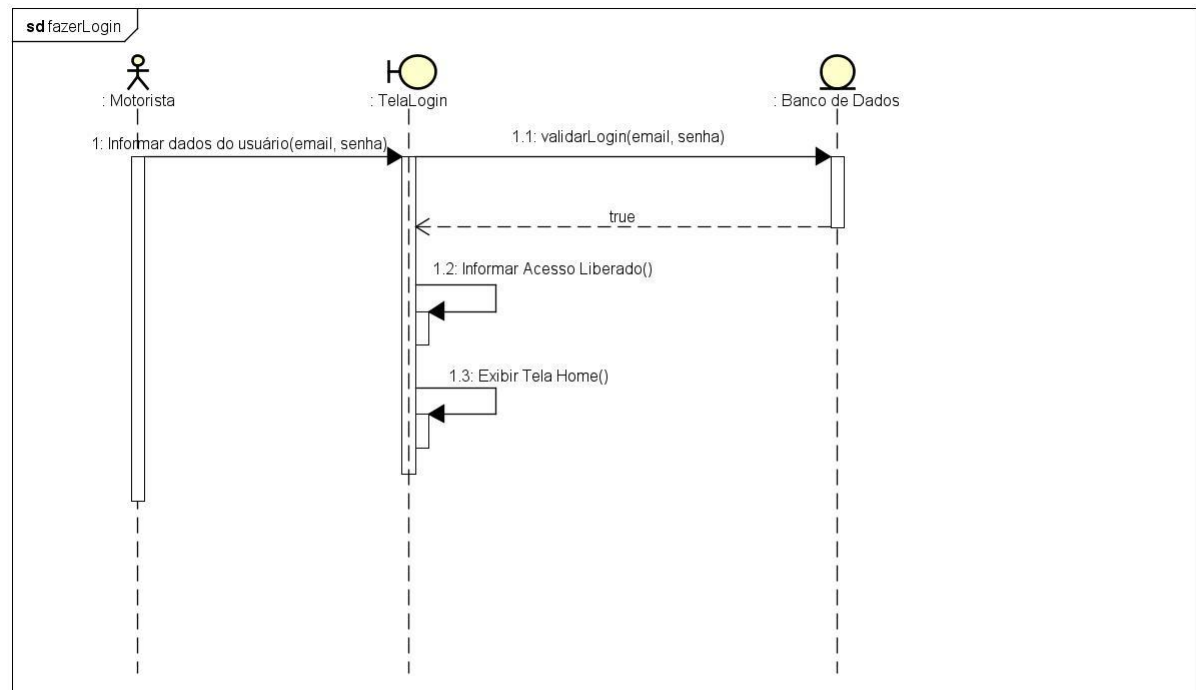
Nesta seção, será apresentado o diagrama de sequência, que enfatiza a ordem temporal das interações, esclarecendo a sequência de ações, os elementos envolvidos, o conteúdo e a ordem em que os processos acontecem. A importância desse diagrama é detalhar a comunicação entre os componentes do sistema e a ordem cronológica dos acontecimentos.

Figura 32 - Diagrama de Sequência: Cadastrar Conta



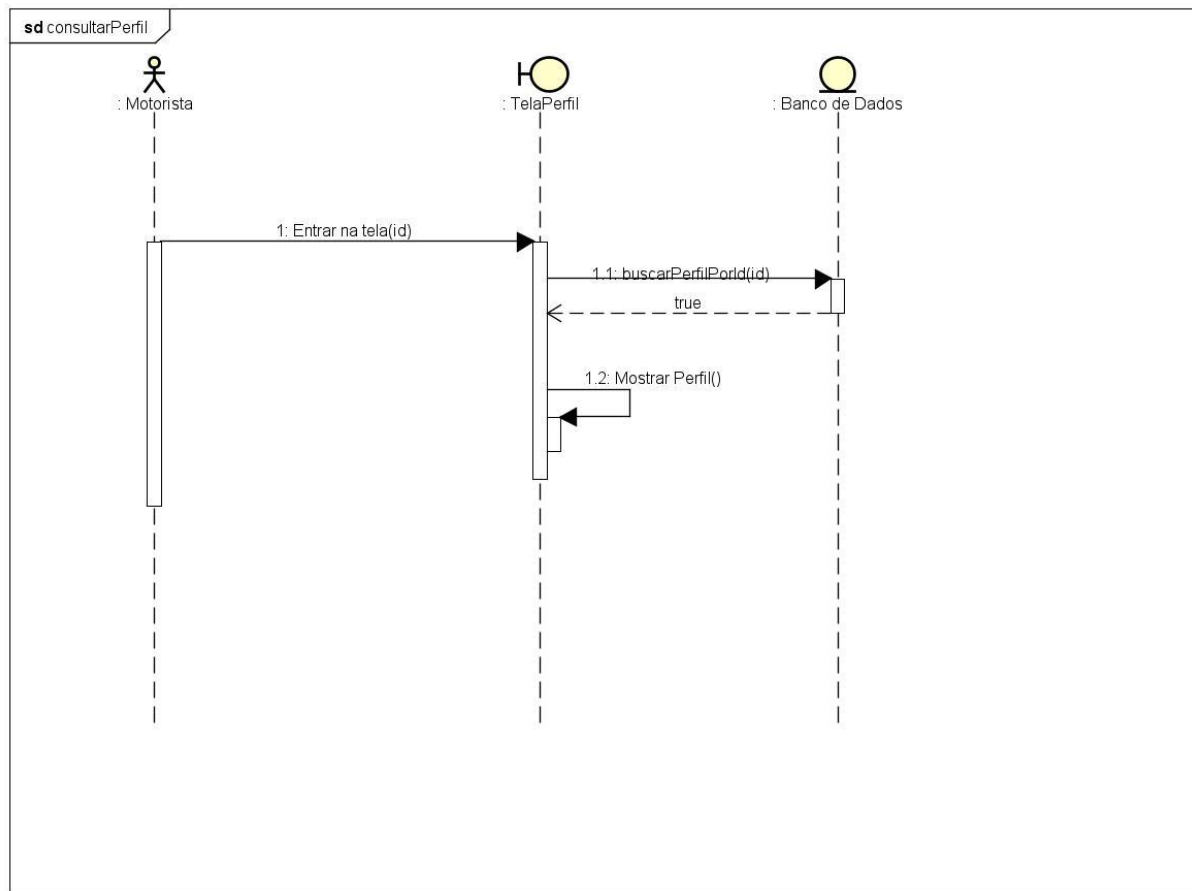
Fonte: Autoria Própria, 2025.

Figura 33 - Diagrama de Sequência: Fazer Login



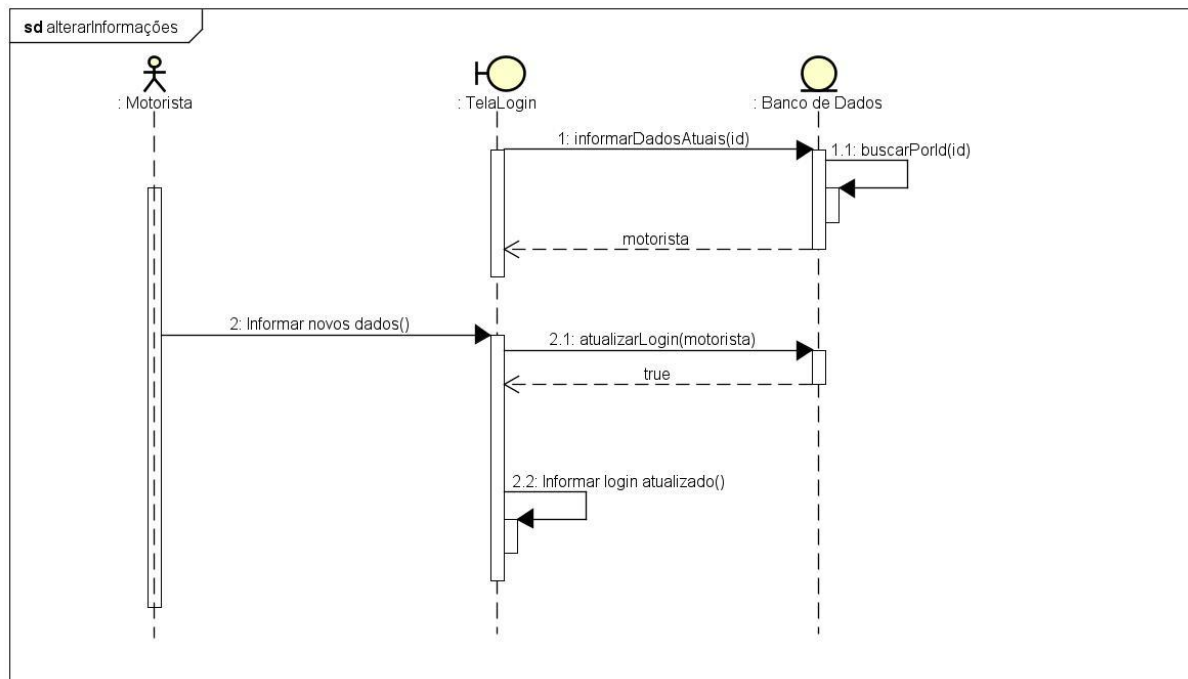
Fonte: Autoria Própria, 2025.

Figura 34 - Diagrama de Sequência: Consultar Perfil



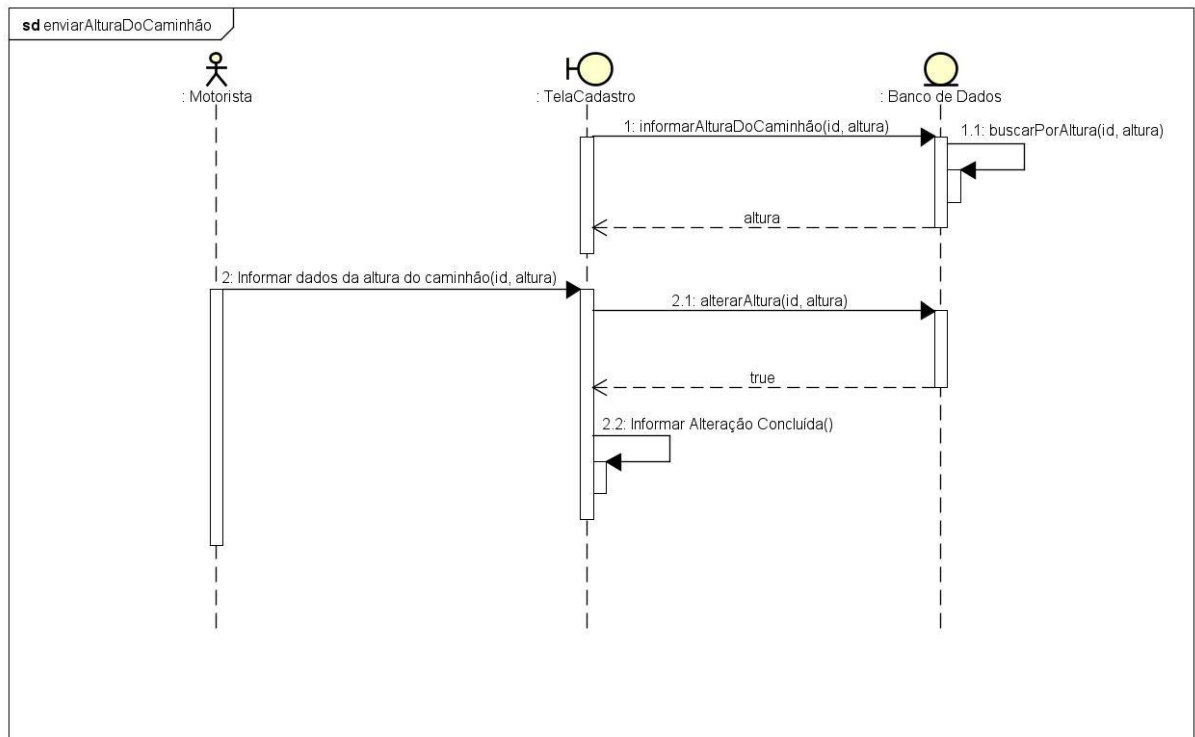
Fonte: Autoria Própria, 2025.

Figura 35 - Diagrama de Sequência: Alterar Informações do Perfil



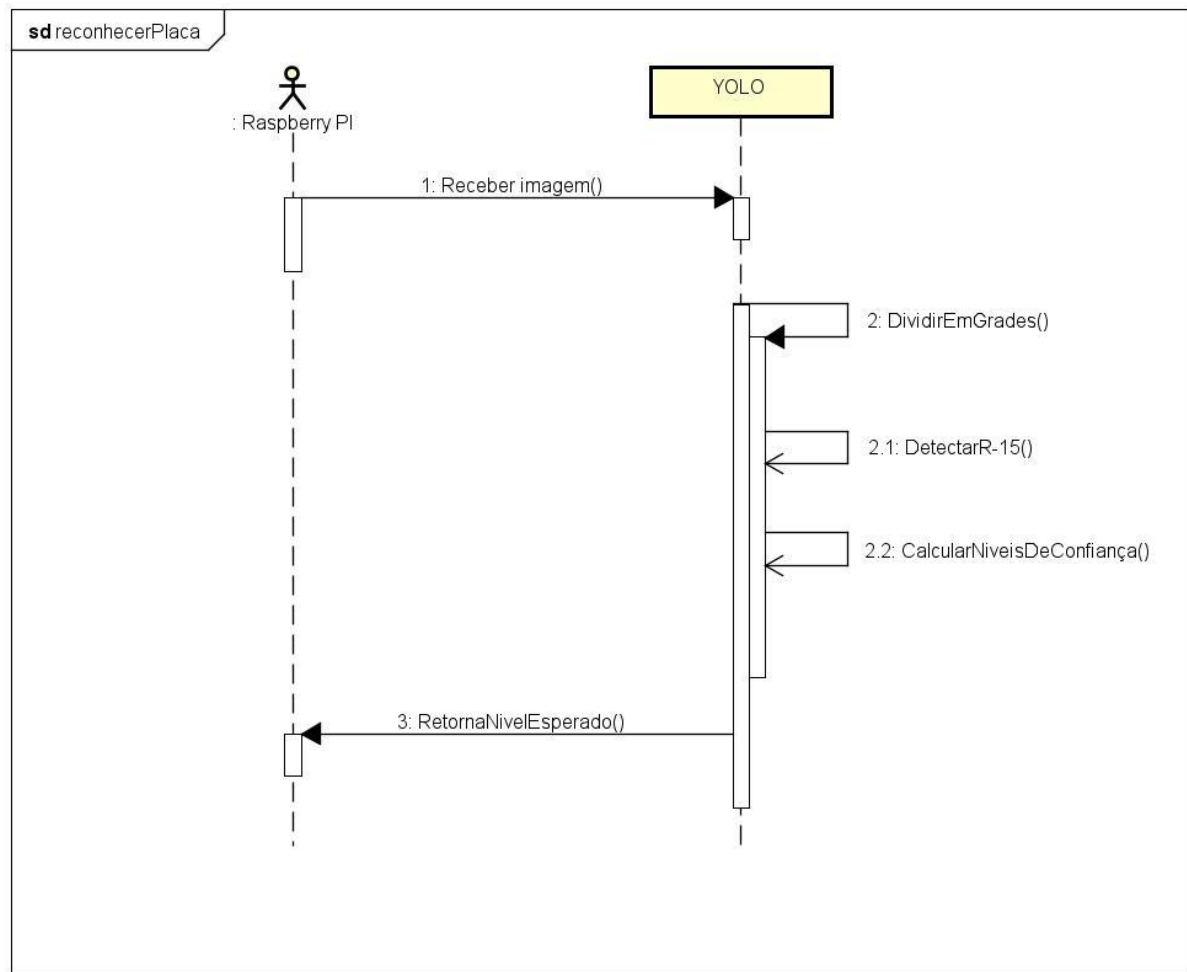
Fonte: Autoria Própria, 2025.

Figura 36 - Diagrama de Sequência: Enviar Altura Do Caminhão



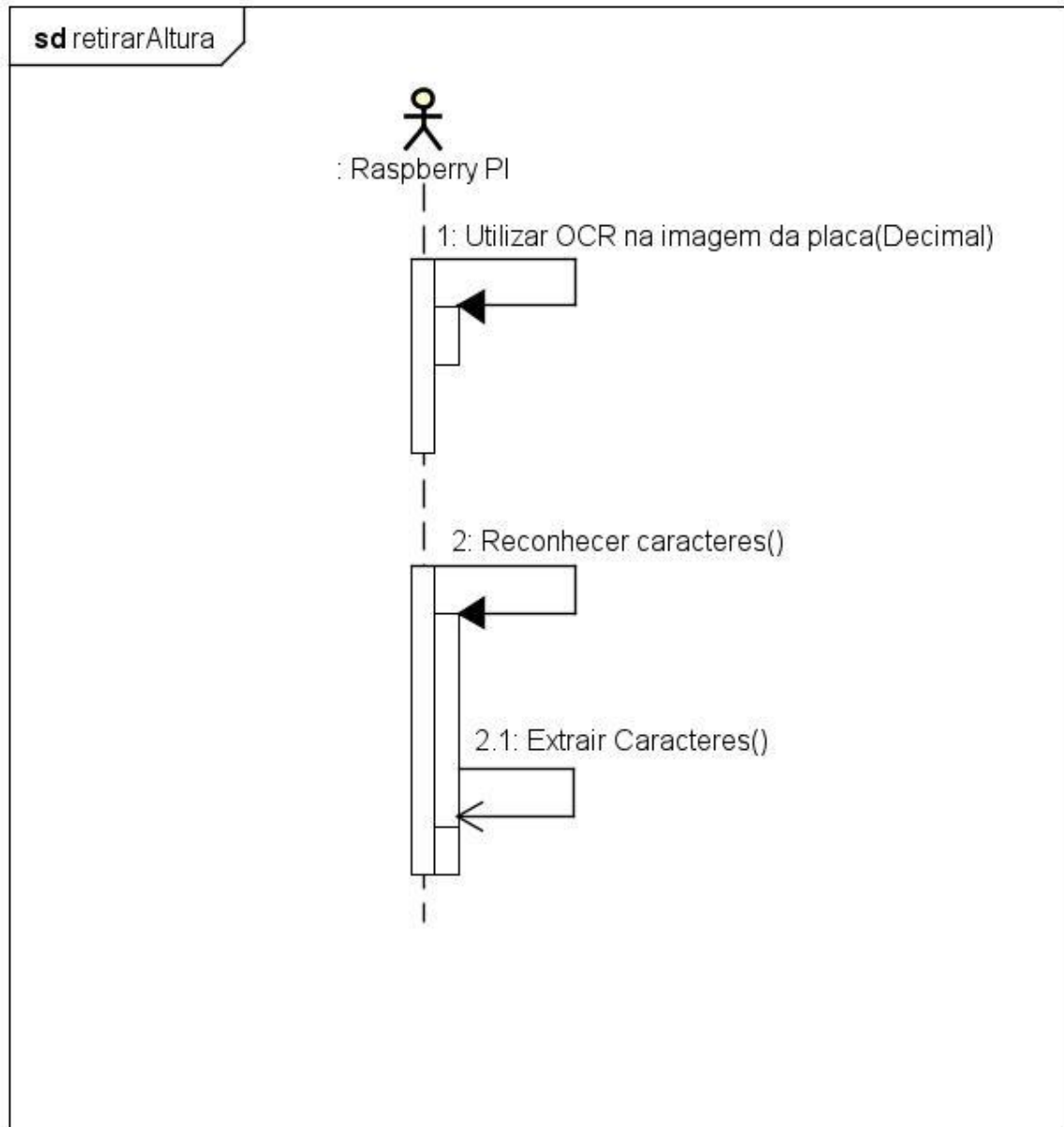
Fonte: Autoria Própria, 2025.

Figura 37 - Diagrama de Sequência: Reconhecer Placa



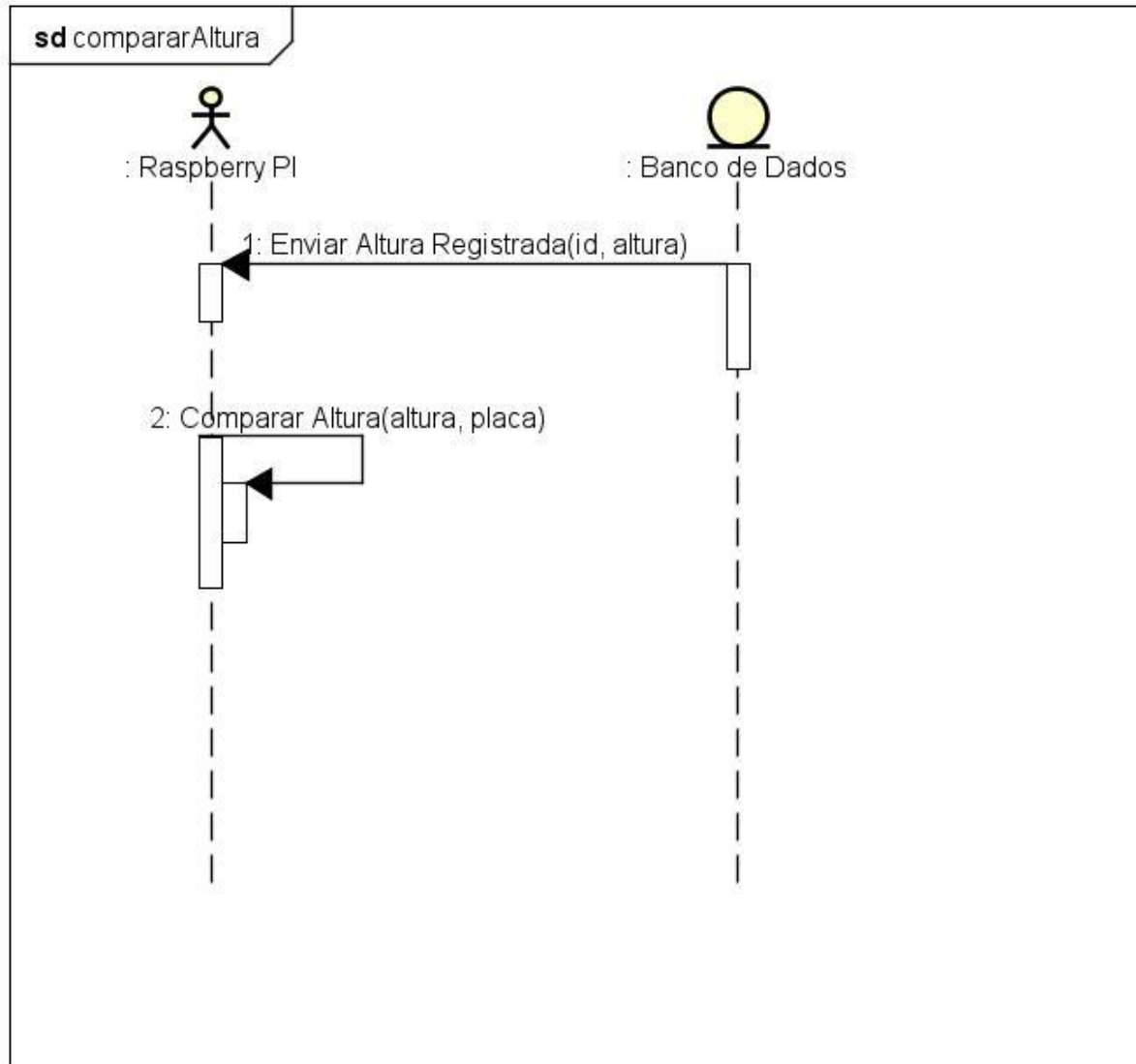
Fonte: Autoria Própria, 2025.

Figura 38 - Diagrama de Sequência: Retirar Altura



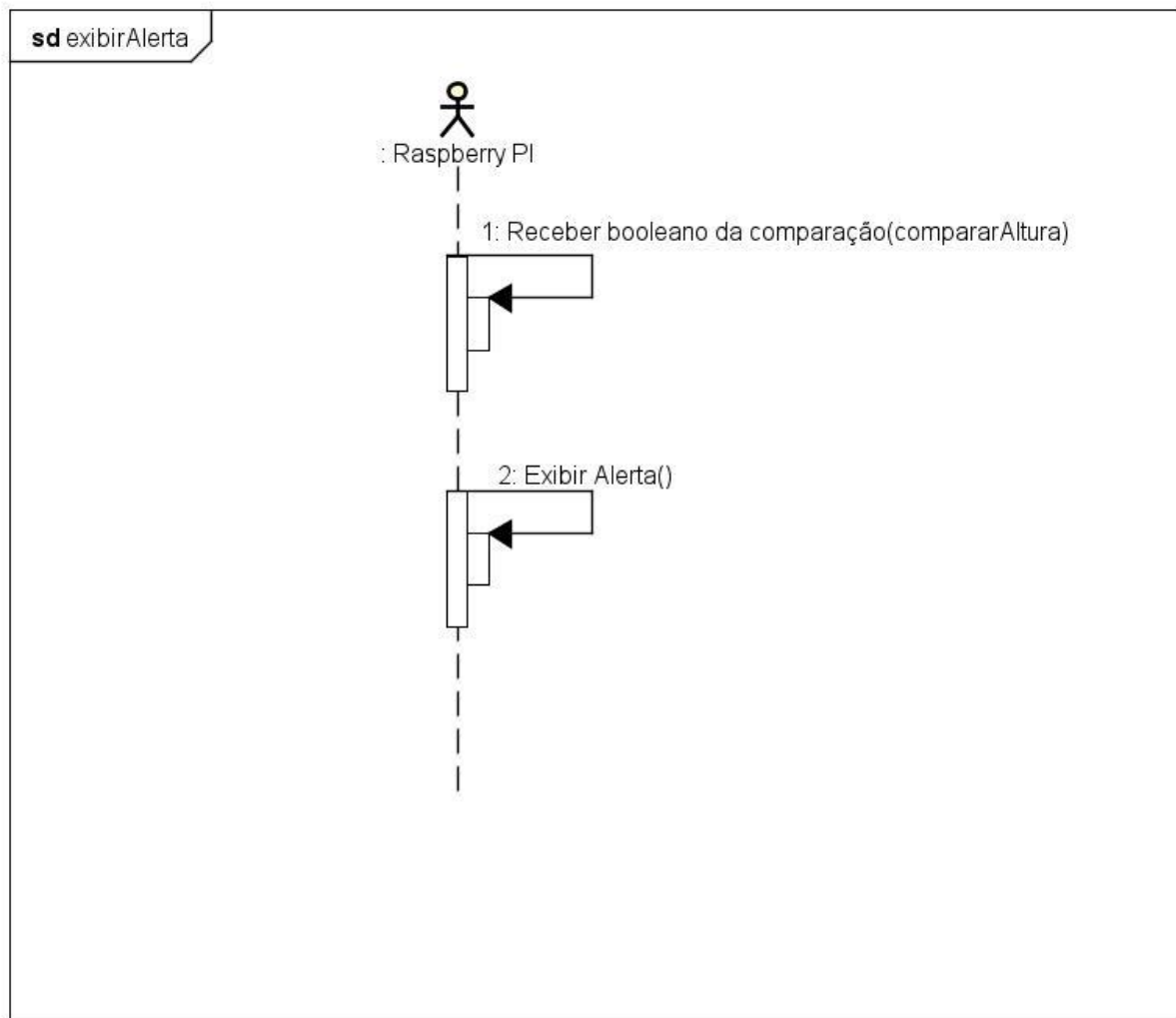
Fonte: Autoria Própria, 2025.

Figura 39 - Diagrama de Sequência: Comparar Altura



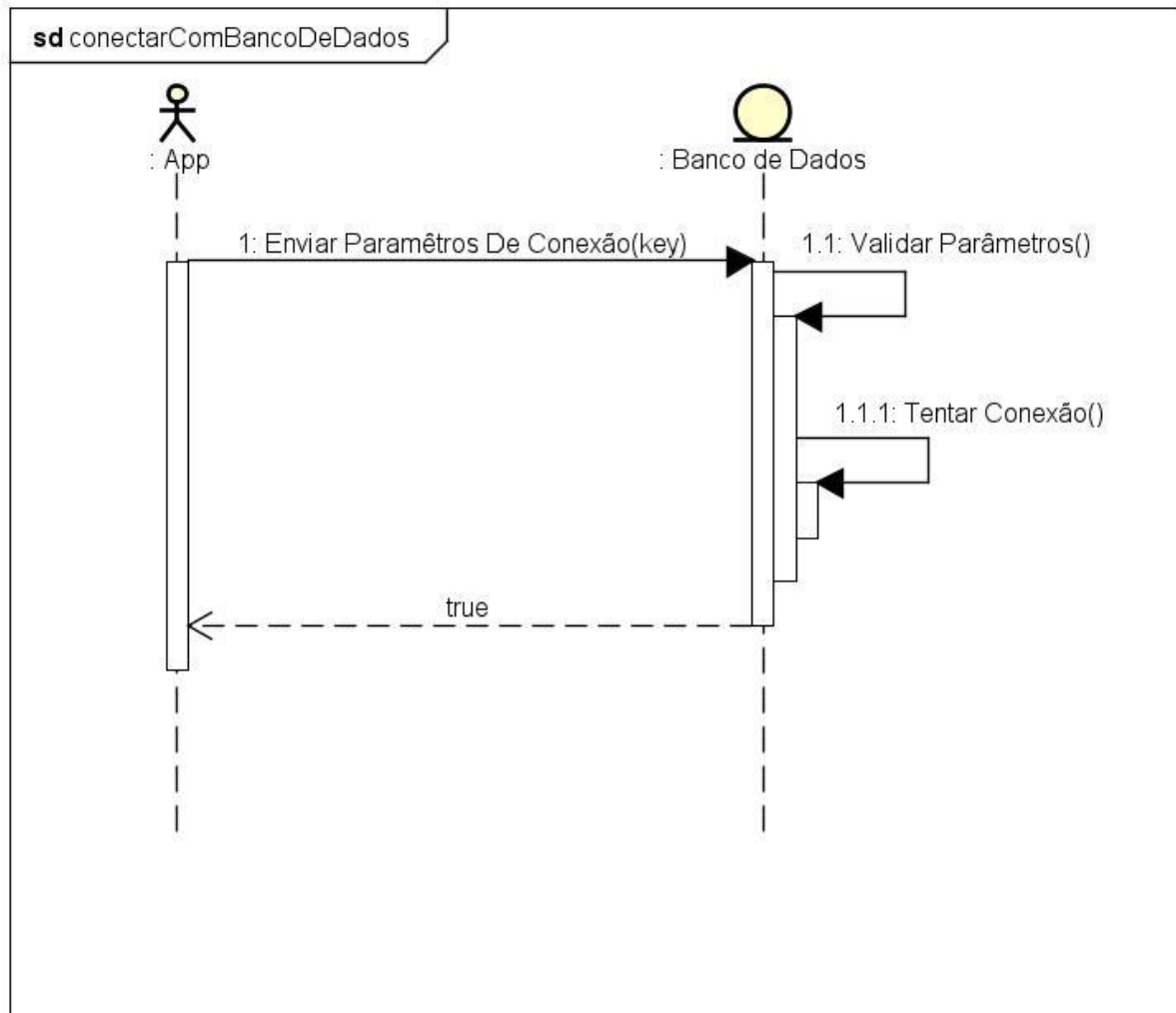
Fonte: Autoria Própria, 2025.

Figura 40 - Diagrama de Sequência: Exibir Alerta



Fonte: Autoria Própria, 2025.

Figura 41 - Diagrama de Sequência: Conectar Com o Banco de Dados



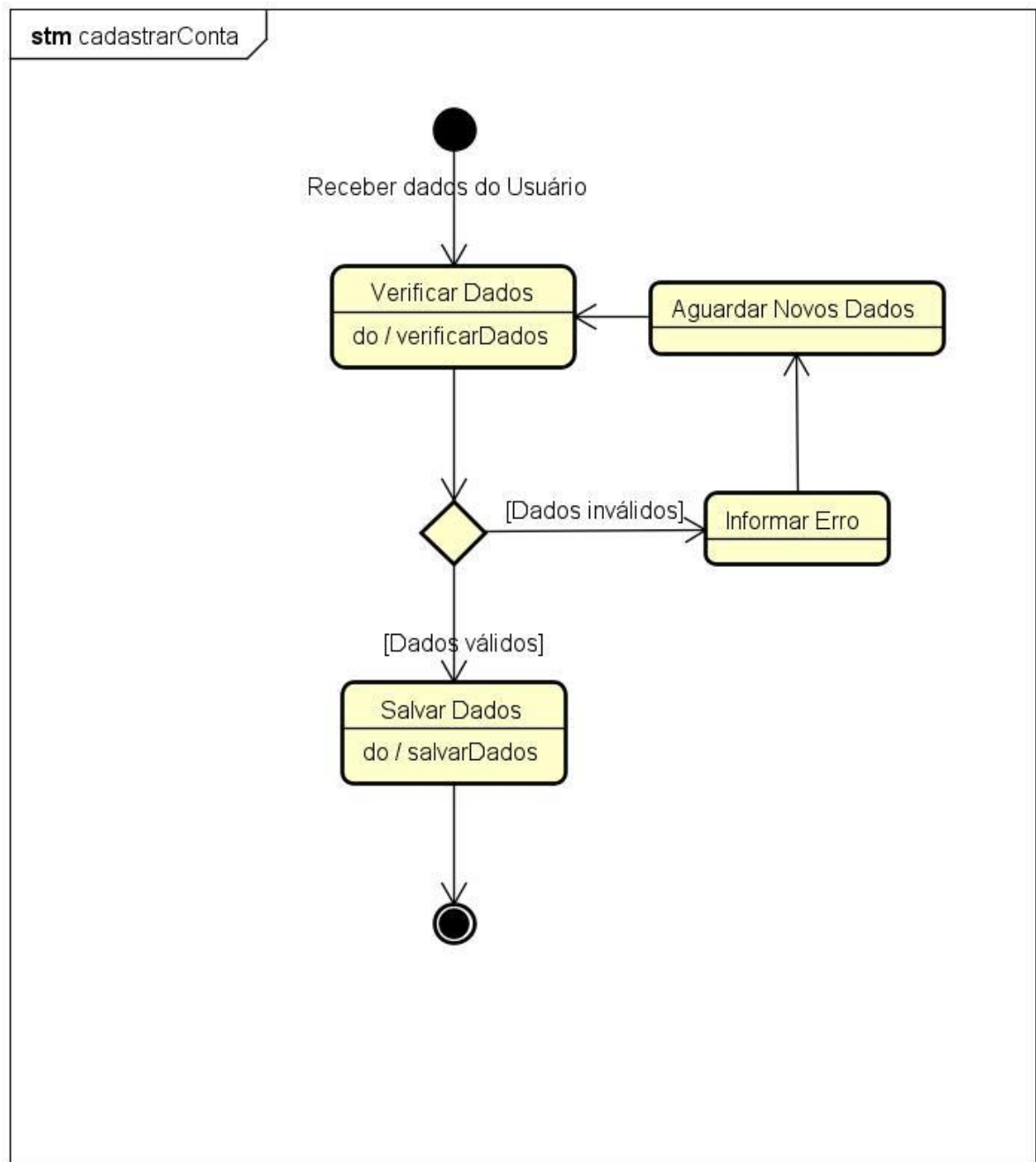
Fonte: Autoria Própria, 2025.

3.4 Diagrama de Máquina de Estado

O diagrama de máquina de estado é empregado para descrever o comportamento do sistema, em função da representação dos diferentes estados que um objeto pode assumir ao longo do tempo diante da ocorrência de um evento.

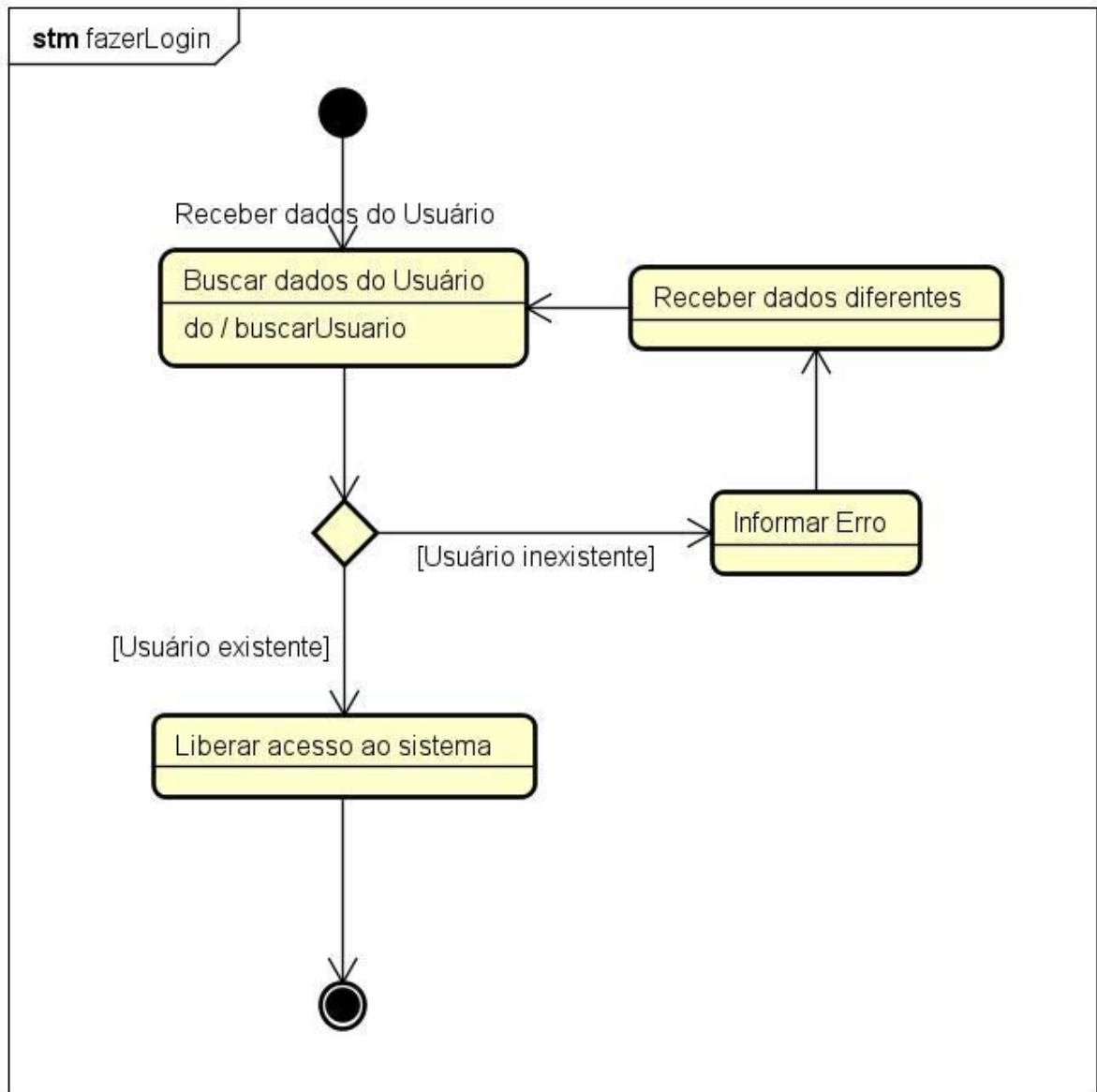
Ademais, este diagrama mostra como o sistema reage a determinadas ações, retratando as transições que acontecem durante a execução dos processos.

Figura 42 - Diagrama de Máquina de Estado: Cadastrar Conta



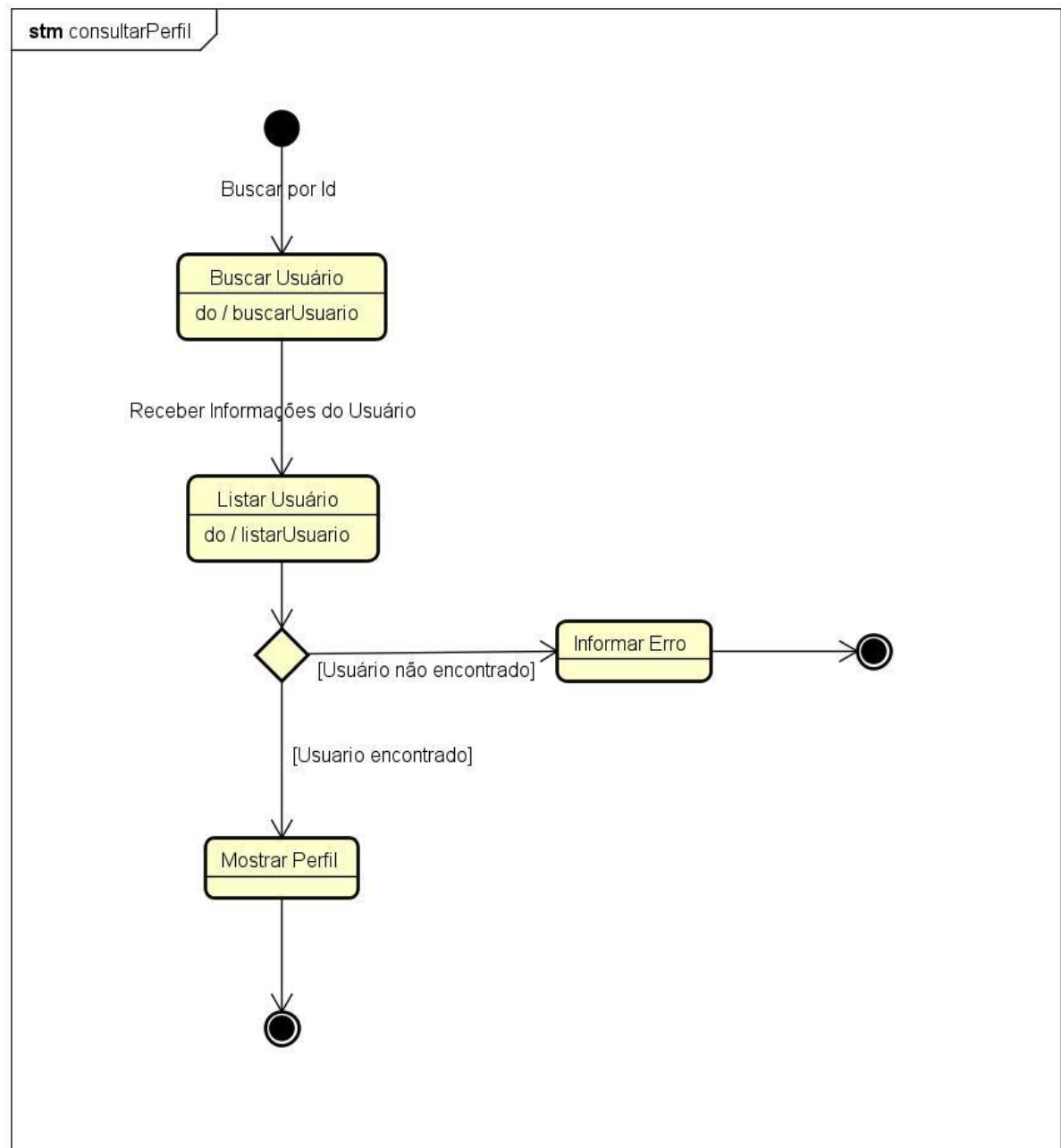
Fonte: Autoria Própria, 2025.

Figura 43 - Diagrama de Máquina de Estado: Fazer Login



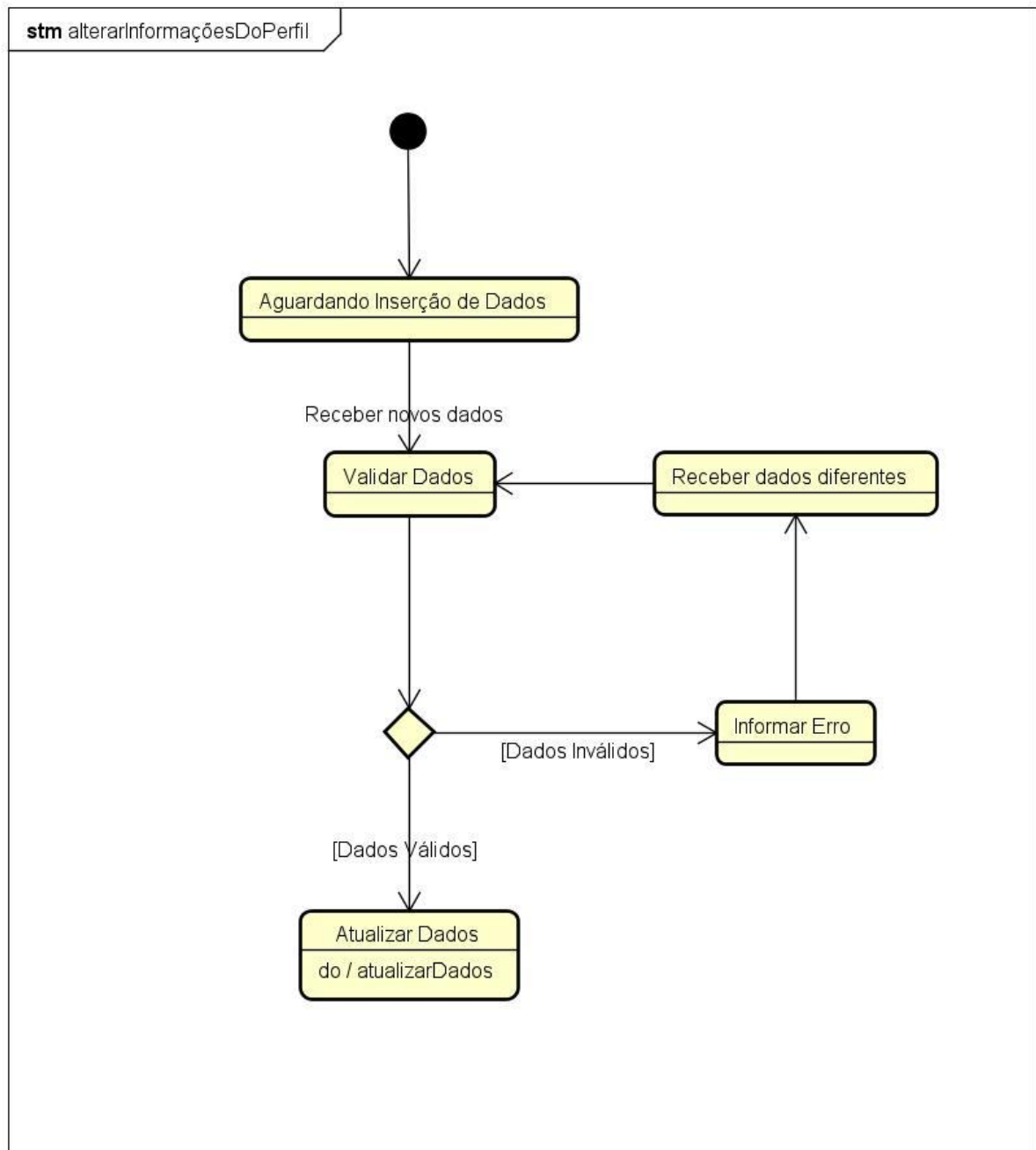
Fonte: Autoria Própria, 2025.

Figura 44 - Diagrama de Máquina de Estado: Consultar Perfil



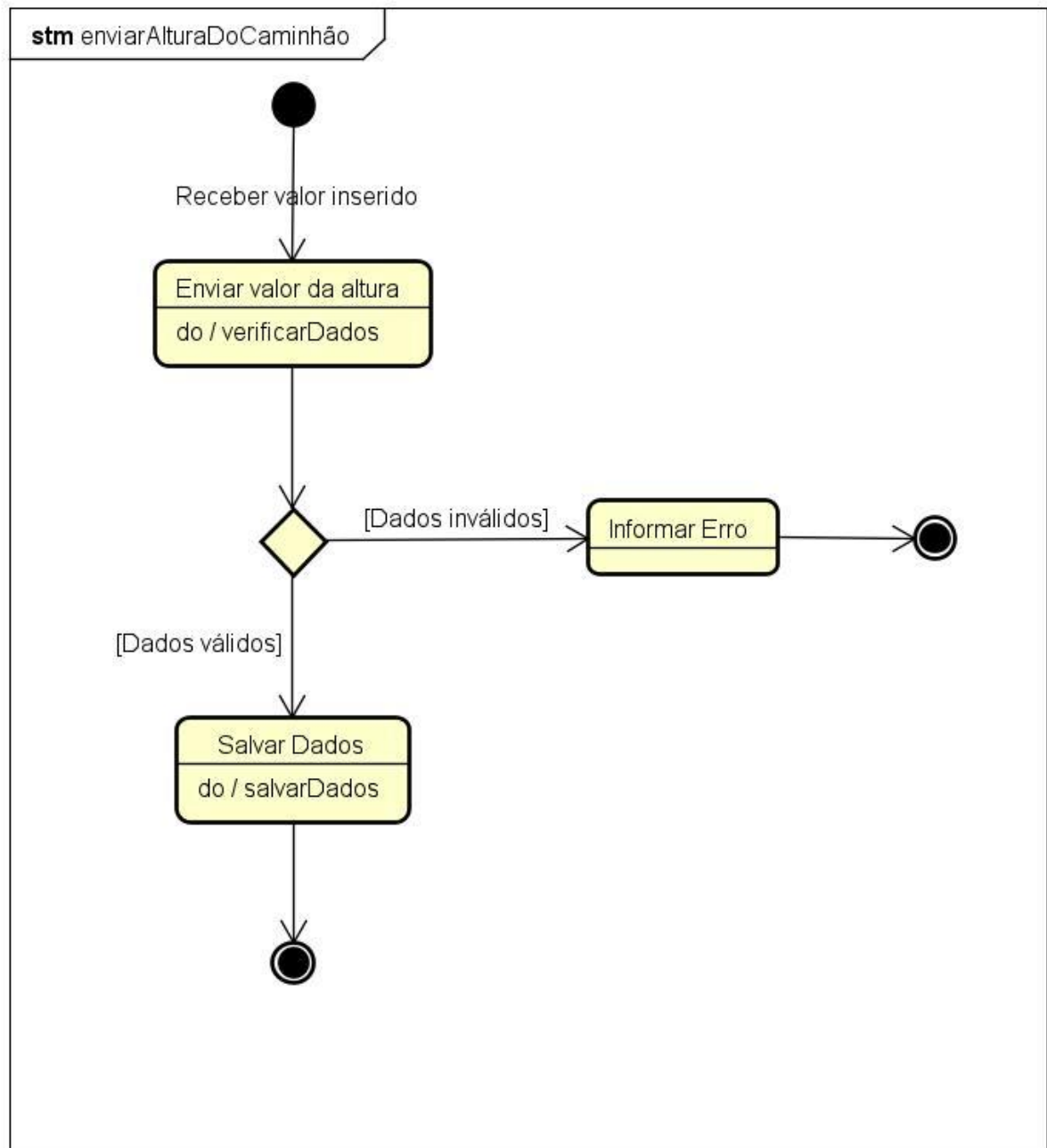
Fonte: Autoria Própria, 2025.

Figura 45 - Diagrama de Máquina de Estado: Alterar Informações do Perfil



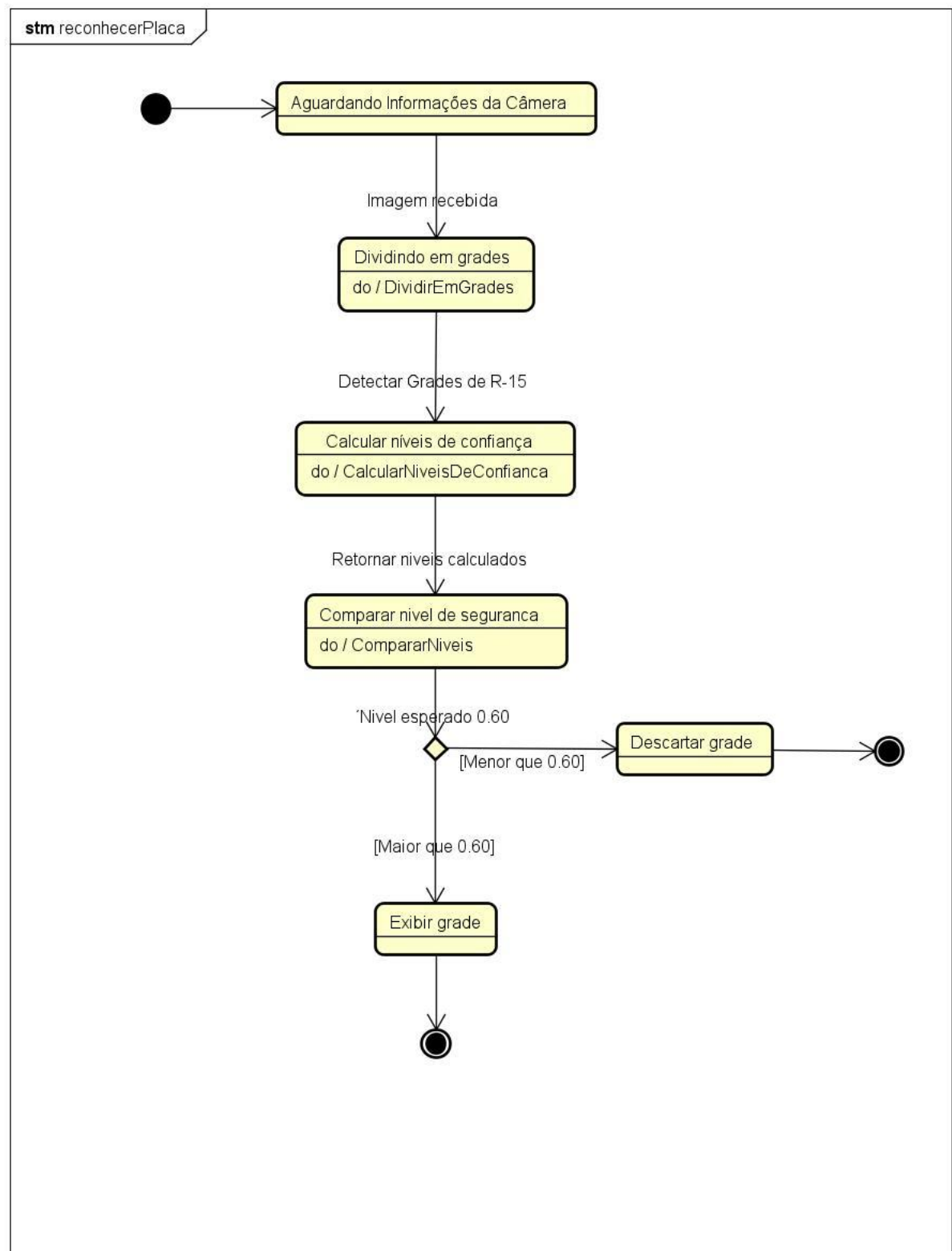
Fonte: Autoria Própria, 2025.

Figura 46 - Diagrama de Máquina de Estado: Enviar Altura do Caminhão



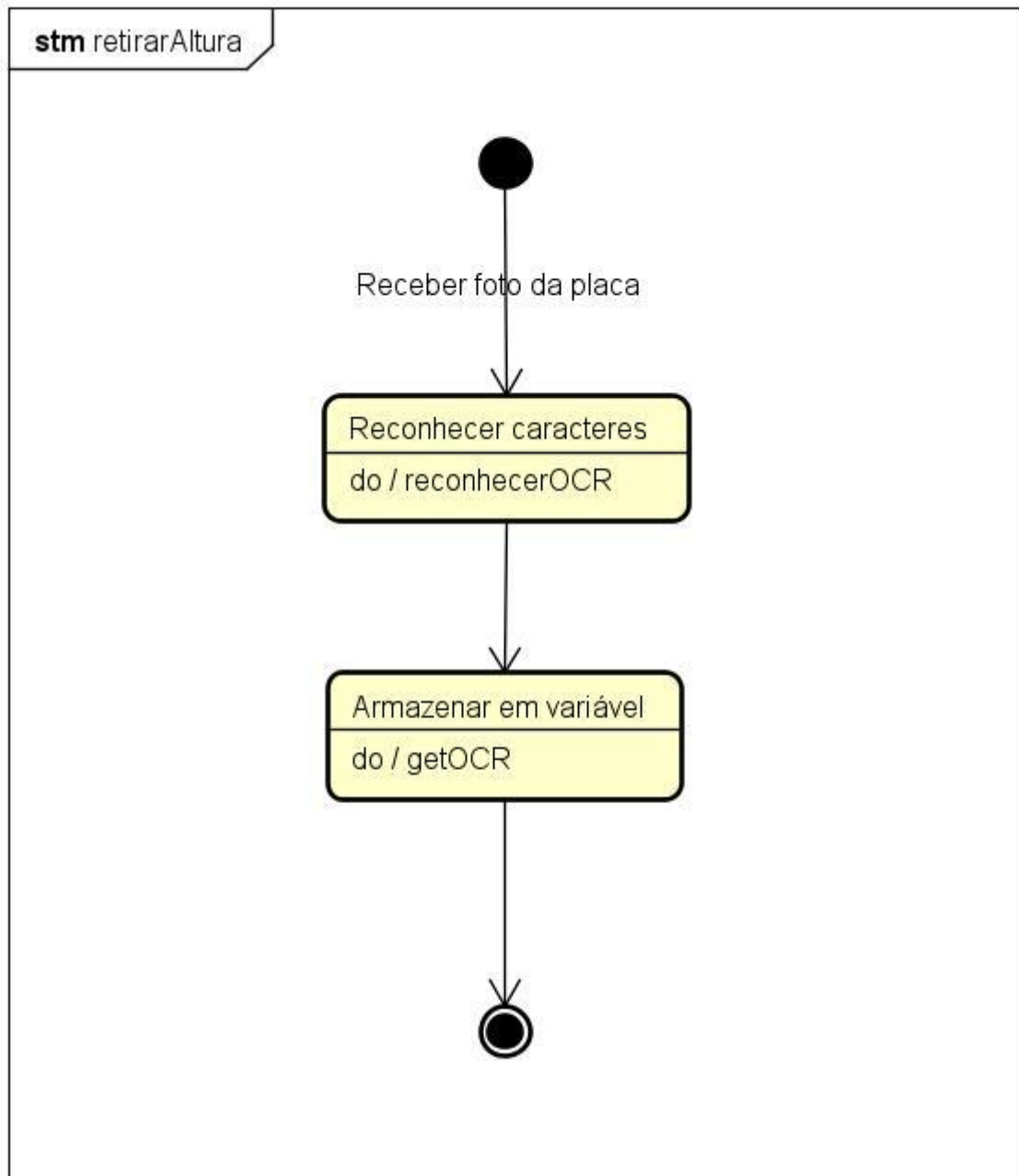
Fonte: Autoria Própria, 2025.

Figura 47 - Diagrama de Máquina de Estado: Reconhecer Placa



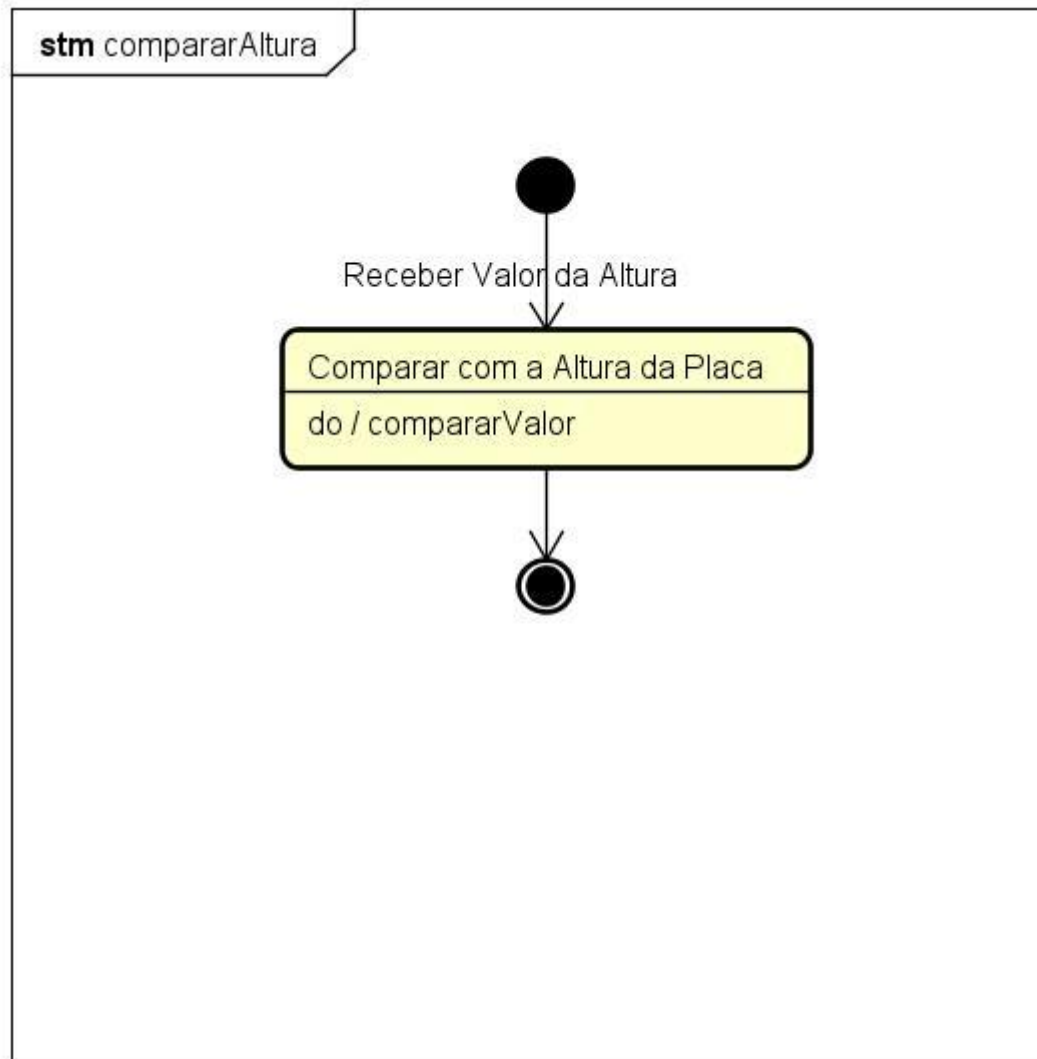
Fonte: Autoria Própria, 2025.

Figura 48 - Diagrama de Máquina de Estado: Retirar Altura



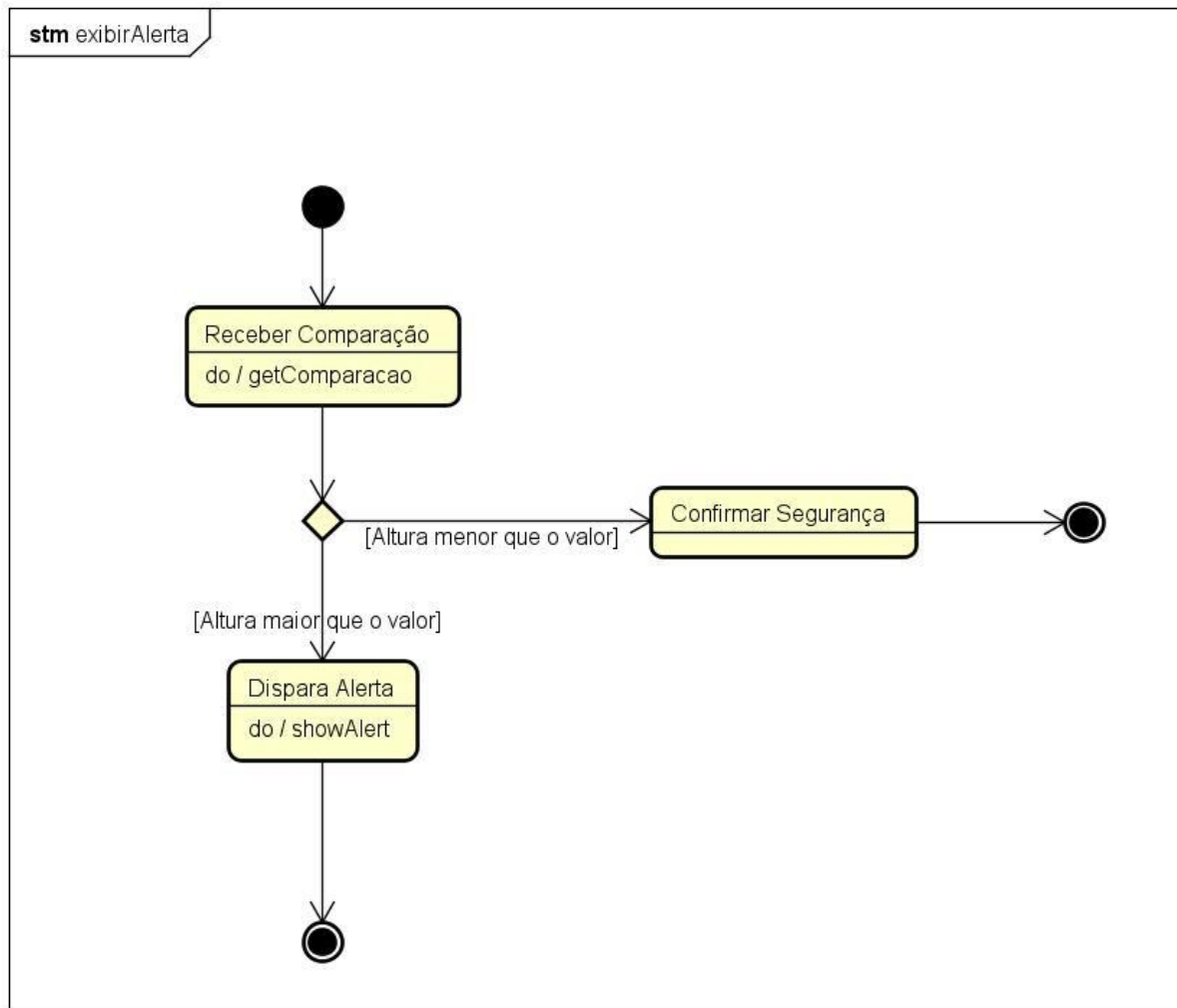
Fonte: Autoria Própria, 2025.

Figura 49 - Diagrama de Máquina de Estado: Comparar Altura



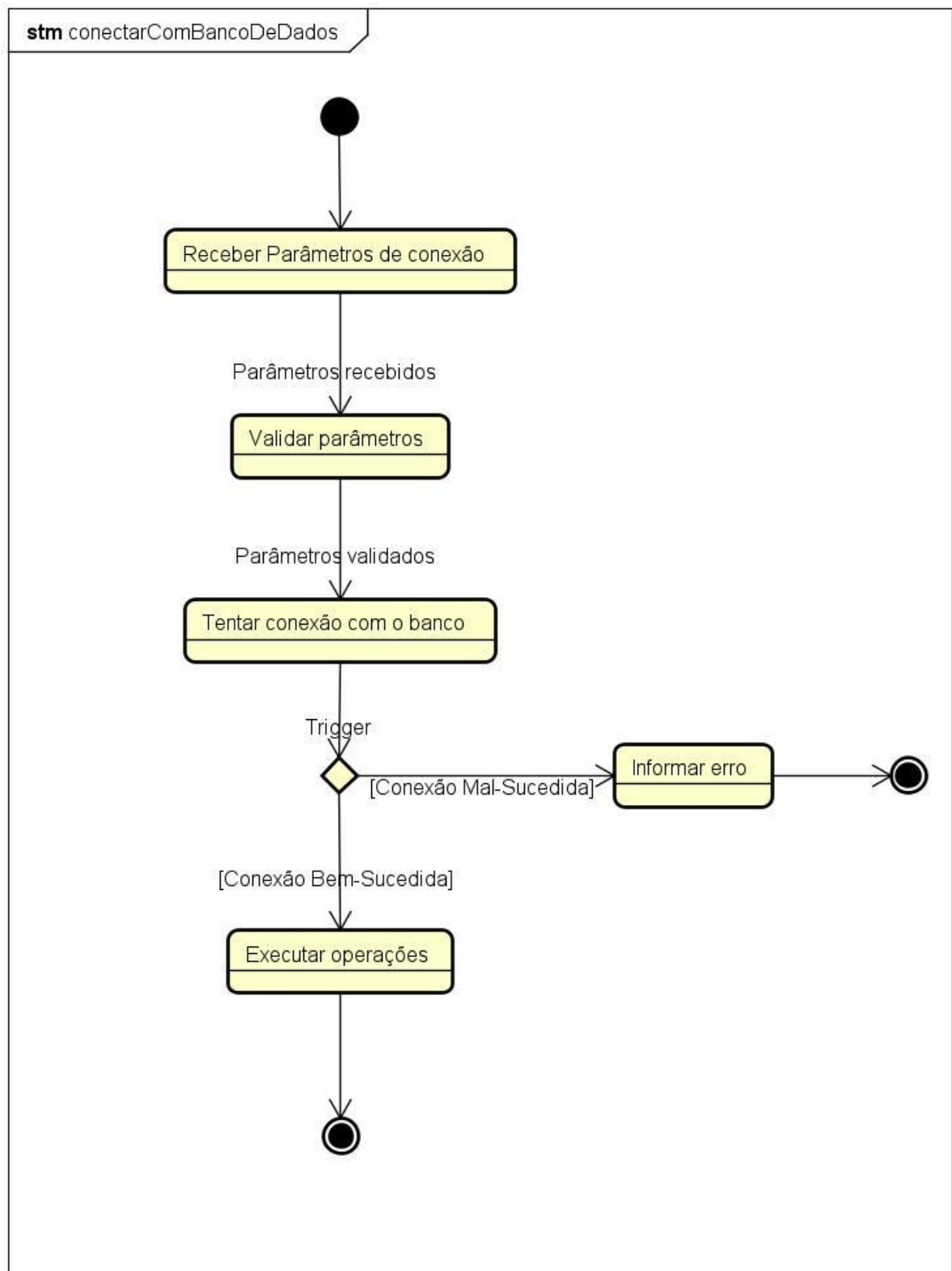
Fonte: Autoria Própria, 2025.

Figura 50 - Diagrama de Máquina de Estado: Exibir Alerta



Fonte: Autoria Própria, 2025.

Figura 51 - Diagrama de Máquina de Estado: Conectar Com o Banco de Dados



Fonte: Autoria Própria, 2025.

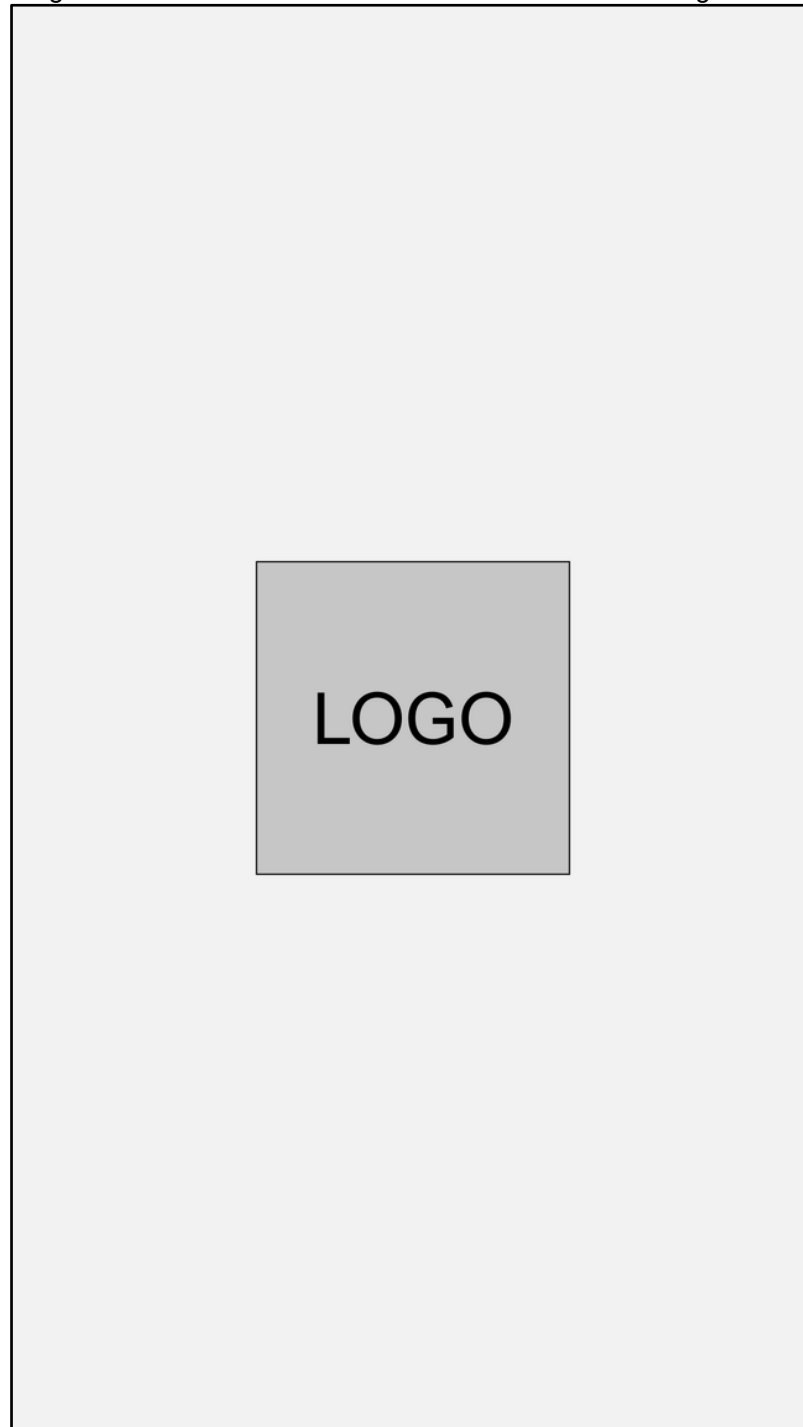
3.5 Prototipação das telas do aplicativo

Esta seção tem como principal objetivo apresentar a prototipação das interfaces gráficas contendo os elementos visuais do aplicativo, as quais foram desenvolvidas por meio de uma técnica de design que consiste em desenhar esboços visuais conhecida como *wireframes* de baixa e alta fidelidade, utilizando recursos visuais para deixar a experiência do usuário mais intuitiva, objetiva, agradável e confortável, que atenda o público-alvo.

O primeiro protótipo de interface desenvolvido foi a tela de carregamento do aplicativo mobile, na qual a logo da marca foi posicionada de forma centralizada, ganhando destaque através de sua cor e de um fundo neutro, que realça ainda mais o logotipo, transmitindo suavidade.

O esboço inicial da tela de carregamento, conhecido como wireframe de baixa fidelidade, não tem a finalidade de foco em cores, estilos ou detalhes gráficos, e sim criar a ideia da estrutura gráfica onde os elementos principais ficaram posicionados:

Figura 52 - Wireframe de Baixa Fidelidade: Tela de carregamento



Fonte: Autoria Própria, 2025.

Em seguida, pode-se observar o wireframe de alta fidelidade, que já contém cores e formas, aproximando-se do design final da tela de carregamento da aplicação mobile:

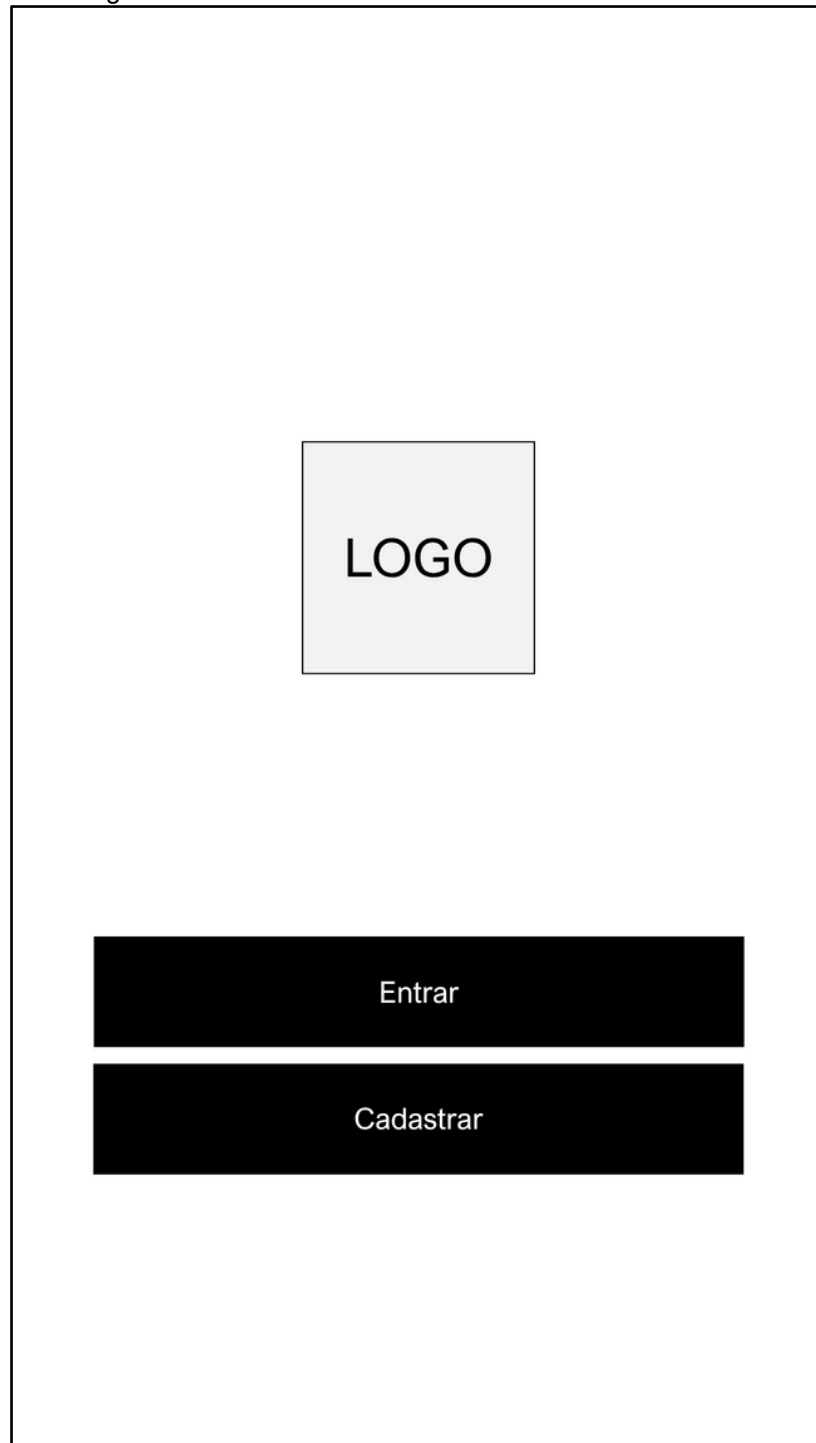
Figura 53 - Wireframe de Alta Fidelidade: Tela de carregamento



Fonte: Autoria Própria, 2025.

O segundo wireframe exibe a interface da tela inicial do aplicativo mobile, que contém como elementos principais dois botões: um que redireciona o usuário para a tela de login e outro para iniciar a etapa de cadastro, levando-o à tela de encontrar dispositivo. Para iniciar a demonstração, utilizamos um wireframe de baixa fidelidade, com o objetivo de esboçar e organizar os elementos principais necessários em um layout básico, sem componentes visuais detalhados:

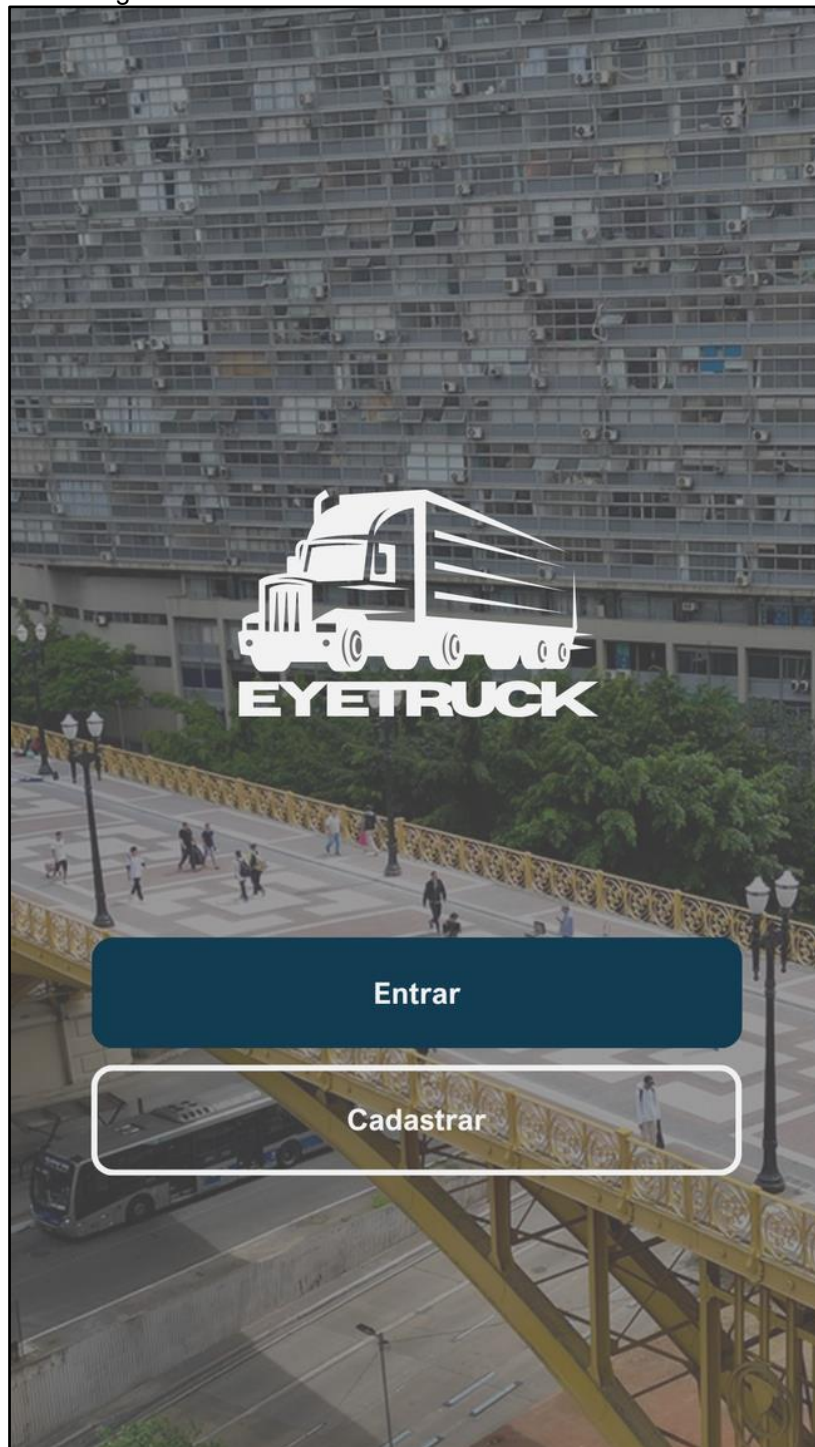
Figura 54 - Wireframe de Baixa Fidelidade: Tela de início



Fonte: Autoria Própria, 2025.

Na figura abaixo, é possível observar o protótipo de alta fidelidade da tela inicial do aplicativo, com um design moderno e detalhado, aplicando a identidade visual do projeto e assemelhando-se à versão final da interface.

Figura 55 - Wireframe de Alta Fidelidade: Tela de início

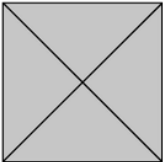


Fonte: Autoria Própria, 2025.

O próximo esboço visual apresenta o layout da tela de encontrar dispositivo, que possui uma breve descrição do objetivo da etapa e um campo de texto que deve ser preenchido com um código presente no aparelho, com a finalidade de localizá-lo no banco de dados. Caso o dispositivo não seja encontrado, exibe uma mensagem de erro e bloqueia a próxima etapa para cadastrar os dados do caminhoneiro. O layout é apresentado inicialmente por um wireframe de baixa fidelidade:

Figura 56 - Wireframe de Baixa Fidelidade: Tela de encontrar dispositivo

< Voltar



Encontrar Dispositivo

Digite o código presente no seu dispositivo

Código do dispositivo

Encontrar

Fonte: Autoria Própria, 2025.

Na sequência, é possível visualizar uma versão do wireframe de alta fidelidade, com elementos e componentes mais detalhados, proporcionando uma experiência mais agradável ao condutor do veículo:

Figura 57 - Wireframe de Alta Fidelidade: Tela de encontrar dispositivo

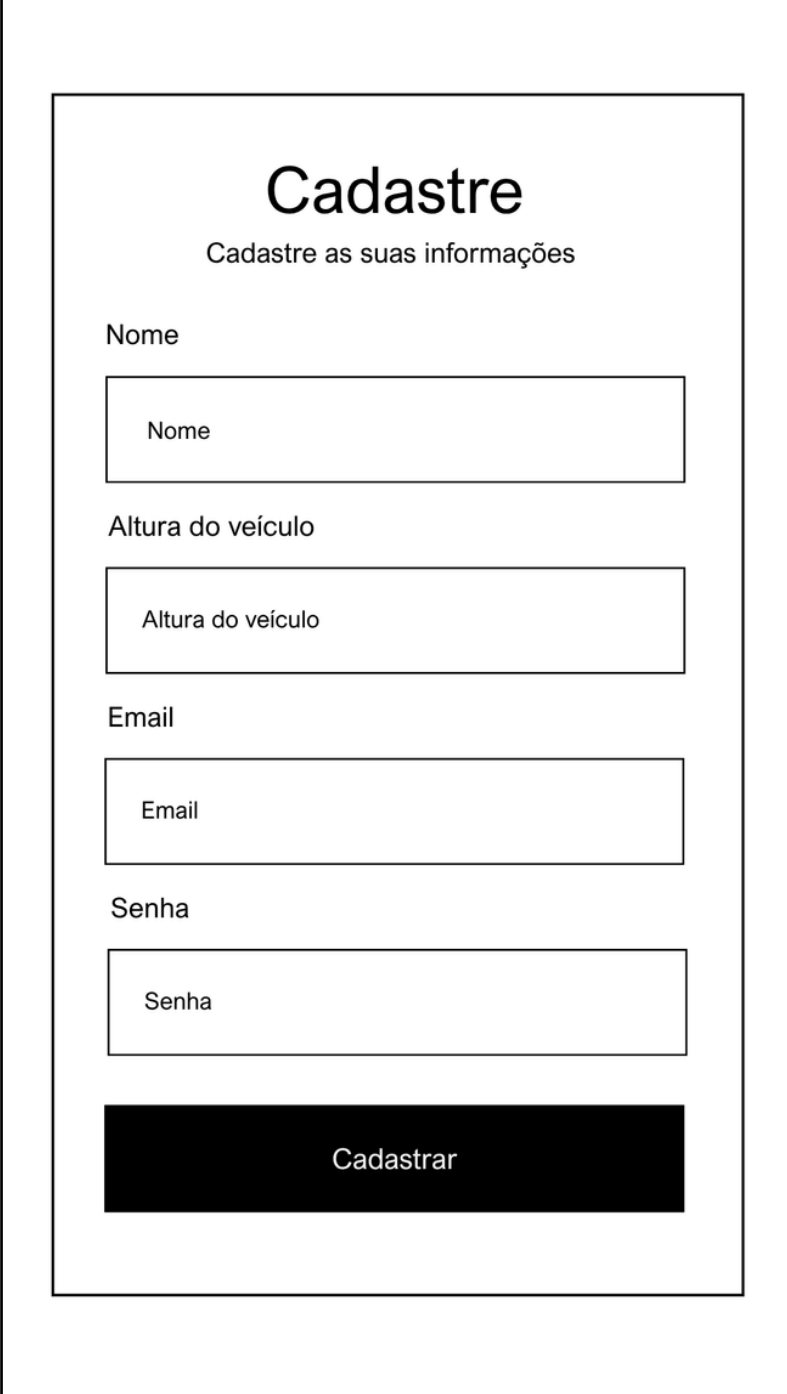


The wireframe shows a mobile application screen for finding a device. At the top left is a back arrow and the text "< Voltar". In the center is a circular logo with concentric arcs and a location pin. Below the logo is the title "Encontrar Dispositivo" in bold, followed by the instruction "Digite o código presente no seu dispositivo". Underneath is a label "Código do dispositivo" and a light gray input field containing the example text "Ex: 52200147". At the bottom is a dark blue button with the text "Encontrar".

Fonte: Autoria Própria, 2025.

O quarto protótipo se refere à tela de cadastro, composta por quatro campos de texto: nome, altura do caminhão, e-mail e senha. Esses dados devem ser fornecidos para que o cadastro do usuário seja realizado no sistema associado ao seu dispositivo. No entanto, é crucial que a etapa anterior, encontrar dispositivo, seja concluída com sucesso, possibilitando a efetivação do cadastro e o redirecionamento do usuário à tela de login. A seguir, apresenta-se o wireframe de baixa fidelidade para idealizar esquematização dos componentes na interface:

Figura 58 - Wireframe de Baixa Fidelidade: Tela de cadastro



The wireframe shows a registration screen with a central white box containing the title 'Cadastro' and subtitle 'Cadastre as suas informações'. Below these are four text input fields labeled 'Nome', 'Altura do veículo', 'Email', and 'Senha'. At the bottom is a black button labeled 'Cadastrar'.

```
graph TD; Title[Cadastro] --- Subtitle[Cadastre as suas informações]; Subtitle --- NomeLabel[Nome]; NomeLabel --- NomeInput[ ]; NomeInput --- AlturaLabel[Altura do veículo]; AlturaLabel --- AlturaInput[ ]; AlturaInput --- EmailLabel[Email]; EmailLabel --- EmailInput[ ]; EmailInput --- SenhaLabel[Senha]; SenhaLabel --- SenhaInput[ ]; SenhaInput --- Cadastrar[Cadastrar];
```

Fonte: Autoria Própria, 2025.

Logo após, é possível observar o wireframe de alta fidelidade, com elementos mais estruturados e melhor apresentados:

Figura 59 - Wireframe de Alta Fidelidade: Tela de cadastro

Cadastre

Cadastre as suas informações

Nome

Ex: João Silva

Altura do veículo

Ex: 3.75

Email

Ex: seuemail@gmail.com

Senha

Senha

Cadastrar

Fonte: Autoria Própria, 2025.

A seguir, apresentamos o quinto desenho visual, que mostra a interface da tela de login. Ela é composta por dois campos de texto principais, essenciais para a autenticação do usuário: e-mail e senha. Caso os dados não sejam encontrados, será exibida uma mensagem de erro; caso contrário, o usuário será redirecionado a home do aplicativo. Há, também, um link que, se clicado, levará o usuário para a primeira etapa do cadastro. Logo abaixo, é possível observar a esquematização da tela com elementos básicos, utilizando a técnica de design wireframe de baixa fidelidade.

Figura 60 - Wireframe de Baixa Fidelidade: Tela de login



The wireframe illustrates a login screen layout. It features a central container with a title 'Login', a subtitle 'Insira seus dados para acessar a sua conta', and two input fields labeled 'Email' and 'Senha'. Below these fields is a prominent black button labeled 'Entrar'. At the bottom of the container is a link that reads 'Não possui uma conta? Registre-se'.

Login

Insira seus dados para acessar a sua conta

Email

Email

Senha

Senha

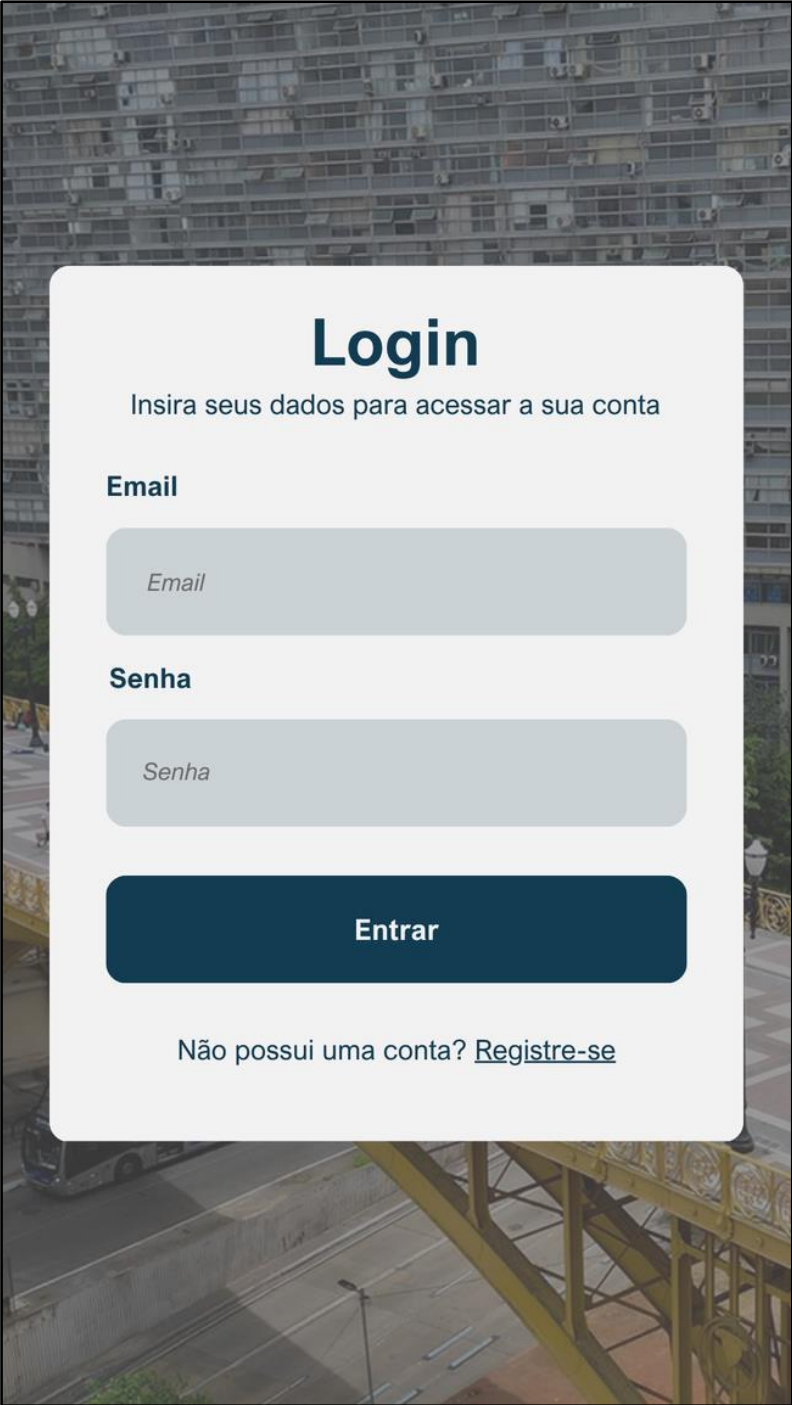
Entrar

Não possui uma conta? Registre-se

Fonte: Autoria Própria, 2025.

Posteriormente, é apresentado o wireframe de alta fidelidade, com um design confortável e detalhado, visando a melhor experiência do motorista:

Figura 61 - Wireframe de Alta Fidelidade: Tela de login

A wireframe de alta fidelidade para a tela de login, exibida sobre uma imagem de fundo de uma cidade. O formulário é centralizado e possui um fundo branco com bordas arredondadas. No topo, o título "Login" é exibido em uma fonte grande e escura. Abaixo dele, há uma instrução: "Insira seus dados para acessar a sua conta". O formulário contém dois campos de entrada: "Email" e "Senha", ambos com rótulos em negrito e placeholders correspondentes. Abaixo dos campos, há um botão "Entrar" em um tom escuro. Na base do formulário, há um link para registro: "Não possui uma conta? [Registre-se](#)".

Login

Insira seus dados para acessar a sua conta

Email

Email

Senha

Senha

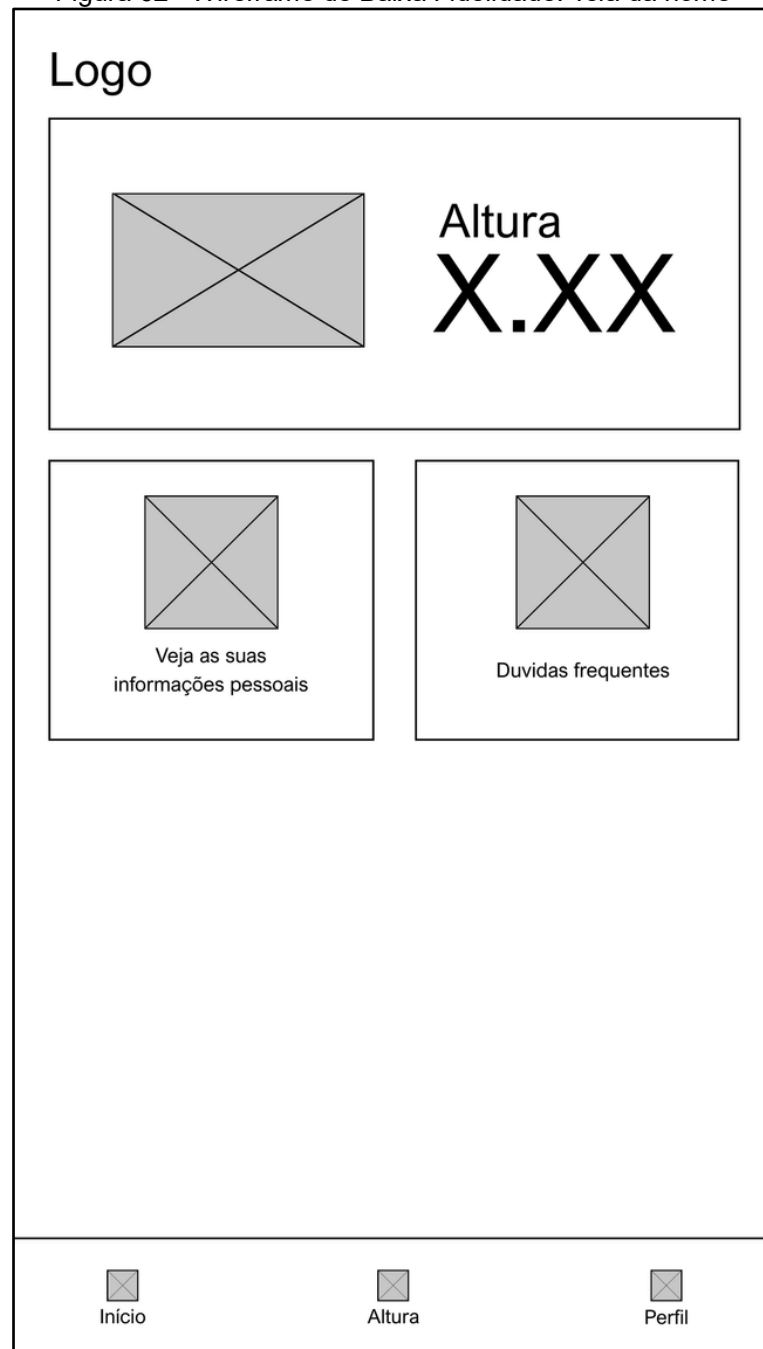
Entrar

Não possui uma conta? [Registre-se](#)

Fonte: Autoria Própria, 2025.

O sexto esboço exibe a interface que contém o dashboard da home do aplicativo, cujo acesso exige a autenticação do usuário. A interface possui três caixas de texto: a primeira mostra a altura do caminhão e, ao ser clicada, encaminha para a tela de edição da altura cadastrada; a segunda e a terceira são cards que, ao serem clicados, direcionam para as telas de edição de perfil e dúvidas frequentes, respectivamente. Na parte inferior da tela, encontra-se a barra de menu que facilita a navegação do usuário no aplicativo. Logo a seguir trata-se de um wireframe de baixa fidelidade, para melhor compreensão de onde os elementos são posicionados na tela:

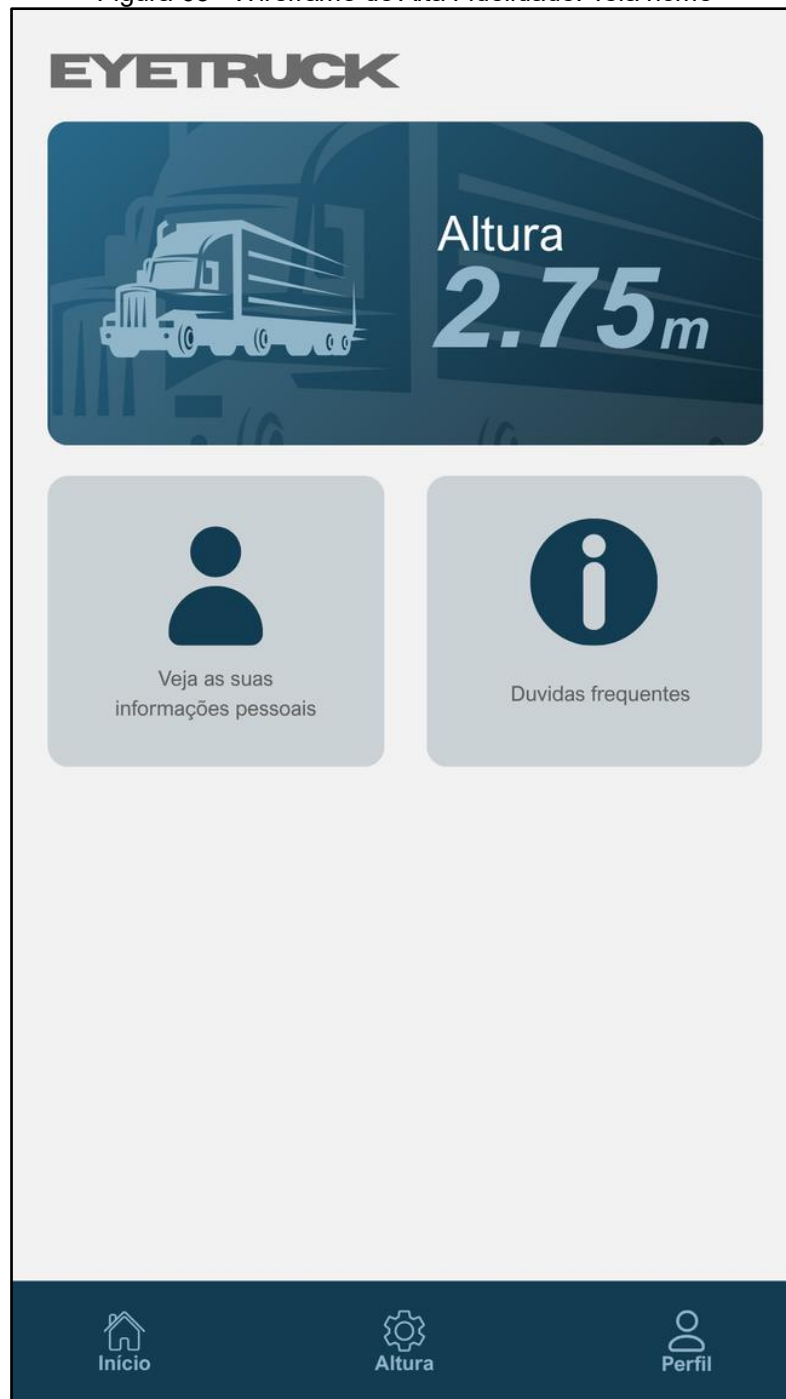
Figura 62 - Wireframe de Baixa Fidelidade: Tela da home



Fonte: Autoria Própria, 2025.

Na figura abaixo, é possível observar o wireframe de alta fidelidade da tela inicial, com um visual mais elaborado, aproximando-se do design final:

Figura 63 - Wireframe de Alta Fidelidade: Tela home



Fonte: Autoria Própria, 2025.

O próximo protótipo tem foco em demonstrar a tela de edição de altura. O elemento principal presente na tela é a caixa de texto, onde é possível visualizar e editar a altura do caminhão previamente cadastrada pelo usuário. A tela também fornece uma breve descrição e uma barra de navegação, permitindo a transição entre as telas. Logo abaixo, é possível visualizar um wireframe de baixa fidelidade da tela:

Figura 64 - Wireframe de Baixa Fidelidade: Tela de editar altura

Altura do Caminhão

Atualize caso necessário a altura do seu caminhão

Altura do caminhão

Altura do caminhão

Atualizar

Início Altura Perfil

Fonte: Autoria Própria, 2025.

Na imagem abaixo, apresenta-se o wireframe de alta fidelidade da tela de edição de altura, com um design mais refinado e alinhado à identidade visual do projeto, aproximando-se da interface final:

Figura 65 - Wireframe de Alta Fidelidade: Tela de editar altura



Altura do Caminhão

Atualize caso necessário a altura do seu caminhão

Altura do caminhão

2.75

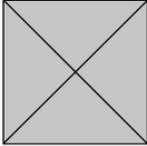
Atualizar

Início Altura Perfil

Fonte: Autoria Própria, 2025.

O sétimo esboço visual apresenta a tela de edição de perfil, que tem como principal objetivo fornecer os dados do usuário extraídos do banco de dados e permitir a atualização dessas informações, caso necessário. A interface é composta por campos de texto e um botão para salvar as alterações. Também é possível observar a opção de sair da conta, caso o motorista veja essa necessidade. A seguir é demonstrado o wireframe de baixa fidelidade com apenas os itens principais e sem estilização:

Figura 66 - Wireframe de Baixa Fidelidade: Tela de editar perfil



Editar Perfil

Atualize caso necessário as suas informações pessoais

Nome

Email

Salvar **Sair**

 Início  Altura  Perfil

Fonte: Autoria Própria, 2025.

Em continuidade, é apresentado o wireframe de alta fidelidade, com os principais elementos estilizados e visualmente atrativos, aproximando-se da interface final:

Figura 67 - Wireframe de Alta Fidelidade: Tela de editar perfil

Fonte: Autoria Própria, 2025.

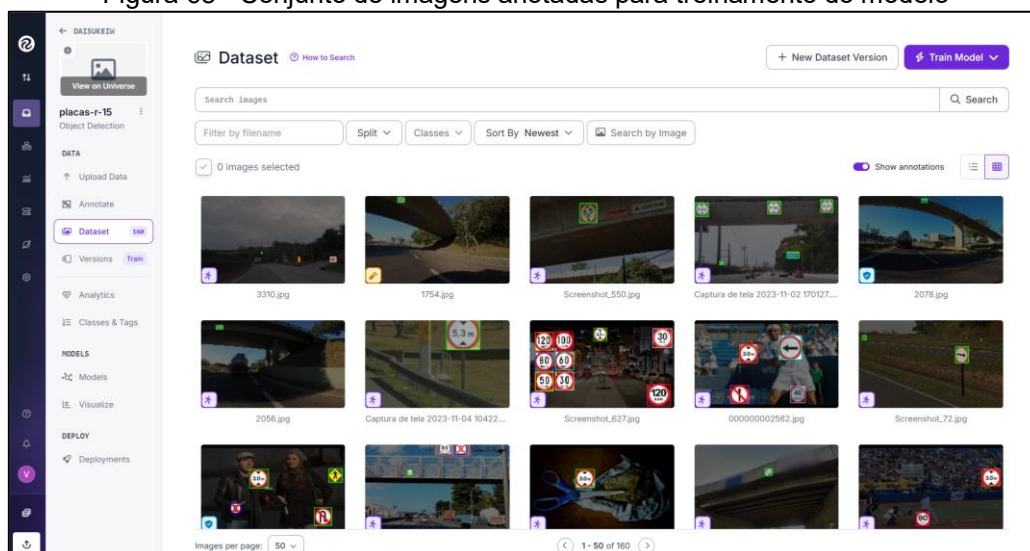
3.6 Desenvolvimento do hardware de detecção de placas

Nesta seção, são apresentadas as etapas que compõem o desenvolvimento do hardware do EyeTruck, responsável pela captura da imagem, detecção da placa R-15 e extração da altura máxima permitida.

Para viabilizar o processamento das imagens, utilizou-se o Raspberry Pi, escolhido por sua relação custo-benefício e por apresentar capacidade de processamento suficiente para executar o YOLO. Esse microcomputador se mostrou adequado para suprir as necessidades do projeto, permitindo a execução do modelo de detecção de objetos, integração com bibliotecas de processamento de imagens e reconhecimento óptico de caracteres.

Inicialmente, foi construído um dataset, e as imagens foram anotadas para o treinamento do modelo através da plataforma Roboflow. Ao todo, foi realizada a anotação de 160 imagens das placas R-15 e A-37, ambas relacionadas à limitação de altura.

Figura 68 - Conjunto de imagens anotadas para treinamento do modelo



Fonte: Autoria Própria, 2025.

Após a preparação do dataset, iniciou-se o processo de treinamento utilizando Python e a biblioteca Ultralytics. O modelo YOLOv8n foi carregado como base para o treinamento, e os parâmetros essenciais foram definidos conforme as necessidades do projeto. O treinamento realizou cinquenta ciclos completos sobre o conjunto de dados preparado, produzindo um modelo otimizado, cuja finalidade é identificar a placa R-15 com elevada confiabilidade.

Figura 69 – Treinamento do modelo

```

3 # Carrega o modelo YOLOv8n
4 model = YOLO('yolov8n.pt')
5
6 # Treina o modelo com o dataset
7 results = model.train(
8     data=r'C:\Users\vinic\Documents\A_deteccao_r15\placas-r-15.v1i.yolov8\data.yaml',
9     epochs=50,
10    imgsz=640,
11    batch=16,
12    name='placas_r15_model4'
13 )
14
15 print("Treinamento concluído!")
16 print(f"Modelo salvo em: {model.trainer.save_dir}")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\vinic\Documents\A_deteccao_r15> C:\Users\vinic\Documents\A_deteccao_r15\venv\Scripts\python.exe train_yolo.py

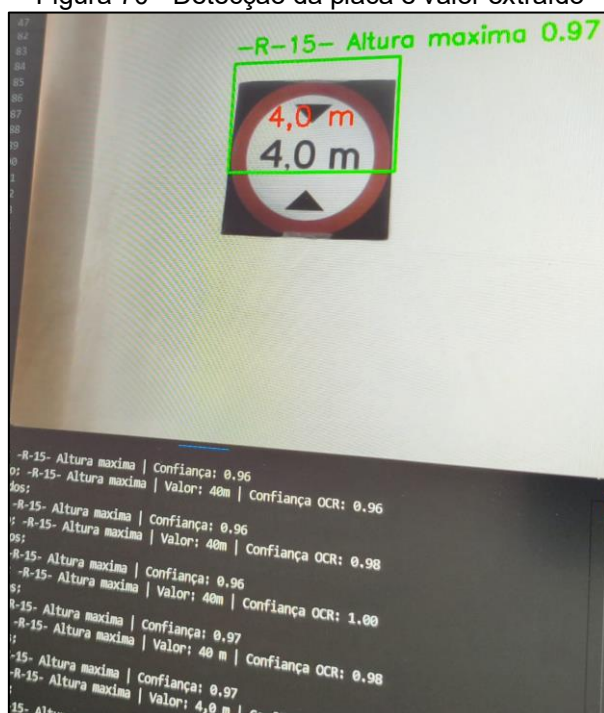
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
7/50	6G	0.622	3.119	0.8367	62	640: 0%
7/50	6G	0.6546	2.897	0.8725	51	640: 14%
7/50	6G	0.6788	2.99	0.8688	62	640: 25%
7/50	6G	0.6929	3.073	0.8926	70	640: 43%
7/50	6G	0.6995	3.115	0.897	56	640: 57%
7/50	6G	0.6983	3.136	0.8989	56	640: 71%
7/50	6G	0.6819	3.098	0.8932	41	640: 86%
7/50	6G	0.6819	3.098	0.8932	41	640: 100%
7/50	6G	0.6819	3.098	0.8932	41	640: 100%
7/7 0.31t/s 23.2s						
		Class	Images	Instances	Box(P)	R
		Class	Images	Instances	Box(P)	R
		Class	Images	Instances	Box(P)	R
		all	37	85	0.8363	0.847
					0.8432	0.8367
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
8/50	6G	0.7709	3.347	0.8097	67	640: 0%
8/50	6G	0.7259	3.127	0.8722	69	640: 14%
8/50	6G	0.7206	3.061	0.8787	54	640: 25%
8/50	6G	0.7004	2.996	0.8808	48	640: 43%
3/7 0.21t/s 14.2s 22.7s						

Fonte: Autoria Própria, 2025.

Durante a execução do código, o terminal exibiu o progresso do treinamento, incluindo métricas, resultados e o diretório de salvamento dos arquivos finais. Esse processo permitiu avaliar o desempenho do modelo e verificar a consistência dos pesos gerados.

Com o modelo de detecção treinado, o próximo passo foi integrar uma etapa de reconhecimento óptico de caracteres para extrair o valor numérico presente na sinalização. Para isso, adotou-se a biblioteca EasyOCR, responsável por identificar o texto na região da placa previamente detectada.

Figura 70 - Detecção da placa e valor extraído



Fonte: Autoria Própria, 2025.

A imagem acima apresenta um teste realizado com a placa R-15 impressa, no qual o modelo YOLO identificou corretamente a sinalização na imagem capturada pela câmera acoplada ao Raspberry Pi. Nesse cenário, o modelo atribuiu uma confiança de 0,97 à detecção, além de permitir a extração dos caracteres “4,0 m” por meio do módulo de reconhecimento óptico de caracteres, demonstrando o funcionamento integrado entre detecção de objeto e leitura de texto.

Figura 71 - Alerta visual exibido ao motorista



Fonte: Autoria Própria, 2025.

Em seguida, o valor de altura extraído da placa é comparado com a altura do caminhão cadastrada no banco de dados. Quando a altura do veículo excede o limite

informado na sinalização, o sistema exibe um alerta visual na tela, com a mensagem “NÃO PODE PASSAR”, conforme ilustrado na Figura 71. Nessa interface, “P” representa o valor de altura obtido a partir da placa e “C” representa a altura do caminhão registrada no Firebase. Em um cenário em que a altura do caminhão fosse menor ao valor indicado na placa, a mensagem exibida ao condutor seria “PODE PASSAR”.

Figura 72 - Trecho de código responsável pela comparação das alturas

```

36 def get_altura_caminhao():
37     """Busca a altura do caminhão no Firebase"""
38     try:
39         # Inicializa Firebase se ainda não foi inicializado
40         if not firebase_admin._apps:
41             cred = credentials.Certificate(FIREBASE_CRED_PATH)
42             firebase_admin.initialize_app(cred)
43
44         # Conecta ao Firestore
45         db = firestore.client()
46
47         # Busca o documento do dispositivo
48         doc_ref = db.collection('tbl_dispositivos').document(DISPOSITIVO_ID)
49         doc = doc_ref.get()
50
51         if doc.exists:
52             dados = doc.to_dict()
53             altura = dados.get('alturaCaminhao')
54             print(f"📄 Altura do caminhão (Firebase): {altura}m")
55             return float(altura) if altura else None
56         else:
57             print("❌ Documento não encontrado no Firebase")
58             return None
59     except Exception as e:
60         print(f"❌ Erro ao buscar altura do caminhão: {e}")
61         return None
62
63 def comparar_alturas(altura_placa, altura_caminhao):
64     """Compara altura da placa com altura do caminhão e gera alerta se necessário"""
65     if altura_placa is None or altura_caminhao is None:
66         return False
67
68     if altura_placa < altura_caminhao:
69         print(f"⚠️ ALERTA: PARE O CAMINHÃO! ⚠️")
70         print(f"📏 Altura da placa: {altura_placa}m")
71         print(f"📏 Altura do caminhão: {altura_caminhao}m")
72         print(f"⚠️ PASSAGEM MUITO BAIXA! ⚠️")
73         return True
74     else:
75         print(f"✅ Passagem segura - Placa: {altura_placa}m / Caminhão: {altura_caminhao}m")
76         return False

```

Fonte: Autoria Própria, 2025.

No algoritmo mostrado na figura 72, a função `get_altura_caminhao()` é responsável por buscar a altura do caminhão armazenada no Firestore. Essa função realiza a inicialização do Firebase, acessa a coleção definida para os dispositivos, localiza o documento correspondente e extrai o campo referente à altura cadastrada. Caso o documento não seja encontrado ou ocorra alguma exceção, o código exibe mensagens de erro no terminal e retorna um valor nulo, garantindo o tratamento adequado das falhas de comunicação. Em complemento, uma segunda função verifica se tanto a altura detectada na placa quanto a altura do caminhão estão disponíveis; se algum desses valores estiver ausente, a comparação não é realizada. Quando ambas as alturas estão presentes, o código avalia se a altura do caminhão é maior do que a altura indicada na placa. Em situação de risco, são exibidas

mensagens informando que a passagem é muito baixa e que o veículo não deve prosseguir; nos demais casos, o sistema indica que a passagem é segura.

Além do alerta visual, foi incluído um alerta sonoro utilizando um buzzer conectado ao hardware, a fim de reforçar a atenção do motorista. O funcionamento desse componente segue dois cenários. O primeiro cenário se dá pela passagem permitida, sendo emitido um único sinal sonoro; já no segundo cenário, em que a altura do caminhão ultrapassa o limite permitido pela placa, são emitidos três sinais sonoros consecutivos. Essa combinação foi pensada considerando que muitos acidentes acontecem devido a desatenção.

Figura 73 - Carregador portátil responsável pela alimentação elétrica



Fonte: Autoria Própria, 2025.

Para garantir a alimentação elétrica adequada, optou-se pelo uso de um carregador portátil com capacidade suficiente para suprir as necessidades energéticas do sistema embarcado e de seus componentes. Essa solução de energia foi considerada adequada ao projeto, pois proporciona autonomia, reduz a dependência de fontes externas e minimiza problemas relacionados a carregamento, durabilidade e desempenho do hardware durante o uso do EyeTruck.

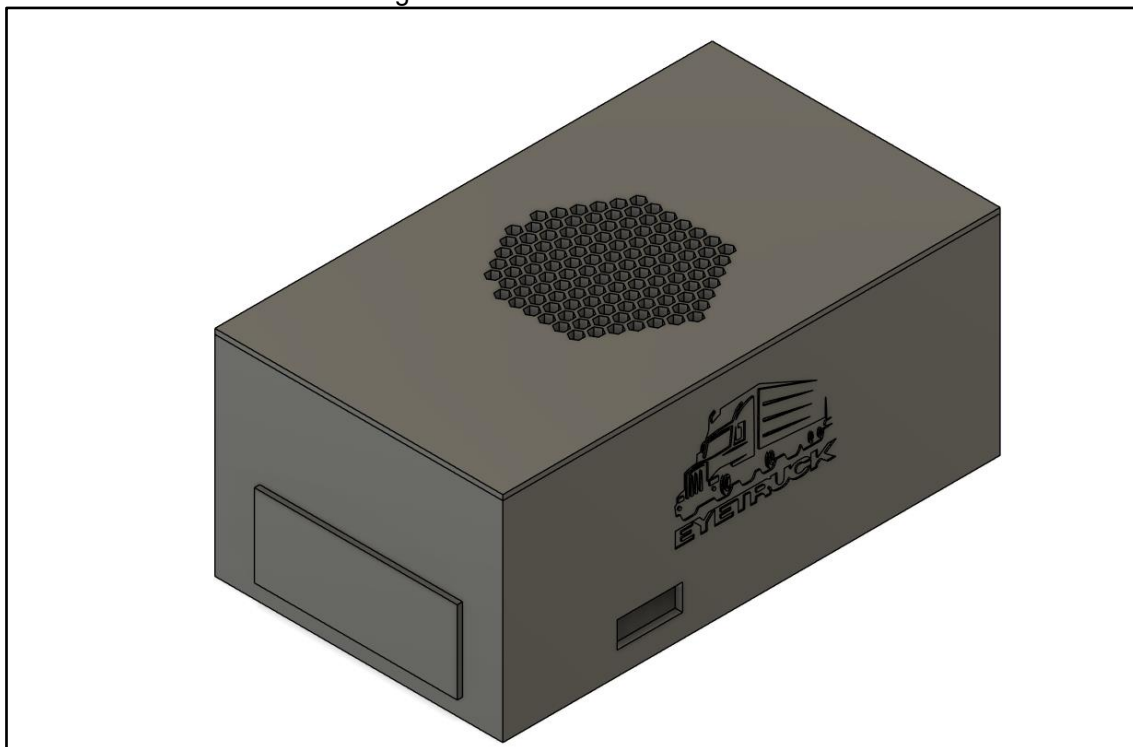
3.7 Desenvolvimento da case

Após a conexão dos componentes elétricos e seus respectivos testes, iniciou a etapa da modelagem 3D, uma fase essencial para a integridade dos equipamentos, fornecendo uma maior segurança para as peças, e também para aprimorar a estética do projeto.

Nesta etapa foi utilizada a ferramenta Autodesk Fusion360, uma ferramenta profissional que unifica, em somente um ambiente, funcionalidades para projetar, simular, analisar e fabricar peças e sistemas mecânicos, provendo maior segurança na visualização antes da construção física.

No Desenvolvimento da case, buscamos um visual dinâmico que passasse uma impressão de versátil e intuitivo para o uso diário do caminhoneiro, essa ideia possibilitou que a case fosse um simples retângulo com encaixes sob medida para os equipamentos, incluindo detalhes cruciais como a logo do Eyetruck da case, o código de acesso único do dispositivo, pequenos orifícios na parte da tampa para fins de ventilação dos equipamentos, uma tampa lateral para o motorista poder retirá-la para carregar o dispositivo e por fim alguns suportes no interior para manter o carregador portátil estabilizado.

Figura 74 - Visão de cima da case



Fonte: Autoria Própria, 2025.

4 CONSIDERAÇÕES FINAIS

A incidência de acidentes envolvendo caminhões por excesso de altura representa um risco significativo para a segurança viária, tanto pelo porte desses veículos quanto pelo potencial de danos materiais e humanos envolvidos. Nesse contexto, este trabalho apresentou o EyeTruck, um sistema de prevenção de colisões de veículos de carga com viadutos e pontes que integra visão computacional, reconhecimento óptico de caracteres, aprendizado de máquina, hardware embarcado e um aplicativo móvel conectado a serviços em nuvem.

O sistema desenvolvido combina um aplicativo em que o motorista cadastra a altura do caminhão, um banco de dados em nuvem utilizando Firebase e um Raspberry Pi, responsável por reconhecer a placa R-15, extrair o valor de altura máxima permitida e compará-lo com a altura registrada no banco de dados. Nos testes realizados em ambiente controlado, utilizando uma placa R-15 impressa, o modelo YOLO foi capaz de identificar corretamente a sinalização com alta confiança e, em seguida, o módulo de OCR extraiu o valor numérico da placa. Quando a altura cadastrada do caminhão excedeu o limite indicado, o sistema emitiu um alerta visual ao motorista, evidenciando a viabilidade funcional do protótipo como sistema de apoio à decisão.

Apesar dos resultados alcançados, algumas limitações foram identificadas. A quantidade de dados utilizada no treinamento do modelo de detecção é consideravelmente pequena, o que restringe a capacidade de detecção em cenários mais complexos, como diferentes condições de iluminação, distâncias e ângulos de captura da placa. A precisão do OCR também pode ser afetada por reflexos, desgaste da sinalização e baixa qualidade da imagem, comprometendo a confiança da leitura da altura em ambientes reais. Somando isso ao fato de que os testes foram conduzidos apenas em ambiente controlado, o que impede, neste momento, afirmar que o uso do EyeTruck em larga escala reduziria de forma comprovada o número de colisões em viadutos e pontes.

Em trabalhos futuros, destaca-se a necessidade de ampliar a base de dados com imagens de placas R-15 e de outras sinalizações relacionadas à limitação de altura, capturadas em diferentes contextos e condições climáticas, de modo a aumentar a robustez do modelo de detecção e a precisão do OCR. Também se propõe

a realização de testes em vias urbanas e rodovias, com acompanhamento dos resultados ao longo do tempo.

Em síntese, o EyeTruck configura-se como uma solução tecnológica promissora para apoiar a prevenção de colisões de caminhões por excesso de altura, contribuindo potencialmente para a redução de acidentes e de custos associados a danos estruturais e materiais. Embora ainda se encontre em estágio de protótipo, o sistema demonstra potencial para ser aperfeiçoado e, futuramente, aplicado em contextos reais, reforçando sua relevância prática e social.

REFERÊNCIAS

ALBINO, Igor Cacerez. **Dominando o CSS: um guia completo para desenvolvedores web**. São Paulo: Clube de Autores, 2023.

ALMEIDA, F. C.; SOUSA, A. L. de; PEREIRA, M. G. A. **Avaliação de técnicas de aprendizado de máquina para classificação de dados meteorológicos em sistemas agrícolas**. Revista Brasileira de Ciência do Solo, v. 43, p. 1-10, 2019. Disponível em: <https://apct.sede.embrapa.br/cct/article/view/26381/14242>.

ARAÚJO, Aline Moura. **Deteção e Destaque em Vídeo de Objetos utilizando YOLO**. 2022. 69 p. Dissertação (Programa de Pós-Graduação em Informática) - Universidade Federal da Paraíba, João Pessoa, 2022.

BARELLI, Felipe. **Introdução à Visão Computacional: Uma abordagem prática com Python e OpenCV**. 1. São Paulo: Casa do Código, 2018.

BOOCH, Grady. **UML: Guia do Usuário**. 1. São Paulo: Campus, 2006.

BORGES, Luiz Eduardo. **Python para Desenvolvedores: Aborda Python 3.3**. 1. ed. São Paulo: Novatec Editora Ltda, 2014.

CARRION, Patricia; QUARESMA, Manuela. Internet da Coisas (IoT): Definições e aplicabilidade aos usuários finais. **Human Factors in Design**, Florianópolis, v. 8, n. 15, p. 049–066, 2019. DOI: 10.5965/2316796308152019049. Disponível em: <https://www.revistas.udesc.br/index.php/hfd/article/view/2316796308152019049>.

CARVALHO, Vinícius. **MySQL: Comece com o principal banco de dados open source do mercado**. São Paulo: Casa do Código, 2015.

COELHO, Pedro Citadin; ANTUNES, Luciano. **Aplicação móvel voltada ao compartilhamento de informações acerca de praias integrando a ferramenta Google Maps e a Biblioteca React Native**. 2023. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) – Universidade do Extremo Sul Catarinense, UNESC, Criciúma, 2023.

CORDEIRO, Michelle Sousa. **Alfabetização assistida por visão computacional: detecção de cartões e reconhecimento de letras**. 2024. Trabalho de Conclusão de Curso (Bacharelado em Ciências e Tecnologia, com ênfase em

Computação Aplicada) – Universidade Federal do Rio Grande do Norte, Escola de Ciências e Tecnologia, Natal, RN, 2024.

CORREA, Beatriz Pinheiro. **Introdução à modelagem 3D com Blender: um estudo de caso aplicado para a facilitação do aprendizado de criação de objetos 3D, voltado para alunos de graduação**. 2023. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Computação) – Universidade Federal do Pará, Campus Castanhal, Castanhal, 2023. Disponível em: <https://bdm.ufpa.br/handle/prefix/7667>. Acesso em: 29 out. 2025.

COSTA, Rayara Pinto; FERNANDES, Luara Lopes de Araujo; MUTA, Luís Filipe; ISATTO, Eduardo Luis; COSTA, Dayana Bastos. **Modelagem 3D de edificação gerada por fotogrametria com uso de Veículos Aéreos Não Tripulados (VANT)**. Ambiente Construído, Porto Alegre, v. 24, e131377, jan./dez. 2024. DOI: 10.1590/s1678-86212024000100737. Disponível em: <https://www.scielo.br/j/ac/a/T4WNQLHsQZz3Ym93thwtLRG/>. Acesso em: 29 out. 2025.

EBERMAM, Elivelto; PESENTE, Guilherme Moraes; RIOS, Renan Osório; PULINI, Igor Carlos. **Programação para leigos com Raspberry Pi**. Vitória: Edifes; João Pessoa: Editora IFPB, 2017.

EIS, Diego. **Guia Front-End: O caminho das pedras para ser um dev Front-End**. São Paulo: Casa do Código, 2015

ESCUDELARIO, Bruno; PINHO, Diego. **React Native: Desenvolvimento de aplicativos mobile com React**. São Paulo: Casa do Código, 2020.

FALCÃO, Filipe Dourado. **Desenvolvimento do aplicativo Turistando Beberibe utilizando React Native**. 2022. Trabalho de Conclusão de Curso (Bacharelado em Sistemas e Mídias Digitais) – Universidade Federal do Ceará, Instituto Universidade Virtual (UFC Virtual), Fortaleza, 2022.

FIGMA. **O que é o Figma?**. Disponível em: <https://help.figma.com/hc/pt-br>. Acesso em: 09 ago. 2025.

FLANAGAN, David. **JavaScript: O Guia Definitivo**. Porto Alegre: Bookman, 2012.

FLORENCIO, Felipe de Almeida. **Estudo comparativo do desempenho de bibliotecas para redes neurais convolucionais em diferentes microarquiteturas**

de GPU. 2020. Dissertação (Mestrado em Ciência da Computação) – Universidade Federal de Sergipe, São Cristóvão, SE, 2020.

FOLHA DE S. PAULO. **Quinze caminhões ficam presos em viadutos todo mês em SP e abalam estruturas**. Folha de S.Paulo, 2019. Disponível em: <https://www.folha.uol.com.br/cotidiano/2019/02/quinze-caminhoes-ficam-presos-em-viadutos-todo-mes-em-sp-e-abalam-estruturas>. Acesso em: 07 abr. 2025.

FOWLER, Martin. **UML Essencial: Um Breve Guia para a Linguagem-padrão de Modelagem de Objetos**. 3. Porto Alegre: Bookman, 2005.

FREITAS, Pabricao Bonfim. **Projeto e desenvolvimento de um sistema de gerenciamento de chaves (SGC) físicas convencionais para instituições de ensino**. 2023. Trabalho de Conclusão de Curso (graduação em Tecnologia em Análise e Desenvolvimento de Sistemas) - Instituto Federal de Educação, Ciência e Tecnologia da Bahia Campus Valença, Valença, 2023.

GALVÃO, Pedro Sereno. **Comprehensive Repository Analysis of Mobile Projects Built with React Native**. 2018. 28 f. Dissertação (Programa de Graduação em Ciência da Computação) – Universidade Federal de Pernambuco, Recife, 2018.

GÉRON, Aurélien. **Mãos à obra: aprendizado de máquina com Scikit-Learn, Keras e TensorFlow: conceitos, ferramentas e técnicas para a construção de sistemas inteligentes**. 2. ed. Rio de Janeiro: Alta Books, 2020.

GONÇALVES, Eduardo. **SQL: Uma abordagem para bancos de dados Oracle**. 1. ed. São Paulo: Casa do Código, 2014.

GOOGLE. **Firestore**. Disponível em: <https://firebase.google.com/>. Acesso em: 11 ago. 2025.

GUEDES, Gilleanes. **UML 2: Uma Abordagem Prática**. 3. São Paulo: Novatec Editora Ltda, 2018.

IEPSEN, Edécio Fernando. **Lógica de Programação e Algoritmos com Javascript**. São Paulo: Novatec, 2018.

IHRIG, Colin J. **Pro Node.js Para Desenvolvedores**. 1. ed. Rio de Janeiro: Ciência Moderna, 2020.

KOVÁCS, Zsolt. **Redes Neurais Artificiais: Fundamentos e Aplicações**. 4. ed. São Paulo: Editora Livraria da Física, 2023.

LEOCÁDIO, Rodolfo R. V. et al. **Detecção de Abelhas Nativas em Colmeias em Campo Utilizando Visão Computacional**. In: WORKSHOP DE COMPUTAÇÃO APLICADA À GESTÃO DO MEIO AMBIENTE E RECURSOS NATURAIS (WCAMA), 12., 2021, Porto Alegre: Sociedade Brasileira de Computação, 2021. p. 59-68.

MACHADO, Kheronn Khennedy. **Angular 11 e Firebase: Construindo uma aplicação integrada com a plataforma do Google**. São Paulo. Casa do Código, 2021.

MAGRANI, Eduardo. **A internet das coisas**. Rio de Janeiro: FGV Editora, 2018.

MARENGONI, M.; STRINGHINI, S. Tutorial: Introdução à Visão Computacional usando OpenCV. **Revista de Informática Teórica e Aplicada**, [S. l.], v. 16, n. 1, p. 125–160, 2010. DOI: 10.22456/2175-2745.11477. Disponível em: https://seer.ufrgs.br/index.php/rita/article/view/rita_v16_n1_p125. Acesso em: 27 maio. 2025.

MATIOLA, Guilherme Schlindwein. **Aplicativo móvel para controle do processamento de dados de amostras de solo com integração da tecnologia RFId**. 2023. Trabalho de Conclusão de Curso (Graduação em Engenharia de Controle e Automação) - Universidade Federal de Santa Catarina Campus Blumenau, Blumenau, 2023.

MATTHES, Eric. **Curso Intensivo de Python: uma Introdução Prática e Baseada em Projetos à Programação**. São Paulo: Novatec Editora Ltda, 2016.

MCKINNEY, Wes. **Python para Análise de Dados: Tratamento de dados com pandas, NumPy e Jupyter**. 3. ed. São Paulo: Novatec Editora Ltda, 2023.

MENEZES, Nilo Ney Coutinho. **Introdução a programação com Python**. São Paulo: Novatec, p. 13, 2010.

MILANO, Danilo de; HONORATO, Luciano Barrozo. **Visão Computacional**. UNICAMP Universidade Estadual de Campinas FT Faculdade de Tecnologia, 2014.

OLIVEIRA, Cláudio Luis Vieira; NABARRO, Cristina Becker Matos; ZANETTI, Humberto Augusto Piovesana. **Raspberry Pi Descomplicado**. 1. ed. São Paulo: Editora Érica, 2018. 224 p.

OLIVEIRA, Sérgio De. **Internet Das Coisas Com Esp8266, Arduino E Raspberry Pi**. 2. ed. São Paulo: Novatec, 2021.

OSÓRIO, Abrão Júnior. **Interface gráfica para busca de profissionais criativos**. 2020. 101 f. Trabalho de Conclusão de Curso (Graduação em Design) – Universidade Federal de Uberlândia, Uberlândia, 2021.

PACHECO, César Augusto Rodrigues. **Deep Learning: Conceitos e Utilização nas Diversas Áreas do Conhecimento**. 2018. 32 f. Trabalho de Conclusão de Curso (Engenharia de Computação) – Centro Universitário de Anápolis, 2018.

PAMCARY. Acidentes com caminhões são quase 5 vezes que furto de cargas. **Pamcary**, 2025. Disponível em: <https://www1.gps-pamcary.com.br/acidentes-com-caminhoes-sao-quase-5-vezes-que-furto-de-cargas/>. Acesso em: 20 out. 2025.

PEREIRA, Caio Ribeiro. **Aplicações web real-time com Node.js**. São Paulo: Casa do Código, 2014.

PEREIRA, Matheus de Matos. **Aprendizado Profundo: Redes LSTM**. 2017. 40 f. Trabalho de Conclusão de Curso (Sistemas de Informação) – Universidade Federal da Grande Dourados, 2017.

R7. **A cada dois dias, um caminhão entala em viadutos de São Paulo**. R7, 2019. Disponível em: <https://noticias.r7.com/sao-paulo/a-cada-dois-dias-um-caminhao-entala-em-viadutos-de-sao-paulo-30102019/>. Acesso em: 10 set. 2025.

ROSA, João Vitor de Moura. **Desenvolvimento de uma aplicação web para simulados preparatórios de vestibular com enfoque no ENEM**. 2023. Trabalho de Conclusão de Curso (Sistemas de Informação) – Universidade Federal de Ouro Preto, João Monlevade, 2023.

SANTOS, José A. dos; SILVA, Felipe L. da. **Um estudo sobre a aplicação de algoritmos de aprendizado de máquina para a detecção de fraudes em sistemas bancários**. Simpósio Brasileiro de Sistemas de Informação (SBSI), v. 1, p. 100-110, 2019. Disponível em: <https://sol.sbc.org.br/index.php/sbsi/article/view/5818/5716>

SANTOS, Leanderson da Silva. **Gerenciador de Projetos da Agenda 2030 da Organização das Nações Unidas (GPA2030)**. 2023. Trabalho de Conclusão de Curso (Tecnologia em Análise e Desenvolvimento de Sistemas) – Instituto Federal de Educação, Ciências e Tecnologia do Amazonas, Campus Manaus Centro, Manaus, 2023.

SANTOS, Sandro. **Introdução à IoT: desvendando a Internet das Coisas**. Santa Catarina: Clube de Autores, 2019

SETTE, Guilherme; CARVALHO, Luiz; BASILE, Antonio. **Reconhecimento facial pela região dos olhos utilizando OpenCV**. 2021. Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) – Faculdade de Computação e Informática, Universidade Presbiteriana Mackenzie, São Paulo, 2021.

SILVA, Daniela Rocha; SOBRAL, Luis Felipe Bentin. **Um Estudo em Larga Escala sobre a Estrutura do Código-fonte de Pacotes JavaScript**. 2017. Graduação (Bacharel em Sistemas de Informação)- UNIRIO, 2017.

SILVA, F. A. S.; IGLESIAS, W. A. N.; BOER, M. T. **Utilização da prototipagem e criação do front-end pelo Figma para um aplicativo móvel de pets**. 2024. Artigo de Graduação (Tecnologia em Sistemas para Internet) – Faculdade de Tecnologia Prof. José Camargo, Jales, 2024.

SILVA, Gabriel de Oliveira; DIAS, Wanderson Roger Azevedo; ESCUDERO, Danilo Pereira. **Análise do Desempenho Computacional da Raspberry Pi Executando o Benchmark SysBench**. Simpósio em Sistemas Computacionais de Alto Desempenho (SSCAD). SBC, 2022.

SILVA, João Victor Negreiros da; CASTRO, Jorge Vinícius Santos. **DOCKERYZER - Desenvolvimento de uma aplicação de linha de comando para criação automática de dockerfiles em projetos NODE.JS com uso de inteligência artificial**. 2025. 28 f. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Computação) – Instituto Federal de Educação, Ciência e Tecnologia da Paraíba, Campina Grande, 2025.

SILVA, Mauricio Samy. **Construindo Sites com CSS e (x)HTML**. 1. ed. São Paulo: Novatec, 2007.

SILVA, Tayná Cristina Sousa. **Detecção e classificação de derramamento de óleo na superfície oceânica baseada em aprendizagem profunda via algoritmo**

YOLO. 2024. 75 f. Dissertação (Programa de Pós-Graduação em Engenharia de Eletricidade/CCET) - Universidade Federal do Maranhão, São Luís, 2024.

SOUZA, Caio Adriano Xavier de. **Previsão da demanda de materiais de construção utilizando redes neurais artificiais: uma comparação entre TensorFlow e PyTorch**. 2024. Monografia (Graduação em Engenharia de Produção) - Universidade Federal de Ouro Preto, Escola de Minas, Ouro Preto, MG, 2024.

VALE, Marina Pires de Castro Aguiar; GHISLENI, Camila Vieira; GASPERIN, Fernanda Fontana de; LEAL, Roger Langone; SILVA, Fabio Pinto da; BRUSCATO, Underléa Miotto. **Proposta de código automatizado para conversão de malhas poligonais de formas orgânicas para superfícies NURBS utilizáveis em softwares CAD. Gestão & Tecnologia de Projetos**, São Carlos, v. 17, n. 2, p. 5–17, 2022. DOI: 10.11606/gtp.v17i2.166353. Disponível em: <https://revistas.usp.br/gestaodeprojetos/article/view/166353>. Acesso em: 29 out. 2025.